

# CASINO PLATFORM

## Technical Specification Document

### Platform Core + Game Engines Architecture

Version 1.0 — February 2026

*Target: Cameroon (Afrique francophone)*

| Attribute     | Value                                                  |
|---------------|--------------------------------------------------------|
| Architecture  | Platform Core + Game Engine Plugins (Modular Monolith) |
| Backend Stack | Python 3.12 + FastAPI + PostgreSQL 16 + Redis 7        |
| Frontend      | Unity (WebGL + Mobile)                                 |
| V1 Game       | European & American Roulette                           |
| V2 Games      | Blackjack, Slots, Poker (within 6 months)              |
| Payment       | MTN MoMo + Orange Money via CinetPay                   |
| Currency      | Virtual chips (integer), 3 wallets                     |
| V1 Timeline   | ~14 weeks (3.5 months), 1-2 developers                 |

# Table of Contents

*(Right-click → Update Field in Word to refresh)*

# Section 1 — Introduction to Roulette (Complete Domain Reference)

## 1.1 Purpose of This Section

This section serves as the **authoritative domain reference** for the entire development team. Every rule, number, color, bet type, and payout described here is the single source of truth for backend logic implementation. Any ambiguity in later sections should be resolved by referring back to this section.

## 1.2 Roulette Variants Supported

The system supports **two variants** from V1:

### 1.2.1 European Roulette

| Property          | Value           |
|-------------------|-----------------|
| Total pockets     | 37              |
| Number range      | 0 – 36          |
| Zero pockets      | Single zero (0) |
| House edge        | 2.70%           |
| Identifier (code) | EUROPEAN        |

### 1.2.2 American Roulette

| Property          | Value         |
|-------------------|---------------|
| Total pockets     | 38            |
| Number range      | 0, 00, 1 – 36 |
| Zero pockets      | 0 and 00      |
| House edge        | 5.26%         |
| Identifier (code) | AMERICAN      |

> **Implementation note:** The double zero (00) is represented internally as the integer **-1** (or a dedicated constant **DOUBLE\_ZERO = -1**) to maintain all numbers as integers while distinguishing it from the single zero (0). This choice avoids string parsing in all bet resolution logic. Alternative: use **37** as the internal representation — team should decide and document. Both approaches are discussed in Section 5.

## 1.3 Wheel Layout (Physical Order)

The physical order of numbers on the wheel is **not sequential**. This order matters for:

- Wheel animation in Unity (mapping spin angle to result)
- "Voisins" (neighbors) bets

- Call bets / Announced bets

### 1.3.1 European Wheel (clockwise, starting from 0)

**37 pockets — index 0 to 36:**

```
Position: 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36
Number:    0 32 15 19  4 21  2 25 17 34  6 27 13 36 11 30  8 23 10  5 24 16 33  1 20 14
31  9 22 18 29  7 28 12 35  3 26
```

**As array (for direct implementation):**

```
python
EUROPEAN_WHEEL_ORDER = [
    0, 32, 15, 19, 4, 21, 2, 25, 17, 34,
    6, 27, 13, 36, 11, 30, 8, 23, 10, 5,
    24, 16, 33, 1, 20, 14, 31, 9, 22, 18,
    29, 7, 28, 12, 35, 3, 26
]
```

### 1.3.2 American Wheel (clockwise, starting from 0)

**38 pockets — index 0 to 37:**

```
Position: 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36 37
Number:    0 28  9 26 30 11  7 20 32 17  5 22 34 15  3 24 36 13  1 00 27 10 25 29 12  8
19 31 18  6 21 33 16  4 23 35 14  2
```

**As array (using -1 for 00):**

```
python
AMERICAN_WHEEL_ORDER = [
    0, 28, 9, 26, 30, 11, 7, 20, 32, 17,
    5, 22, 34, 15, 3, 24, 36, 13, 1, -1,
    27, 10, 25, 29, 12, 8, 19, 31, 18, 6,
    21, 33, 16, 4, 23, 35, 14, 2
]
# Note: -1 represents "00" (double zero)
```

> **Verification:** Both arrays contain exactly the correct count (37 and 38) with no duplicates. The backend must validate this at startup with an assertion.

## 1.4 Number Colors

### 1.4.1 Color Mapping

Colors are **identical** for both European and American roulette (the 00 is simply added as green).

| Color | Numbers                            | Count  |
|-------|------------------------------------|--------|
| GREEN | 0 (and 00 for American)            | 1 or 2 |
| RED   | 1, 3, 5, 7, 9, 12, 14, 16, 18, 19, | 18     |

| Color | Numbers                                                                  | Count |
|-------|--------------------------------------------------------------------------|-------|
|       | 21, 23, 25, 27, 30, 32, 34, 36                                           |       |
| BLACK | 2, 4, 6, 8, 10, 11, 13, 15, 17,<br>20, 22, 24, 26, 28, 29, 31, 33,<br>35 | 18    |

As sets (for direct implementation):

```
python
RED_NUMBERS = {1, 3, 5, 7, 9, 12, 14, 16, 18, 19, 21, 23, 25, 27, 30, 32, 34, 36}
BLACK_NUMBERS = {2, 4, 6, 8, 10, 11, 13, 15, 17, 20, 22, 24, 26, 28, 29, 31, 33, 35}
GREEN_NUMBERS_EUROPEAN = {0}
GREEN_NUMBERS_AMERICAN = {0, -1} # -1 = 00
```

### 1.4.2 Color Determination Function

```
python
def get_color(number: int, variant: str = "EUROPEAN") -> str:
    if number == 0 or (number == -1 and variant == "AMERICAN"):
        return "GREEN"
    if number in RED_NUMBERS:
        return "RED"
    if number in BLACK_NUMBERS:
        return "BLACK"
    raise ValueError(f"Invalid number: {number}")
```

### 1.4.3 Common Misconception

The color pattern is **NOT** strictly alternating on the table layout. It alternates on the **wheel**, but on the betting table (grid layout), the pattern appears irregular. The mapping above is the definitive reference — it must be hardcoded, not computed.

## 1.5 Table Layout (Betting Grid)

The betting table is a 3-column × 12-row grid plus special areas:

|   |              |      |   |              |       |    |              |       |    |    |    |    |            |
|---|--------------|------|---|--------------|-------|----|--------------|-------|----|----|----|----|------------|
| 0 | 3            | 6    | 9 | 12           | 15    | 18 | 21           | 24    | 27 | 30 | 33 | 36 | ← Column 3 |
|   | 2            | 5    | 8 | 11           | 14    | 17 | 20           | 23    | 26 | 29 | 32 | 35 | ← Column 2 |
|   | 1            | 4    | 7 | 10           | 13    | 16 | 19           | 22    | 25 | 28 | 31 | 34 | ← Column 1 |
|   | 1st 12 (P12) |      |   | 2nd 12 (M12) |       |    | 3rd 12 (D12) |       |    |    |    |    |            |
|   | 1-18         | EVEN |   | RED          | BLACK |    | ODD          | 19-36 |    |    |    |    |            |

For American roulette, the zero area becomes:

|    |
|----|
| 00 |
|----|



### 1.5.1 Grid Coordinates (for bet placement validation)

The table grid uses a coordinate system where each number occupies a cell:

| Number | Row | Column | Formula                                          |
|--------|-----|--------|--------------------------------------------------|
| 1      | 1   | 1      | row = number, col = 1                            |
| 2      | 1   | 2      | row = ((n-1) // 3) + 1,<br>col = ((n-1) % 3) + 1 |
| ...    | ... | ...    | Generalized below                                |

**Grid position formula (for numbers 1–36):**

```
python
def get_grid_position(number: int) -> tuple[int, int]:
    """Returns (row, column) where row=1..12, column=1..3"""
    assert 1 <= number <= 36
    row = (number - 1) // 3 + 1    # 1-indexed row (1 to 12)
    column = (number - 1) % 3 + 1 # 1-indexed column (1 to 3)
    return (row, column)
```

This formula is critical for validating Split, Corner, Street, and Six Line bets (see Section 1.7).

## 1.6 Bet Types — Complete Reference

### 1.6.1 Inside Bets

Inside bets are placed directly on numbers or on the lines between numbers on the grid.

#### Straight Up (Plein)

| Property        | Value            |
|-----------------|------------------|
| Numbers covered | 1                |
| Payout          | 35:1             |
| Valid numbers   | 0, 00 (-1), 1–36 |
| Code            | STRAIGHT_UP      |

**Description:** Bet on a single number. The chip is placed in the center of the number's cell.

#### Split (Cheval)

| Property        | Value                     |
|-----------------|---------------------------|
| Numbers covered | 2                         |
| Payout          | 17:1                      |
| Valid pairs     | See adjacency rules below |

| Property | Value        |
|----------|--------------|
| Code     | <b>SPLIT</b> |

**Description:** Bet on two adjacent numbers. The chip is placed on the line between them.

**Adjacency rules — two numbers can form a Split if:**

1. **Horizontal neighbors on the grid:** Same row, adjacent columns → numbers differ by exactly 1, AND `ceil(n1/3) == ceil(n2/3)`
2. **Vertical neighbors on the grid:** Same column, adjacent rows → numbers differ by exactly 3
3. **Zero splits (European):** 0-1, 0-2, 0-3
4. **Zero splits (American):** 0-1, 0-2, 0-3, 00-2, 00-3, and 0-00

**Complete validation:**

```
python
def is_valid_split(n1: int, n2: int, variant: str = "EUROPEAN") -> bool:
    a, b = sorted([n1, n2])

    # Zero-related splits
    if variant == "AMERICAN" and a == -1: # 00
        return b in {0, 2, 3}
    if a == 0:
        return b in {1, 2, 3}

    # Both numbers must be 1-36
    if not (1 <= a <= 36 and 1 <= b <= 36):
        return False

    # Vertical neighbor (same column, adjacent rows)
    if b - a == 3:
        return True

    # Horizontal neighbor (same row, adjacent columns)
    if b - a == 1 and (a - 1) // 3 == (b - 1) // 3:
        return True

    return False
```

### Street (Transversale Pleine)

| Property        | Value               |
|-----------------|---------------------|
| Numbers covered | 3                   |
| Payout          | 11:1                |
| Valid streets   | 12 possible streets |
| Code            | <b>STREET</b>       |

**Description:** Bet on a row of three numbers. The chip is placed on the outer edge of the row.

**All valid Streets:**

|                        |                         |
|------------------------|-------------------------|
| Street 1: [1, 2, 3]    | Street 7: [19, 20, 21]  |
| Street 2: [4, 5, 6]    | Street 8: [22, 23, 24]  |
| Street 3: [7, 8, 9]    | Street 9: [25, 26, 27]  |
| Street 4: [10, 11, 12] | Street 10: [28, 29, 30] |
| Street 5: [13, 14, 15] | Street 11: [31, 32, 33] |
| Street 6: [16, 17, 18] | Street 12: [34, 35, 36] |

**Validation:** A street is valid if the lowest number  $n$  satisfies  $n \geq 1$  and  $(n - 1) \% 3 == 0$ , and the three numbers are  $[n, n+1, n+2]$ .

### Trio

| Property        | Value                             |
|-----------------|-----------------------------------|
| Numbers covered | 3                                 |
| Payout          | 11:1                              |
| Valid trios     | European: {0,1,2}, {0,2,3}        |
|                 | American: adds {0,00,2}, {00,2,3} |
| Code            | TRIO                              |

**Description:** A three-number bet that includes zero. The chip is placed at the intersection of the three numbers.

### Corner / Square (Carré)

| Property        | Value               |
|-----------------|---------------------|
| Numbers covered | 4                   |
| Payout          | 8:1                 |
| Valid corners   | 22 possible corners |
| Code            | CORNER              |

**Description:** Bet on four numbers forming a square. The chip is placed at the intersection of the four cells.

**Validation:** A Corner is defined by its **top-left number**  $n$ . It covers  $[n, n+1, n+3, n+4]$  and is valid if:

- $1 \leq n \leq 32$
- $(n - 1) \% 3 \neq 2$  (i.e.,  $n$  is NOT in column 3, otherwise  $n+1$  wraps to the next row)

python

```
def is_valid_corner(numbers: list[int]) -> bool:
    s = sorted(numbers)
    if len(s) != 4:
        return False
    n = s[0]
    if not (1 <= n <= 32):
        return False
    if (n - 1) % 3 == 2: # n is in column 3
        return False
```



```
return s == [n, n + 1, n + 3, n + 4]
```

**All valid Corners (top-left number):**

```
1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17,
19, 20, 22, 23, 25, 26, 28, 29, 31, 32
```

**Six Line (Sixain / Transversale Simple)**

| Property        | Value       |
|-----------------|-------------|
| Numbers covered | 6           |
| Payout          | 5:1         |
| Valid six lines | 11 possible |
| Code            | SIX_LINE    |

**Description:** Bet on two adjacent Streets (6 numbers). The chip is placed at the intersection of the outer edges of the two rows.

**Validation:** Defined by the lower street's starting number  $n$ . Covers  $[n, n+1, n+2, n+3, n+4, n+5]$  where  $(n-1) \% 3 == 0$  and  $n \leq 31$ .

**All valid Six Lines (starting number):**

```
1, 4, 7, 10, 13, 16, 19, 22, 25, 28, 31
```

**First Four / Basket (American only: Top Line)**

| Property        | Value                                             |
|-----------------|---------------------------------------------------|
| Numbers covered | European: 4 (0,1,2,3)<br>American: 5 (0,00,1,2,3) |
| Payout          | European 4-number: 8:1<br>American 5-number: 6:1  |
| Code            | FIRST_FOUR / TOP_LINE                             |

**Description:**

- **European (First Four):** Covers 0, 1, 2, 3. Payout is 8:1. Chip placed on the corner of 0/1 and 0/3.
- **American (Top Line / Basket):** Covers 0, 00, 1, 2, 3. Payout is 6:1. This is the **worst bet in roulette** (house edge 7.89%). Chip placed on the corner of 0, 00, and 2.

**1.6.2 Outside Bets**

Outside bets cover larger groups of numbers. **Important: all outside bets LOSE when the result is 0 or 00.** (Exception: special rules like La Partage — see Section 1.8).

**Red / Black (Rouge / Noir)**

| Property        | Value       |
|-----------------|-------------|
| Numbers covered | 18          |
| Payout          | 1:1         |
| Loses on        | 0, 00       |
| Code            | RED / BLACK |

**Even / Odd (Pair / Impair)**

| Property        | Value      |
|-----------------|------------|
| Numbers covered | 18         |
| Payout          | 1:1        |
| Loses on        | 0, 00      |
| Code            | EVEN / ODD |

**Note:** 0 and 00 are **neither even nor odd** for betting purposes, even though 0 is mathematically even.

**Low / High (Manque / Passe)**

| Property        | Value                        |
|-----------------|------------------------------|
| Numbers covered | 18                           |
| Payout          | 1:1 (Low: 1–18, High: 19–36) |
| Loses on        | 0, 00                        |
| Code            | LOW / HIGH                   |

**Dozens (Douzaines)**

| Property        | Value                 |
|-----------------|-----------------------|
| Numbers covered | 12                    |
| Payout          | 2:1                   |
| Loses on        | 0, 00                 |
| 1st Dozen (P12) | 1–12 → Code: DOZEN_1  |
| 2nd Dozen (M12) | 13–24 → Code: DOZEN_2 |
| 3rd Dozen (D12) | 25–36 → Code: DOZEN_3 |

**Columns (Colonnes)**

| Property        | Value |
|-----------------|-------|
| Numbers covered | 12    |

| Property | Value                                                  |
|----------|--------------------------------------------------------|
| Payout   | 2:1                                                    |
| Loses on | 0, 00                                                  |
| Column 1 | 1, 4, 7, 10, 13, 16, 19, 22, 25, 28, 31, 34 → COLUMN_1 |
| Column 2 | 2, 5, 8, 11, 14, 17, 20, 23, 26, 29, 32, 35 → COLUMN_2 |
| Column 3 | 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36 → COLUMN_3 |

**Column formula:** Number  $n$  belongs to Column  $((n - 1) \% 3) + 1$ .

### 1.6.3 Call Bets / Announced Bets (Paris Annoncés)

These are predefined bet combinations based on **wheel sectors** (not the table grid). They are traditionally placed by calling them out to the croupier. In our game, they are accessible via a dedicated UI (the "racetrack" overlay on the wheel).

> **Implementation note:** Call bets are **not atomic bet types**. They are macros that decompose into multiple Inside Bets. The backend should expand them into their constituent bets at placement time.

#### Voisins du Zéro (Neighbors of Zero) — European only

**Covers 17 numbers** surrounding 0 on the European wheel:

22, 18, 29, 7, 28, 12, 35, 3, 26, 0, 32, 15, 19, 4, 21, 2, 25

**Decomposition (9 chips):**

| Chips | Bet Type | Numbers        |
|-------|----------|----------------|
| 2     | TRIO     | 0, 2, 3        |
| 1     | SPLIT    | 4, 7           |
| 1     | SPLIT    | 12, 15         |
| 1     | SPLIT    | 18, 21         |
| 1     | SPLIT    | 19, 22         |
| 2     | CORNER   | 25, 26, 28, 29 |
| 1     | SPLIT    | 32, 35         |

**Total cost:** 9 units

#### Tiers du Cylindre (Thirds of the Wheel) — European only

**Covers 12 numbers** on the opposite side of the wheel from zero:

27, 13, 36, 11, 30, 8, 23, 10, 5, 24, 16, 33

**Decomposition (6 chips):**

| Chips | Bet Type | Numbers |
|-------|----------|---------|
| 1     | SPLIT    | 5, 8    |

| Chips | Bet Type | Numbers |
|-------|----------|---------|
| 1     | SPLIT    | 10, 11  |
| 1     | SPLIT    | 13, 16  |
| 1     | SPLIT    | 23, 24  |
| 1     | SPLIT    | 27, 30  |
| 1     | SPLIT    | 33, 36  |

**Total cost:** 6 units

### Orphelins (Orphans) — European only

**Covers 8 numbers** not in Voisins or Tiers:

1, 20, 14, 31, 9, 17, 34, 6

**Decomposition (5 chips) — Orphelins en Plein:**

| Chips | Bet Type    | Numbers |
|-------|-------------|---------|
| 1     | STRAIGHT_UP | 1       |
| 1     | SPLIT       | 6, 9    |
| 1     | SPLIT       | 14, 17  |
| 1     | SPLIT       | 17, 20  |
| 1     | SPLIT       | 31, 34  |

**Note:** Number 17 is covered by two splits, giving it a double chance.

**Total cost:** 5 units

### Neighbors (Voisins d'un numéro)

**Covers 5 numbers** — the selected number and its 2 neighbors on each side on the wheel.

**Example:** "17 and neighbors" on European wheel:

- Wheel order near 17: ... 25, **17**, 34 ... (position 8)
- 2 left: 2, 25
- 2 right: 34, 6
- Result: {2, 25, 17, 34, 6}

**Decomposition:** 5 × Straight Up bets (5 chips total)

**Implementation:**

```
python
def get_neighbors(number: int, count: int, wheel_order: list[int]) -> list[int]:
    """Get `count` neighbors on each side of `number` on the wheel."""
    idx = wheel_order.index(number)
    size = len(wheel_order)
    neighbors = []
    for i in range(-count, count + 1):
        neighbors.append(wheel_order[(idx + i) % size])
```

```
return neighbors
```

> **Note on American variant:** Call bets (Voisins du Zéro, Tiers, Orphelins) are traditionally associated with European roulette only. The "Neighbors" bet can work on both wheels. The backend should enforce variant-specific availability.

### 1.6.4 Complete Bet Type Summary

| Code        | Name              | Covers | Payout | Variant  |
|-------------|-------------------|--------|--------|----------|
| STRAIGHT_UP | Straight Up       | 1      | 35:1   | Both     |
| SPLIT       | Split             | 2      | 17:1   | Both     |
| STREET      | Street            | 3      | 11:1   | Both     |
| TRIO        | Trio              | 3      | 11:1   | Both     |
| CORNER      | Corner            | 4      | 8:1    | Both     |
| FIRST_FOUR  | First Four        | 4      | 8:1    | European |
| TOP_LINE    | Top Line / Basket | 5      | 6:1    | American |
| SIX_LINE    | Six Line          | 6      | 5:1    | Both     |
| COLUMN_1    | Column 1          | 12     | 2:1    | Both     |
| COLUMN_2    | Column 2          | 12     | 2:1    | Both     |
| COLUMN_3    | Column 3          | 12     | 2:1    | Both     |
| DOZEN_1     | First Dozen       | 12     | 2:1    | Both     |
| DOZEN_2     | Second Dozen      | 12     | 2:1    | Both     |
| DOZEN_3     | Third Dozen       | 12     | 2:1    | Both     |
| RED         | Red               | 18     | 1:1    | Both     |
| BLACK       | Black             | 18     | 1:1    | Both     |
| EVEN        | Even              | 18     | 1:1    | Both     |
| ODD         | Odd               | 18     | 1:1    | Both     |
| LOW         | Low (1-18)        | 18     | 1:1    | Both     |
| HIGH        | High (19-36)      | 18     | 1:1    | Both     |

**Call bets (decomposed into above types at placement time):**

| Code              | Name                  | Chips  | Variant       |
|-------------------|-----------------------|--------|---------------|
| CALL_VOISINS_ZERO | Voisins du Zéro       | 9      | European only |
| CALL_TIERS        | Tiers du Cylindre     | 6      | European only |
| CALL_ORPHELINS    | Orphelins             | 5      | European only |
| CALL_NEIGHBORS    | Neighbors ( $\pm N$ ) | $2N+1$ | Both          |

## 1.7 Payout Formula

### 1.7.1 Standard Payout Calculation

When a bet wins:

$$\begin{aligned}\text{Total return} &= \text{Stake} \times (\text{Payout} + 1) \\ \text{Net profit} &= \text{Stake} \times \text{Payout}\end{aligned}$$

**Examples (with a 10-credit stake):**

| Bet Type    | Payout | Total Return | Net Profit |
|-------------|--------|--------------|------------|
| Straight Up | 35:1   | 360          | 350        |
| Split       | 17:1   | 180          | 170        |
| Street      | 11:1   | 120          | 110        |
| Corner      | 8:1    | 90           | 80         |
| Six Line    | 5:1    | 60           | 50         |
| Dozen       | 2:1    | 30           | 20         |
| Red/Black   | 1:1    | 20           | 10         |

### 1.7.2 Generic Payout Formula

For standard roulette, the payout for any bet covering  $n$  numbers on a 37-pocket (European) wheel is:

$$\text{Payout} = (36 / n) - 1$$

This formula yields:

- 1 number:  $(36/1) - 1 = 35$
- 2 numbers:  $(36/2) - 1 = 17$
- 3 numbers:  $(36/3) - 1 = 11$
- etc.

**Exception:** The American Top Line (5 numbers) pays 6:1 instead of the formula's 6.2:1. This is why it has a worse house edge (7.89%).

### 1.7.3 House Edge Calculation

**European:**  $1/37 = 2.70\%$  (the single zero gives the house its edge)

**American:**  $2/38 = 5.26\%$  (two zeros)

**American Top Line:**  $3/38 \approx 7.89\%$  (worst bet)

> **For our social game:** These are informational. We implement standard payouts but don't need to guarantee mathematical house edge since no real money is involved. However, maintaining correct odds keeps the game authentic.

## 1.8 Special Rules (Optional Features)

These rules are **commonly found in European roulette** and can be implemented as optional table settings.

### 1.8.1 La Partage Rule

**Trigger:** The ball lands on 0 AND the player has an even-money bet (Red/Black, Even/Odd, Low/High).

**Effect:** The player loses only **half** their stake. The other half is returned.

**House edge impact:** Reduces European house edge from 2.70% to 1.35% for even-money bets.

**Implementation:**

```
python
if result == 0 and bet.type in EVEN_MONEY_BETS and table.rules.la_partage:
    return bet.amount / 2 # Return half the stake
```

### 1.8.2 En Prison Rule

**Trigger:** Same as La Partage.

**Effect:** The bet is "imprisoned" — left on the table for the next spin. If the imprisoned bet wins on the next spin, the full stake is returned (no profit). If it loses, the stake is lost entirely. If 0 hits again, the bet remains imprisoned (or is lost, depending on house rules).

**Implementation complexity:** Medium — requires tracking imprisoned bets across spins. Needs a new bet status: **IMPRISONED**.

### 1.8.3 Recommendation for V1

| Rule       | V1 Implementation | Reason                                 |
|------------|-------------------|----------------------------------------|
| La Partage | Optional (toggle) | Simple to implement, adds authenticity |
| En Prison  | Post-V1           | Requires cross-spin state management   |

## 1.9 Game Flow

### 1.9.1 Phase Sequence

WAITING → BETTING → BETTING\_CLOSING → SPINNING → RESOLVING → COMPLETED → WAITING

| Phase           | Duration (configurable) | Player Actions Allowed           |
|-----------------|-------------------------|----------------------------------|
| WAITING         | Until game starts       | Join table                       |
| BETTING         | 30–60 seconds           | Place bets, cancel bets          |
| BETTING_CLOSING | 5–10 seconds            | Place bets (last chance warning) |
| SPINNING        | 5–15 seconds            | None (watch animation)           |

| Phase     | Duration (configurable) | Player Actions Allowed        |
|-----------|-------------------------|-------------------------------|
| RESOLVING | 2–5 seconds             | None (results calculated)     |
| COMPLETED | 5–10 seconds            | View results, next round auto |

### 1.9.2 Multiplayer vs Solo Differences

| Aspect         | Solo Mode               | Multiplayer Mode            |
|----------------|-------------------------|-----------------------------|
| Betting timer  | Optional (player-paced) | Mandatory (synchronized)    |
| Spin trigger   | Player clicks "Spin"    | Timer expiration            |
| Wheel result   | Private to player       | Shared across all players   |
| Bet visibility | N/A                     | Optional (see others' bets) |
| Table limits   | Per-player              | Per-table + per-player      |

## 1.10 Glossary of Terms

| Term                | English         | Definition                             |
|---------------------|-----------------|----------------------------------------|
| Plein               | Straight Up     | Single-number bet                      |
| Cheval              | Split           | Two-number bet                         |
| Transversale Pleine | Street          | Three-number row bet                   |
| Carré               | Corner / Square | Four-number bet                        |
| Sixain              | Six Line        | Six-number bet (two rows)              |
| Douzaine            | Dozen           | 12-number group bet                    |
| Colonne             | Column          | 12-number column bet                   |
| Manque              | Low             | Numbers 1–18                           |
| Passe               | High            | Numbers 19–36                          |
| Pair                | Even            | Even numbers                           |
| Impair              | Odd             | Odd numbers                            |
| Rouge               | Red             | Red numbers                            |
| Noir                | Black           | Black numbers                          |
| Croupier            | Dealer          | Person (or system) running the game    |
| Tapis               | Betting Layout  | The surface where bets are placed      |
| Jeton               | Chip            | Virtual currency unit used for betting |
| Faites vos jeux     | Place your bets | Announcement opening the betting phase |
| Rien ne va plus     | No more bets    | Announcement closing the betting phase |



| Term       | English                        | Definition                                            |
|------------|--------------------------------|-------------------------------------------------------|
| Cylindre   | Wheel                          | The spinning wheel with numbered pockets              |
| Voisins    | Neighbors                      | Numbers adjacent on the wheel (not the table)         |
| La Partage | The Sharing                    | Rule returning half stake on zero for even-money bets |
| En Prison  | In Prison                      | Rule imprisoning bet on zero for a second chance      |
| PRNG       | Pseudo-Random Number Generator | Algorithm generating random-like numbers              |
| House Edge | House Edge                     | Statistical advantage of the game operator            |

## 1.11 Mathematical Reference

### 1.11.1 Probability Table (European)

| Bet            | Winning outcomes | Probability | Percentage |
|----------------|------------------|-------------|------------|
| Straight Up    | 1 / 37           | 0.0270      | 2.70%      |
| Split          | 2 / 37           | 0.0541      | 5.41%      |
| Street         | 3 / 37           | 0.0811      | 8.11%      |
| Corner         | 4 / 37           | 0.1081      | 10.81%     |
| Six Line       | 6 / 37           | 0.1622      | 16.22%     |
| Dozen / Column | 12 / 37          | 0.3243      | 32.43%     |
| Even-money     | 18 / 37          | 0.4865      | 48.65%     |

### 1.11.2 Probability Table (American)

| Bet            | Winning outcomes | Probability | Percentage |
|----------------|------------------|-------------|------------|
| Straight Up    | 1 / 38           | 0.0263      | 2.63%      |
| Split          | 2 / 38           | 0.0526      | 5.26%      |
| Street         | 3 / 38           | 0.0789      | 7.89%      |
| Corner         | 4 / 38           | 0.1053      | 10.53%     |
| Top Line       | 5 / 38           | 0.1316      | 13.16%     |
| Six Line       | 6 / 38           | 0.1579      | 15.79%     |
| Dozen / Column | 12 / 38          | 0.3158      | 31.58%     |
| Even-money     | 18 / 38          | 0.4737      | 47.37%     |

### 1.11.3 Expected Value

For any bet on European roulette:

```
EV = (winning_probability × payout) - (losing_probability × stake)
EV = (n/37 × (36/n - 1)) - ((37-n)/37 × 1)
EV = -1/37 ≈ -0.027 per unit staked
```

This means for every 100 credits wagered, a player is expected to lose approximately 2.70 credits over the long run (European) or 5.26 credits (American).

*End of Section 1*

## Section 2 — Player Experience & Functional Requirements

### 2.1 Purpose of This Section

This section defines **what the player sees, does, and experiences** at every stage of interacting with the game. It serves as the bridge between the domain rules (Section 1) and the technical implementation (Sections 3+). Every feature described here maps to concrete user stories, and every user story maps to backend endpoints and frontend screens.

This section is the **primary reference for the Unity frontend team** and for QA test case generation.

### 2.2 Player Journey Overview

#### 2.2.1 High-Level Flow

```
[Download App] → [Register/Login] → [Purchase Chips] → [Select Table] → [Play] → [View Results] → [Continue/Leave]
```

#### 2.2.2 Detailed Player Journey

1. ONBOARDING
  - └─ Download app (WebGL / Mobile)
  - └─ Register (email, phone, or social login)
  - └─ Verify phone number (SMS OTP – required for Cameroon market)
  - └─ Receive welcome bonus (free chips)
  - └─ Tutorial / first game walkthrough (optional)
2. LOBBY
  - └─ View chip balance
  - └─ Purchase chips (MTN Mobile Money / Orange Money)
  - └─ Select game mode (Solo / Multiplayer)
  - └─ Select roulette variant (European / American)
  - └─ Select table (based on min/max bet limits)
  - └─ View personal history & statistics
  - └─ Access settings & profile
3. AT THE TABLE
  - └─ View table layout, wheel, other players (multiplayer)
  - └─ Place bets (drag chips onto table positions)
  - └─ Modify bets (add, remove, undo, double, clear)
  - └─ Confirm readiness (solo: click Spin / multiplayer: wait for timer)
  - └─ Watch wheel animation
  - └─ See result (winning number, color, attributes)
  - └─ See win/loss summary with animation
  - └─ Chips credited/debited with visual feedback
  - └─ Next round or leave table
4. POST-SESSION
  - └─ View session summary (total wagered, won, lost, duration)
  - └─ Return to lobby

— Logout (optional)

## 2.3 Chip Economy (Virtual Currency System)

### 2.3.1 Chip Model

| Property          | Value                                    |
|-------------------|------------------------------------------|
| Currency name     | <b>Chips</b> (internal unit)             |
| Smallest unit     | 1 chip                                   |
| Decimal precision | 0 (integer only — avoids floating point) |
| Display format    | "1,000 chips" or "1K chips"              |
| Storage           | Integer (bigint in DB)                   |

> **Design decision:** Chips are integers. No fractional chips. This eliminates floating-point rounding issues entirely. All calculations (bets, payouts, purchases) operate on integers.

### 2.3.2 Chip Denominations (for betting)

Players select chip values to place on the table:

| Chip Value | Color (visual) | Availability     |
|------------|----------------|------------------|
| 10         | White          | All tables       |
| 50         | Blue           | All tables       |
| 100        | Red            | All tables       |
| 500        | Green          | Medium+ tables   |
| 1,000      | Orange         | Medium+ tables   |
| 5,000      | Purple         | High tables only |
| 10,000     | Black          | VIP tables only  |

> **Note:** These are the denominations of individual chips the player can select. They are NOT currency tiers — a player with 250 chips can place a bet of 250 by stacking chips.

### 2.3.3 Chip Purchase Packages

Players buy chips with real money via Mobile Money.

| Package Name | Price (XAF) | Chips Received | Bonus Chips | Total Chips | Unit Price    |
|--------------|-------------|----------------|-------------|-------------|---------------|
| Starter      | 500 XAF     | 500            | 0           | 500         | 1.00 XAF/chip |
| Popular      | 2,000 XAF   | 2,000          | 200         | 2,200       | 0.91 XAF/chip |
| Best Value   | 5,000 XAF   | 5,000          | 1,000       | 6,000       | 0.83 XAF/chip |
| Premium      | 10,000 XAF  | 10,000         | 3,000       | 13,000      | 0.77 XAF/chip |
| VIP          | 25,000 XAF  | 25,000         | 10,000      | 35,000      | 0.71 XAF/chip |

- > **XAF** = Central African CFA Franc (currency used in Cameroon).
- > Prices are indicative and should be validated with market research.
- > Bonus chips incentivize larger purchases (standard F2P monetization pattern).

#### Implementation notes:

- Packages are **server-defined** (not hardcoded in client) to allow A/B testing and updates without app release
- Each purchase creates an immutable transaction record
- Bonus chips are tracked separately for potential future cashout rules (V2: bonus chips may not be cashout-eligible)

### 2.3.4 Payment Gateways (Cameroon)

| Provider         | Type         | Market Share (approx.) | Integration      |
|------------------|--------------|------------------------|------------------|
| MTN Mobile Money | Mobile Money | ~60% in Cameroon       | MTN MoMo API     |
| Orange Money     | Mobile Money | ~30% in Cameroon       | Orange Money API |

- > **V1 scope:** MTN Mobile Money + Orange Money cover ~90% of the addressable market.
- > **Integration approach:** Use an aggregator (e.g., CinetPay, Flutterwave, or PayDunya) to abstract both providers behind a single API. This avoids maintaining two separate integrations.

### 2.3.5 Purchase Flow

```

Player taps "Buy Chips"
  → Selects package
  → Selects payment method (MTN / Orange)
  → Enters phone number (pre-filled from profile)
  → Confirms purchase
  → Backend initiates payment request to aggregator
  → Aggregator sends USSD push to player's phone
  → Player confirms on phone (enters Mobile Money PIN)
  → Aggregator sends webhook to backend (success/failure)
  → Backend credits chips to player wallet
  → Player sees updated balance with success animation

```

**Critical:** Chips are credited **ONLY** after receiving a confirmed webhook from the payment aggregator. Never credit on client-side confirmation alone.

### 2.3.6 Welcome Bonus

| Trigger                  | Bonus      | Condition              |
|--------------------------|------------|------------------------|
| Account creation         | 500 chips  | One-time, automatic    |
| First purchase completed | +50% bonus | One-time, on first buy |

- > Bonus values are configurable server-side. Welcome bonus enables new players to try the game without purchasing immediately.

### 2.3.7 V2 Cashout Architecture Preparation

Although cashout is NOT in V1, the data model must be **ready** from day one:

- Every chip has an origin: **PURCHASED**, **BONUS**, **WINNINGS**
- Wallet tracks **purchasedBalance**, **bonusBalance**, **winningsBalance** separately
- Only **PURCHASED** and **WINNINGS** chips will be cashout-eligible in V2
- **BONUS** chips must be wagered X times before becoming cashout-eligible (wagering requirement — standard industry practice)
- All transactions are immutable with full audit trail

## 2.4 Table System

### 2.4.1 Table Tiers

| Table Tier | Min Bet | Max Bet | Max Total Bet | Variant  | Mode         |
|------------|---------|---------|---------------|----------|--------------|
| Beginner   | 10      | 500     | 2,000         | European | Solo         |
| Standard   | 50      | 2,000   | 10,000        | Both     | Solo / Multi |
| Premium    | 100     | 5,000   | 25,000        | Both     | Solo / Multi |
| VIP        | 500     | 10,000  | 50,000        | Both     | Multi only   |

#### > Design rationale:

- > - Beginner tables are European-only (lower house edge, simpler for new players)
- > - VIP tables are multiplayer-only (social prestige element)
- > - Table tiers are server-configured, not hardcoded

### 2.4.2 Table Selection UI

#### Information displayed per table:

- Table name (e.g., "Douala Lounge", "Yaoundé VIP")
- Variant (European / American)
- Mode (Solo / Multiplayer)
- Bet limits (min–max)
- Player count (multiplayer: current/max)
- Current game phase (for multiplayer: "Betting Open", "Spinning", etc.)
- Hot/cold numbers (last 10 results)

### 2.4.3 Multiplayer Table Rules

| Property              | Value                         |
|-----------------------|-------------------------------|
| Max players per table | 8 (configurable)              |
| Min players to start  | 1 (game starts even with 1)   |
| Spectator mode        | Yes (watch without betting)   |
| Bet visibility        | Own bets only (others hidden) |

| Property              | Value                            |
|-----------------------|----------------------------------|
| Chat                  | Optional (text only, filtered)   |
| Player join mid-round | Yes, but can only bet next round |
| AFK handling          | Skipped after 3 inactive rounds  |

## 2.5 Betting User Experience

### 2.5.1 Bet Placement

#### How the player places a bet:

1. Select a chip denomination from the chip rack (bottom of screen)
2. Tap/click on a valid position on the betting table
3. Each tap adds one chip of the selected denomination to that position
4. Multiple taps on the same position stack chips (bet accumulates)
5. The bet is registered server-side immediately (optimistic UI with rollback on rejection)

#### Valid tap positions:

| Position          | Bet Type Created | Visual Indicator               |
|-------------------|------------------|--------------------------------|
| Center of number  | Straight Up      | Chip on number                 |
| Line between 2    | Split            | Chip on border line            |
| Start of row      | Street           | Chip on left edge of row       |
| Corner of 4       | Corner           | Chip on intersection           |
| Edge of 2 rows    | Six Line         | Chip on left edge between rows |
| "1st 12" box      | Dozen 1          | Chip on box                    |
| "2nd 12" box      | Dozen 2          | Chip on box                    |
| "3rd 12" box      | Dozen 3          | Chip on box                    |
| Column bottom     | Column 1/2/3     | Chip on column marker          |
| Even-money boxes  | RED/BLACK/etc.   | Chip on box                    |
| Racetrack overlay | Call bets        | Chips on racetrack sector      |

### 2.5.2 Bet Modification

| Action      | How                           | Server Effect                  |
|-------------|-------------------------------|--------------------------------|
| Add chips   | Tap same position again       | New bet or increased amount    |
| Remove last | Right-click / long-press      | Reduces amount by 1 chip       |
| Undo        | "Undo" button                 | Reverts last bet action        |
| Double      | "x2" button                   | Doubles ALL current bets       |
| Clear all   | "Clear" button                | Removes ALL bets               |
| Repeat      | "Repeat" button (after round) | Places same bets as last round |

**Rules:**

- All modifications only allowed during **BETTING** and **BETTING\_CLOSING** phases
- "Double" fails if player has insufficient balance for the doubled total
- "Undo" maintains a stack of actions (client-side), each undo sends a cancel/modify to server
- "Repeat" is client-side convenience — sends individual bet placement requests

**2.5.3 Bet Confirmation & Feedback**

| Event                | Visual Feedback                     | Audio Feedback       |
|----------------------|-------------------------------------|----------------------|
| Bet placed           | Chip appears on table with bounce   | Chip placement sound |
| Bet rejected         | Red flash + error tooltip           | Error sound          |
| Insufficient balance | Balance flashes red + message       | Error sound          |
| Betting closing (5s) | Timer turns red + "Rien ne va plus" | Warning tone         |
| All bets locked      | Table overlay dims slightly         | Lock sound           |

**2.6 Game Round Experience****2.6.1 Round Phases (Player Perspective)****Phase 1 — BETTING (30–60s)**

- Timer visible at top of screen
- Player places/modifies bets
- Multiplayer: see player count, optional chat
- Last 10 results displayed on sidebar

**Phase 2 — BETTING\_CLOSING (5–10s)**

- "Rien ne va plus!" announcement
- Timer turns red, urgency indicator
- Bets still accepted but with visual warning
- "x2" and "Repeat" buttons disabled

**Phase 3 — SPINNING (5–15s)**

- Wheel animation starts
- Ball spins and decelerates
- Camera follows ball (Unity animation)
- No player actions possible
- Suspense building audio

**Phase 4 — RESOLVING (2–5s)**

- Ball lands on number
- Winning number announced with zoom effect
- Number highlighted on table layout



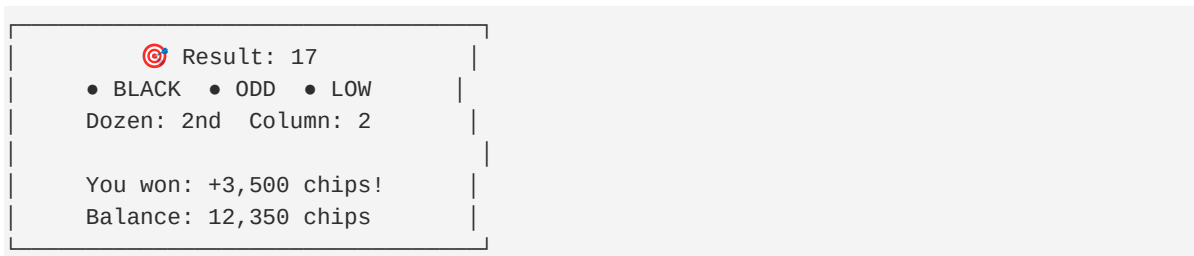
- Winning bet positions glow
- Losing chips swept away (animation)
- Win amount displayed with counter animation

#### Phase 5 — COMPLETED (5–10s)

- Summary panel: number, color, parity, high/low
- Player's win/loss for this round
- Updated balance with animation
- "Repeat Bet" button available
- Countdown to next round (multiplayer)
- "Spin Again" button (solo)

### 2.6.2 Result Display

The winning number display must show all relevant attributes:



## 2.7 Player Account

### 2.7.1 Registration

| Field            | Required | Validation                         |
|------------------|----------|------------------------------------|
| Phone number     | Yes      | Cameroon format (+237), SMS OTP    |
| Display name     | Yes      | 3–20 chars, unique, filtered       |
| Email            | Optional | Email format, confirmation link    |
| Password         | Yes      | Min 8 chars, 1 uppercase, 1 digit  |
| Date of birth    | Yes      | Must be 18+ (gambling regulation)  |
| Terms acceptance | Yes      | Must accept ToS and privacy policy |

> **Phone number as primary ID:** In Cameroon, phone number is more reliable than email for user identity. It also maps directly to Mobile Money accounts for purchases.

## 2.7.2 Authentication

| Method           | V1  | Notes                               |
|------------------|-----|-------------------------------------|
| Phone + Password | Yes | Primary login method                |
| SMS OTP          | Yes | For registration and password reset |
| Social login     | No  | V2 consideration (Google/Facebook)  |
| Biometric        | No  | V2 for mobile app                   |

### Session management:

- JWT tokens (access + refresh)
- Access token: 15-minute expiry
- Refresh token: 30-day expiry
- Single active session per device (configurable)
- Force logout on password change

## 2.7.3 Player Profile

### Visible to player:

- Display name + avatar (preset selection)
- Phone number (masked: +237 6 45)
- Chip balance (total and breakdown by source in V2)
- Account creation date
- Total games played
- Total wagered / won / lost (lifetime)
- Favorite variant (most played)
- Purchase history

### Visible to other players (multiplayer):

- Display name + avatar only
- No financial information exposed

## 2.8 History & Statistics

### 2.8.1 Game History

#### Per-round record (visible to player):

| Field          | Example                      |
|----------------|------------------------------|
| Date/time      | 2026-02-11 14:32 WAT         |
| Table          | Standard - European          |
| Winning number | 17 (Black, Odd, Low)         |
| My bets        | Straight 17 (100), Red (200) |

| Field         | Example                    |
|---------------|----------------------------|
| Total wagered | 300                        |
| Total won     | 3,600 (Straight 17: 3,600) |
| Net result    | +3,300                     |

**Retention policy:** Last 500 rounds or 90 days (whichever comes first). Older records archived.

## 2.8.2 Table Statistics (Live)

Displayed at the table and updated after each round:

| Statistic           | Description                           |
|---------------------|---------------------------------------|
| Last 20 results     | Number sequence with colors           |
| Red/Black ratio     | Percentage over last 100 spins        |
| Even/Odd ratio      | Percentage over last 100 spins        |
| High/Low ratio      | Percentage over last 100 spins        |
| Hot numbers         | Top 5 most frequent (last 100 spins)  |
| Cold numbers        | Top 5 least frequent (last 100 spins) |
| Dozen distribution  | Bar chart over last 100 spins         |
| Column distribution | Bar chart over last 100 spins         |

> **Important disclaimer:** These statistics are for entertainment only. Each spin is independent — past results do not influence future outcomes. This disclaimer must be visible near the statistics panel (responsible gaming).

## 2.9 User Stories

### 2.9.1 Onboarding & Account

| ID    | User Story                                                                                          | Priority |
|-------|-----------------------------------------------------------------------------------------------------|----------|
| US-01 | As a new player, I want to register with my phone number so that I can create an account quickly.   | P0       |
| US-02 | As a new player, I want to receive free welcome chips so that I can try the game before purchasing. | P0       |
| US-03 | As a player, I want to log in with my phone and password so that I can access my account.           | P0       |
| US-04 | As a player, I want to reset my password via SMS OTP so that I can recover my account.              | P0       |
| US-05 | As a player, I want to edit my                                                                      | P1       |

| ID    | User Story                                                                                 | Priority |
|-------|--------------------------------------------------------------------------------------------|----------|
|       | display name and avatar so that I can personalize my identity.                             |          |
| US-06 | As a player, I want to see my complete profile with stats so that I can track my activity. | P1       |

### 2.9.2 Chip Purchase

| ID    | User Story                                                                                                 | Priority |
|-------|------------------------------------------------------------------------------------------------------------|----------|
| US-10 | As a player, I want to buy chips with MTN Mobile Money so that I can play the game.                        | P0       |
| US-11 | As a player, I want to buy chips with Orange Money so that I have an alternative payment option.           | P0       |
| US-12 | As a player, I want to see available packages with prices and bonuses so that I can choose the best value. | P0       |
| US-13 | As a player, I want to receive a confirmation after purchase so that I know the transaction succeeded.     | P0       |
| US-14 | As a player, I want to see my purchase history so that I can track my spending.                            | P1       |
| US-15 | As a player, I want to receive my first-purchase bonus so that I feel rewarded for buying.                 | P1       |

### 2.9.3 Table Selection

| ID    | User Story                                                                                      | Priority |
|-------|-------------------------------------------------------------------------------------------------|----------|
| US-20 | As a player, I want to browse available tables so that I can choose one matching my budget.     | P0       |
| US-21 | As a player, I want to filter tables by variant and mode so that I find the game type I prefer. | P0       |
| US-22 | As a player, I want to see player count on multiplayer                                          | P1       |

| ID    | User Story                                                                                               | Priority |
|-------|----------------------------------------------------------------------------------------------------------|----------|
|       | tables so that I can join active games.                                                                  |          |
| US-23 | As a player, I want to see recent results per table so that I can pick a table based on trends.          | P2       |
| US-24 | As a player, I want to spectate a multiplayer table before joining so that I can observe the game first. | P2       |

## 2.9.4 Gameplay — Betting

| ID    | User Story                                                                                                  | Priority |
|-------|-------------------------------------------------------------------------------------------------------------|----------|
| US-30 | As a player, I want to select chip denominations so that I can control my bet amounts precisely.            | P0       |
| US-31 | As a player, I want to tap on the table to place bets so that betting is intuitive and fast.                | P0       |
| US-32 | As a player, I want to see my total bet amount updating in real-time so that I know how much I'm risking.   | P0       |
| US-33 | As a player, I want to undo my last bet so that I can correct mistakes easily.                              | P0       |
| US-34 | As a player, I want to clear all my bets at once so that I can start over quickly.                          | P0       |
| US-35 | As a player, I want to double all my bets so that I can quickly increase stakes.                            | P1       |
| US-36 | As a player, I want to repeat my previous round's bets so that I can play the same strategy easily.         | P1       |
| US-37 | As a player, I want to place call bets via the racetrack so that I can bet on wheel sectors.                | P1       |
| US-38 | As a player, I want to see which numbers my bet covers (highlighted) when hovering so I understand the bet. | P1       |

| ID    | User Story                                                                                          | Priority |
|-------|-----------------------------------------------------------------------------------------------------|----------|
| US-39 | As a player, I want a clear error message when a bet is rejected so that I know why and can fix it. | P0       |

### 2.9.5 Gameplay — Spinning & Results

| ID    | User Story                                                                                                 | Priority |
|-------|------------------------------------------------------------------------------------------------------------|----------|
| US-40 | As a player, I want to see the wheel spin with realistic animation so that the experience feels authentic. | P0       |
| US-41 | As a player, I want to see the winning number clearly announced so that I know the result immediately.     | P0       |
| US-42 | As a player, I want to see my winning bets highlighted and losing bets removed so the outcome is clear.    | P0       |
| US-43 | As a player, I want to see my winnings animated into my balance so that wins feel rewarding.               | P0       |
| US-44 | As a player, I want to see the result's attributes (color, parity, high/low) so I can evaluate my bets.    | P1       |
| US-45 | As a player, I want a countdown to the next round so that I know when to bet again.                        | P1       |

### 2.9.6 History & Statistics

| ID    | User Story                                                                                               | Priority |
|-------|----------------------------------------------------------------------------------------------------------|----------|
| US-50 | As a player, I want to see the last 20 results at the table so that I can spot patterns (entertainment). | P0       |
| US-51 | As a player, I want to see hot and cold numbers so that I can make informed betting choices.             | P1       |
| US-52 | As a player, I want to view my game history so that I can review past sessions.                          | P1       |

| ID    | User Story                                                                                             | Priority |
|-------|--------------------------------------------------------------------------------------------------------|----------|
| US-53 | As a player, I want to see my lifetime win/loss statistics so that I can track my overall performance. | P2       |

## 2.9.7 Responsible Gaming

| ID    | User Story                                                                                          | Priority |
|-------|-----------------------------------------------------------------------------------------------------|----------|
| US-60 | As a player, I want to set a daily spending limit so that I don't spend more than I intend.         | P1       |
| US-61 | As a player, I want to set a session time reminder so that I'm aware of how long I've been playing. | P1       |
| US-62 | As a player, I want to self-exclude for a period so that I can take a break from the game.          | P1       |
| US-63 | As a player, I want to see a disclaimer that stats don't predict outcomes so I'm not misled.        | P0       |

## 2.10 Priority Legend

| Priority | Meaning                                 | Target      |
|----------|-----------------------------------------|-------------|
| P0       | Must-have for V1 launch                 | V1.0        |
| P1       | Should-have, include if timeline allows | V1.0 / V1.1 |
| P2       | Nice-to-have, planned for post-launch   | V1.2+       |

## 2.11 Edge Cases & Business Rules

### 2.11.1 Insufficient Balance

- Player attempts to place a bet exceeding their balance → Bet rejected with clear message
- Player attempts "Double" but lacks sufficient chips → "Double" fails, bets unchanged, message displayed
- Player balance reaches 0 → Prompted to purchase chips, cannot place bets

### 2.11.2 Disconnection During Game

| Phase when disconnected | Behavior                                                    |
|-------------------------|-------------------------------------------------------------|
| BETTING                 | Placed bets remain valid. Player can reconnect and modify.  |
| BETTING_CLOSING         | Placed bets remain valid. No further modification possible. |
| SPINNING                | Bets are locked. Game completes normally.                   |
| RESOLVING               | Winnings are calculated and credited regardless.            |
| COMPLETED               | Player sees results on reconnect.                           |

**Key rule:** Disconnection never cancels placed bets. Winnings are always credited server-side. The player sees results upon reconnection.

### 2.11.3 Concurrent Sessions

- Same account, multiple devices → Latest session takes priority, previous session receives "logged out" event
- Same account, multiple tables → Allowed (each table has independent bets and state)
- Same account, same table, two browsers → Blocked (single session per table)

### 2.11.4 Table Full / Table Closing

- Multiplayer table full (8/8) → Player sees "Table Full", offered alternatives
- Table closing (admin action) → Current round completes, players notified, auto-redirected to lobby
- No activity for 10 minutes (solo) → Session auto-closed, player notified

### 2.11.5 Payment Edge Cases

| Scenario                              | Handling                                               |
|---------------------------------------|--------------------------------------------------------|
| Payment initiated, no callback        | Timeout after 5 minutes, mark as PENDING, retry check  |
| Payment callback: success             | Credit chips immediately                               |
| Payment callback: failure             | No chips credited, notify player                       |
| Duplicate callback                    | Idempotent — second callback ignored (dedup by txn ID) |
| Player charged but chips not credited | Reconciliation job checks pending txns every 15 min    |

*End of Section 2*



## Section 3 — Game Mechanics (Developer Perspective)

### 3.1 Purpose of This Section

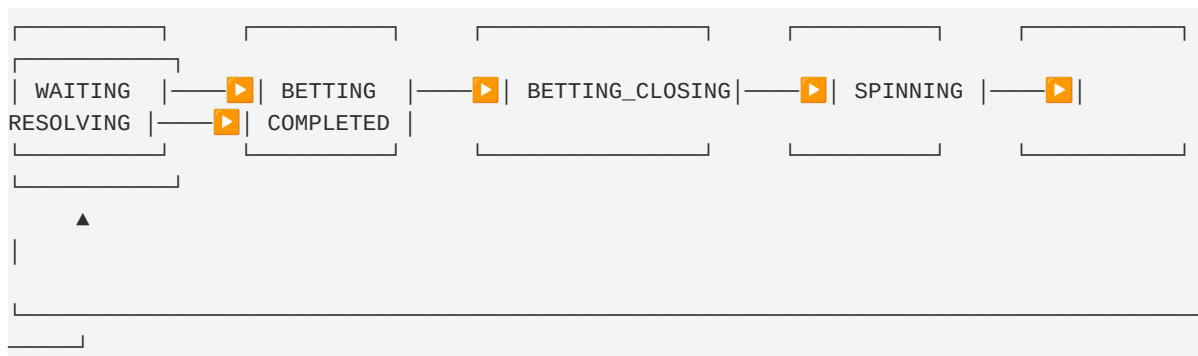
This section translates the player-facing rules (Sections 1–2) into **precise technical specifications** for backend developers. It defines the game state machine, all validation algorithms, the random number generation strategy, win/loss resolution, and the interaction patterns between components.

Every algorithm in this section must be implemented **server-side**. The Unity client is a rendering layer only — it never computes game outcomes, validates bets, or manages balances.

**Golden rule:** \_The server is the single source of truth. The client is a display.\_

### 3.2 Game State Machine

#### 3.2.1 States



#### 3.2.2 State Definitions

| State           | Entry Condition                           | Duration              | Allowed Actions                   | Exit Condition                                                         |
|-----------------|-------------------------------------------|-----------------------|-----------------------------------|------------------------------------------------------------------------|
| WAITING         | Table created OR previous round completed | Until trigger         | Join table, leave table           | Solo: player clicks "Start" / Multi: min 1 player present + auto-timer |
| BETTING         | Game round initiated                      | Configurable (30–60s) | Place bet, cancel bet, modify bet | Timer expires                                                          |
| BETTING_CLOSING | Betting timer reaches threshold           | Configurable (5–10s)  | Place bet only (no cancel/modify) | Timer expires                                                          |
| SPINNING        | Betting fully closed                      | Configurable (5–15s)  | None                              | Spin animation time elapsed                                            |
| RESOLVING       | Spin completed, result generated          | Deterministic (~1–3s) | None                              | All bets resolved, balances updated                                    |
| COMPLETED       | Resolution finished                       | Configurable (5–10s)  | View results, repeat bet prep     | Timer expires → WAITING                                                |

### 3.2.3 State Transition Rules

```
python
VALID_TRANSITIONS = {
    GameStatus.WAITING: [GameStatus.BETTING],
    GameStatus.BETTING: [GameStatus.BETTING_CLOSING],
    GameStatus.BETTING_CLOSING: [GameStatus.SPINNING],
    GameStatus.SPINNING: [GameStatus.RESOLVING],
    GameStatus.RESOLVING: [GameStatus.COMPLETED],
    GameStatus.COMPLETED: [GameStatus.WAITING],
}

def transition(current: GameStatus, target: GameStatus) -> bool:
    if target not in VALID_TRANSITIONS[current]:
        raise InvalidStateTransition(f"Cannot go from {current} to {target}")
    return True
```

#### Critical invariants:

- Transitions are **unidirectional** — no skipping, no going backward
- Each transition emits a **WebSocket event** to all connected clients
- Each transition is **logged** with timestamp for audit
- The **RESOLVING** state is **atomic** — once entered, it runs to completion without interruption

### 3.2.4 Solo vs Multiplayer State Differences

| Aspect                    | Solo Mode                                 | Multiplayer Mode                       |
|---------------------------|-------------------------------------------|----------------------------------------|
| WAITING → BETTING         | Player clicks "New Round" or auto         | Auto-triggered after COMPLETED timer   |
| BETTING duration          | No timer (player-paced) OR optional timer | Mandatory timer (server-enforced)      |
| BETTING → BETTING_CLOSING | Player clicks "Spin"                      | Timer-driven (remaining 5–10s)         |
| BETTING_CLOSING behavior  | Instant (0s) — skipped in solo            | 5–10s countdown with "Rien ne va plus" |
| SPINNING trigger          | Immediate after player confirms           | Automatic after BETTING_CLOSING timer  |
| Player leaves mid-round   | Bets remain, results calculated, credited | Same — bets are never cancelled        |

### 3.2.5 State Machine — Mermaid Diagram

```
mermaid
stateDiagram-v2
    [*] --> WAITING

    WAITING --> BETTING : start_round()
    note right of WAITING
        Solo: player clicks Start
        Multi: auto after COMPLETED timer
```

```

    or manual via admin
end note

BETTING --> BETTING_CLOSING : timer reaches closing_threshold
note right of BETTING
    Duration: configurable (30-60s)
    Actions: place_bet, cancel_bet, modify_bet
end note

BETTING_CLOSING --> SPINNING : timer expires (0s remaining)
note right of BETTING_CLOSING
    Duration: configurable (5-10s)
    Actions: place_bet ONLY
    Solo mode: may be skipped (0s)
end note

SPINNING --> RESOLVING : spin_complete()
note right of SPINNING
    RNG result generated at state ENTRY
    Animation duration: 5-15s (cosmetic)
    Result already determined server-side
end note

RESOLVING --> COMPLETED : all_bets_resolved()
note right of RESOLVING
    Atomic phase – no interruption
    Calculate all wins/losses
    Update all balances
    Emit results to clients
end note

COMPLETED --> WAITING : completed_timer_expires()
note right of COMPLETED
    Duration: 5-10s
    Show results
    Allow repeat bet preparation
end note

```

## 3.3 Random Number Generation (RNG)

### 3.3.1 Requirements

| Requirement             | Specification                                       |
|-------------------------|-----------------------------------------------------|
| Fairness                | Uniform distribution across all pockets             |
| Unpredictability        | No player or operator can predict the next result   |
| Reproducibility (audit) | Each result must be verifiable with its seed/params |
| Performance             | Generation must complete in < 1ms                   |

### 3.3.2 Implementation Strategy

#### V1 (Social game — no gambling license required):

Use Python's `secrets` module, which is backed by the OS-level CSPRNG (`/dev/urandom` on Linux). This is cryptographically secure and sufficient for a social game.

```
python
import secrets

def spin_wheel(variant: str) -> int:
    """
    Generate a random result for the given roulette variant.
    Returns: integer representing the winning number.
           0 = zero, -1 = double zero (American only), 1-36 = standard numbers
    """
    if variant == "EUROPEAN":
        # Pick random index 0-36 → maps to wheel position
        index = secrets.randbelow(37)
        return EUROPEAN_WHEEL_ORDER[index]
    elif variant == "AMERICAN":
        # Pick random index 0-37 → maps to wheel position
        index = secrets.randbelow(38)
        return AMERICAN_WHEEL_ORDER[index]
    else:
        raise ValueError(f"Unknown variant: {variant}")
```

### 3.3.3 Result Generation Timing

**Critical design decision:** The result is generated at the **entry of the SPINNING state**, NOT at the end. The animation duration is purely cosmetic — the server already knows the result.

```
Timeline:
  BETTING_CLOSING ends
  → SPINNING state entered
  → result = spin_wheel(variant) ← Result determined HERE
  → result stored in game state (not yet broadcast)
  → [5-15 seconds of animation on client]
  → SPINNING ends
  → RESOLVING state entered
  → result broadcast to all clients
  → wins/losses calculated
```

#### Why generate early?

- Allows pre-computation of wins during animation time
- Ensures result is immutable once SPINNING starts
- Animation can be driven by the known result (Unity calculates landing position)

### 3.3.4 Result Record

Every spin result is persisted with the following data:

```
python
@dataclass
```

```

class SpinResult:
    id: str                # UUID
    game_round_id: str      # FK to game round
    table_id: str           # FK to table
    variant: str            # EUROPEAN or AMERICAN
    winning_number: int     # -1 (00), 0, or 1-36
    winning_color: str      # RED, BLACK, or GREEN
    winning_parity: str     # EVEN, ODD, or NONE (for 0/00)
    winning_range: str      # LOW, HIGH, or NONE (for 0/00)
    winning_dozen: int      # 1, 2, 3, or 0 (for 0/00)
    winning_column: int     # 1, 2, 3, or 0 (for 0/00)
    wheel_position: int     # Index on physical wheel (0-36 or 0-37)
    generated_at: datetime  # Timestamp of generation
    seed_hash: str          # SHA-256 hash of the RNG state (for audit)

```

### 3.3.5 Provably Fair System (Optional — recommended)

For player trust, implement a **commit-reveal scheme**:

1. Before BETTING starts, server generates a `server_seed` and broadcasts its SHA-256 hash
2. When SPINNING starts, server publishes the actual `server_seed`
3.  $\text{Result} = \text{hash}(\text{server\_seed} + \text{round\_id}) \bmod \text{total\_pockets}$
4. Players can independently verify the result matches the pre-committed hash

**Complexity:** Low — adds 2 fields to the round record and 1 API endpoint for verification.

**Recommendation:** Include in V1. Builds player trust with minimal effort.

### 3.3.6 V2 Considerations (Licensed Gambling)

When cashout is introduced, the RNG must be:

- Certified by an independent testing lab (e.g., GLI, BMM, eCOGRA)
- Using a certified algorithm (e.g., Fortuna, ChaCha20-based)
- Periodically audited with statistical tests (chi-square, runs test, serial correlation)

The V1 architecture (`secrets.randbelow()`) can be replaced without structural changes since the `spin_wheel()` function is the only RNG entry point.

## 3.4 Bet Validation Engine

### 3.4.1 Validation Pipeline

Every bet passes through a **sequential validation pipeline**. If any step fails, the bet is rejected with a specific error code.

```

Incoming bet request
|
├─ Step 1: Game state check (is betting open?)
├─ Step 2: Player state check (is player at this table?)
├─ Step 3: Balance check (can player afford this bet?)
├─ Step 4: Table limits check (min/max per bet, max total)
├─ Step 5: Bet structure check (valid type + valid numbers)
└─ Step 6: Variant check (is this bet type allowed for this variant?)

```

```
|
└ All passed → Bet accepted, balance reserved, bet stored
```

### 3.4.2 Error Codes

| Code  | Name                 | Description                                      |
|-------|----------------------|--------------------------------------------------|
| E1001 | BETTING_CLOSED       | Game is not in BETTING or BETTING_CLOSING state  |
| E1002 | PLAYER_NOT_AT_TABLE  | Player has not joined this table                 |
| E1003 | INSUFFICIENT_BALANCE | Player balance < bet amount                      |
| E1004 | BELOW_MIN_BET        | Bet amount < table minimum bet                   |
| E1005 | ABOVE_MAX_BET        | Bet amount > table maximum bet per position      |
| E1006 | ABOVE_MAX_TOTAL      | Player's total bets this round > table max total |
| E1007 | INVALID_BET_TYPE     | Bet type not recognized                          |
| E1008 | INVALID_BET_NUMBERS  | Numbers don't match the bet type rules           |
| E1009 | BET_NOT_AVAILABLE    | Bet type not available for this variant          |
| E1010 | CANCEL_NOT_ALLOWED   | Cannot cancel bet in BETTING_CLOSING state       |
| E1011 | MODIFY_NOT_ALLOWED   | Cannot modify bet in BETTING_CLOSING state       |
| E1012 | TABLE_FULL           | Maximum players reached (multiplayer)            |

### 3.4.3 Bet Structure Validation — Complete Rules

This is the core validation logic. Each bet type has specific rules for which numbers are valid.

```
python
from enum import Enum
from typing import Optional

class BetType(str, Enum):
    # Inside bets
    STRAIGHT_UP = "STRAIGHT_UP"
    SPLIT = "SPLIT"
    STREET = "STREET"
    TRIO = "TRIO"
    CORNER = "CORNER"
    FIRST_FOUR = "FIRST_FOUR"
    TOP_LINE = "TOP_LINE"
    SIX_LINE = "SIX_LINE"
    # Outside bets
```

```

COLUMN_1 = "COLUMN_1"
COLUMN_2 = "COLUMN_2"
COLUMN_3 = "COLUMN_3"
DOZEN_1 = "DOZEN_1"
DOZEN_2 = "DOZEN_2"
DOZEN_3 = "DOZEN_3"
RED = "RED"
BLACK = "BLACK"
EVEN = "EVEN"
ODD = "ODD"
LOW = "LOW"
HIGH = "HIGH"
# Call bets (expanded at placement time)
CALL_VOISINS_ZERO = "CALL_VOISINS_ZERO"
CALL_TIERS = "CALL_TIERS"
CALL_ORPHELINS = "CALL_ORPHELINS"
CALL_NEIGHBORS = "CALL_NEIGHBORS"

# ——— Constants ———

DOUBLE_ZERO = -1 # Internal representation of "00"

RED_NUMBERS = frozenset({1, 3, 5, 7, 9, 12, 14, 16, 18, 19, 21, 23, 25, 27, 30, 32, 34,
36})
BLACK_NUMBERS = frozenset({2, 4, 6, 8, 10, 11, 13, 15, 17, 20, 22, 24, 26, 28, 29, 31,
33, 35})

ALL_NUMBERS_EUROPEAN = frozenset(range(0, 37)) # {0, 1, 2, ..., 36}
ALL_NUMBERS_AMERICAN = ALL_NUMBERS_EUROPEAN | {DOUBLE_ZERO} # adds -1

EUROPEAN_WHEEL_ORDER = [
    0, 32, 15, 19, 4, 21, 2, 25, 17, 34,
    6, 27, 13, 36, 11, 30, 8, 23, 10, 5,
    24, 16, 33, 1, 20, 14, 31, 9, 22, 18,
    29, 7, 28, 12, 35, 3, 26
]

AMERICAN_WHEEL_ORDER = [
    0, 28, 9, 26, 30, 11, 7, 20, 32, 17,
    5, 22, 34, 15, 3, 24, 36, 13, 1, DOUBLE_ZERO,
    27, 10, 25, 29, 12, 8, 19, 31, 18, 6,
    21, 33, 16, 4, 23, 35, 14, 2
]

# ——— Grid Helpers ———

def grid_row(n: int) -> int:
    """Row on betting grid (1-12) for numbers 1-36."""
    return (n - 1) // 3 + 1

def grid_col(n: int) -> int:
    """Column on betting grid (1-3) for numbers 1-36."""

```

```

    return (n - 1) % 3 + 1

# — Validation Functions per Bet Type —————

def validate_straight_up(numbers: list[int], variant: str) -> bool:
    """Any single valid number."""
    if len(numbers) != 1:
        return False
    n = numbers[0]
    valid = ALL_NUMBERS_AMERICAN if variant == "AMERICAN" else ALL_NUMBERS_EUROPEAN
    return n in valid

def validate_split(numbers: list[int], variant: str) -> bool:
    """Two adjacent numbers on the grid or zero-related pairs."""
    if len(numbers) != 2:
        return False
    a, b = sorted(numbers)

    # 00-related (American only)
    if a == DOUBLE_ZERO:
        if variant != "AMERICAN":
            return False
        return b in {0, 2, 3}

    # 0-related
    if a == 0:
        return b in {1, 2, 3}

    # Both must be 1-36
    if not (1 <= a <= 36 and 1 <= b <= 36):
        return False

    # Vertical neighbor: same column, adjacent rows (differ by 3)
    if b - a == 3:
        return True

    # Horizontal neighbor: same row, adjacent columns (differ by 1, same row)
    if b - a == 1 and grid_row(a) == grid_row(b):
        return True

    return False

def validate_street(numbers: list[int], variant: str) -> bool:
    """Three consecutive numbers forming a row: [n, n+1, n+2] where (n-1)%3==0."""
    if len(numbers) != 3:
        return False
    s = sorted(numbers)
    n = s[0]
    if not (1 <= n <= 34):
        return False
    if (n - 1) % 3 != 0:

```



```

        return False
    return s == [n, n + 1, n + 2]

def validate_trio(numbers: list[int], variant: str) -> bool:
    """Three numbers including at least one zero."""
    s = sorted(numbers)
    if len(s) != 3:
        return False
    valid_trios_european = [{0, 1, 2}, {0, 2, 3}]
    valid_trios_american = valid_trios_european + [{DOUBLE_ZERO, 0, 2}, {DOUBLE_ZERO, 2,
3}]
    valid = valid_trios_american if variant == "AMERICAN" else valid_trios_european
    return set(s) in valid

def validate_corner(numbers: list[int], variant: str) -> bool:
    """Four numbers forming a 2x2 square on the grid."""
    if len(numbers) != 4:
        return False
    s = sorted(numbers)
    n = s[0]
    if not (1 <= n <= 32):
        return False
    if grid_col(n) == 3: # Column 3 can't be top-left of a square
        return False
    return s == [n, n + 1, n + 3, n + 4]

def validate_first_four(numbers: list[int], variant: str) -> bool:
    """European only: 0, 1, 2, 3."""
    if variant != "EUROPEAN":
        return False
    return sorted(numbers) == [0, 1, 2, 3]

def validate_top_line(numbers: list[int], variant: str) -> bool:
    """American only: 00, 0, 1, 2, 3."""
    if variant != "AMERICAN":
        return False
    return sorted(numbers) == [DOUBLE_ZERO, 0, 1, 2, 3]

def validate_six_line(numbers: list[int], variant: str) -> bool:
    """Six numbers = two adjacent streets."""
    if len(numbers) != 6:
        return False
    s = sorted(numbers)
    n = s[0]
    if not (1 <= n <= 31):
        return False
    if (n - 1) % 3 != 0:
        return False
    return s == [n, n + 1, n + 2, n + 3, n + 4, n + 5]

```

```

# Outside bets don't need number validation – they have fixed coverage.
# The `numbers` field is auto-populated by the server based on bet type.

# — Master Validation Dispatcher —————

VALIDATORS = {
    BetType.STRAIGHT_UP: validate_straight_up,
    BetType.SPLIT: validate_split,
    BetType.STREET: validate_street,
    BetType.TRIO: validate_trio,
    BetType.CORNER: validate_corner,
    BetType.FIRST_FOUR: validate_first_four,
    BetType.TOP_LINE: validate_top_line,
    BetType.SIX_LINE: validate_six_line,
}

OUTSIDE_BETS = {
    BetType.RED, BetType.BLACK, BetType.EVEN, BetType.ODD,
    BetType.LOW, BetType.HIGH,
    BetType.DOZEN_1, BetType.DOZEN_2, BetType.DOZEN_3,
    BetType.COLUMN_1, BetType.COLUMN_2, BetType.COLUMN_3,
}

VARIANT_RESTRICTED = {
    BetType.FIRST_FOUR: "EUROPEAN",
    BetType.TOP_LINE: "AMERICAN",
    BetType.CALL_VOISINS_ZERO: "EUROPEAN",
    BetType.CALL_TIERS: "EUROPEAN",
    BetType.CALL_ORPHELINS: "EUROPEAN",
}

def validate_bet_structure(bet_type: BetType, numbers: list[int], variant: str) ->
tuple[bool, str]:
    """
    Validate that the numbers are valid for the given bet type and variant.
    Returns (is_valid, error_message).
    """
    # Check variant restriction
    if bet_type in VARIANT_RESTRICTED:
        if VARIANT_RESTRICTED[bet_type] != variant:
            return False, f"{bet_type} is not available for {variant} roulette"

    # Outside bets: no number validation needed (server auto-populates)
    if bet_type in OUTSIDE_BETS:
        return True, ""

    # Inside bets: validate numbers
    validator = VALIDATORS.get(bet_type)
    if validator is None:
        return False, f"Unknown bet type: {bet_type}"

```

```

if validator(numbers, variant):
    return True, ""
else:
    return False, f"Invalid numbers {numbers} for bet type {bet_type}"

```

### 3.4.4 Full Bet Validation Pipeline — Implementation

```

python
def validate_bet(
    bet_type: BetType,
    numbers: list[int],
    amount: int,
    player_id: str,
    game_round: GameRound,
    player: Player,
    table: Table,
) -> tuple[bool, Optional[str]]:
    """
    Complete bet validation pipeline.
    Returns (is_valid, error_code_or_none).
    """

    # Step 1: Game state check
    if game_round.status not in (GameStatus.BETTING, GameStatus.BETTING_CLOSING):
        return False, "E1001"

    # Step 2: Player at table check
    if player_id not in game_round.seated_players:
        return False, "E1002"

    # Step 3: Balance check
    if player.balance < amount:
        return False, "E1003"

    # Step 4: Table limits
    if amount < table.min_bet:
        return False, "E1004"

    if amount > table.max_bet:
        return False, "E1005"

    player_total = sum(b.amount for b in game_round.bets if b.player_id == player_id)
    if player_total + amount > table.max_total_bet:
        return False, "E1006"

    # Step 5: Bet structure validation
    is_valid, msg = validate_bet_structure(bet_type, numbers, table.variant)
    if not is_valid:
        return False, "E1008"

    # Step 6: Variant restriction
    if bet_type in VARIANT_RESTRICTED and VARIANT_RESTRICTED[bet_type] != table.variant:

```

```

        return False, "E1009"

    # Step 7: Cancel/Modify restriction in BETTING_CLOSING
    # (handled at the endpoint level, not here)

    return True, None

```

## 3.5 Covered Numbers Resolution

When a bet is placed, the server must determine which numbers it covers (for win resolution). Outside bets have fixed coverage; inside bets derive coverage from the provided numbers.

```

python
def get_covered_numbers(bet_type: BetType, numbers: list[int]) -> frozenset[int]:
    """
    Return the set of numbers covered by this bet.
    For inside bets: returns the numbers as provided (already validated).
    For outside bets: returns the predefined set.
    """

    # Inside bets: coverage = the numbers provided
    if bet_type in (BetType.STRAIGHT_UP, BetType.SPLIT, BetType.STREET,
                    BetType.TRIO, BetType.CORNER, BetType.FIRST_FOUR,
                    BetType.TOP_LINE, BetType.SIX_LINE):
        return frozenset(numbers)

    # Outside bets: predefined coverage
    COVERAGE = {
        BetType.RED: RED_NUMBERS,
        BetType.BLACK: BLACK_NUMBERS,
        BetType.EVEN: frozenset(n for n in range(1, 37) if n % 2 == 0),
        BetType.ODD: frozenset(n for n in range(1, 37) if n % 2 == 1),
        BetType.LOW: frozenset(range(1, 19)),
        BetType.HIGH: frozenset(range(19, 37)),
        BetType.DOZEN_1: frozenset(range(1, 13)),
        BetType.DOZEN_2: frozenset(range(13, 25)),
        BetType.DOZEN_3: frozenset(range(25, 37)),
        BetType.COLUMN_1: frozenset(n for n in range(1, 37) if n % 3 == 1),
        BetType.COLUMN_2: frozenset(n for n in range(1, 37) if n % 3 == 2),
        BetType.COLUMN_3: frozenset(n for n in range(1, 37) if n % 3 == 0),
    }

    return COVERAGE.get(bet_type, frozenset())

```

## 3.6 Win/Loss Resolution

### 3.6.1 Payout Table

```

python
PAYOUTS: dict[BetType, int] = {
    BetType.STRAIGHT_UP: 35,
    BetType.SPLIT: 17,

```

```

BetType.STREET:      11,
BetType.TRIO:        11,
BetType.CORNER:       8,
BetType.FIRST_FOUR:   8,
BetType.TOP_LINE:     6,
BetType.SIX_LINE:     5,
BetType.COLUMN_1:     2,
BetType.COLUMN_2:     2,
BetType.COLUMN_3:     2,
BetType.DOZEN_1:      2,
BetType.DOZEN_2:      2,
BetType.DOZEN_3:      2,
BetType.RED:          1,
BetType.BLACK:        1,
BetType.EVEN:          1,
BetType.ODD:           1,
BetType.LOW:          1,
BetType.HIGH:         1,
}

```

### 3.6.2 Resolution Algorithm

```

python
@dataclass
class BetResult:
    bet_id: str
    player_id: str
    bet_type: BetType
    amount: int
    covered_numbers: frozenset[int]
    is_winner: bool
    payout_multiplier: int
    win_amount: int          # Total return (stake + profit). 0 if lost.
    net_result: int          # Profit or loss. Negative if lost.

def resolve_bet(bet: Bet, winning_number: int, la_partage: bool = False) -> BetResult:
    """
    Resolve a single bet against the winning number.

    Returns a BetResult with all computed fields.
    """
    covered = get_covered_numbers(bet.type, bet.numbers)
    is_winner = winning_number in covered

    if is_winner:
        multiplier = PAYOUTS[bet.type]
        win_amount = bet.amount * (multiplier + 1)  # stake returned + profit
        net_result = bet.amount * multiplier        # profit only
    else:
        # Check La Partage rule
        if (la_partage
            and winning_number == 0

```

```

        and bet.type in {BetType.RED, BetType.BLACK, BetType.EVEN,
                        BetType.ODD, BetType.LOW, BetType.HIGH}):
            win_amount = bet.amount // 2 # Return half stake (integer division)
            net_result = -(bet.amount - win_amount) # Lost half
        else:
            win_amount = 0
            net_result = -bet.amount

    return BetResult(
        bet_id=bet.id,
        player_id=bet.player_id,
        bet_type=bet.type,
        amount=bet.amount,
        covered_numbers=covered,
        is_winner=is_winner,
        payout_multiplier=PAYOUTS[bet.type] if is_winner else 0,
        win_amount=win_amount,
        net_result=net_result,
    )

def resolve_round(game_round: GameRound, winning_number: int) -> list[BetResult]:
    """
    Resolve ALL bets for a round. Returns list of BetResult.
    This function must be called exactly once per round, inside the RESOLVING state.
    """
    results = []
    la_partage = game_round.table.rules.get("la_partage", False)

    for bet in game_round.bets:
        result = resolve_bet(bet, winning_number, la_partage)
        results.append(result)

    return results

```

### 3.6.3 Balance Update (Transactional)

```

python
def apply_round_results(results: list[BetResult], session) -> dict[str, int]:
    """
    Apply all bet results to player balances in a single database transaction.
    Returns dict of {player_id: new_balance}.

    CRITICAL: This must run inside a database transaction.
    If any update fails, ALL updates are rolled back.
    """
    player_summaries: dict[str, int] = {}

    for result in results:
        if result.player_id not in player_summaries:
            player_summaries[result.player_id] = 0
        player_summaries[result.player_id] += result.net_result

```

```

# Apply to database in single transaction
updated_balances = {}
for player_id, net_change in player_summaries.items():
    player = session.get(Player, player_id)
    # Bets were already deducted at placement time, so:
    # - If net_change > 0: player won, add winnings (stake was already deducted)
    # - If net_change < 0: nothing more to deduct (stake already taken)
    # - Actually: we need to return stake + profit for wins
    #
    # Correct approach:
    # At bet placement: balance -= bet.amount (reserved)
    # At resolution: balance += win_amount (0 for losses, stake+profit for wins)
    player.balance += result.win_amount # Return 0 (loss) or stake+profit (win)
    updated_balances[player_id] = player.balance

session.commit()
return updated_balances

```

### 3.6.4 Balance Flow Diagram

Player starts with 1000 chips.

Places: Straight Up on 17 for 100 chips, Red for 200 chips.

AT BET PLACEMENT:

balance:  $1000 - 100 - 200 = 700$  (reserved 300 total)

SCENARIO A — Number 17 hits (Black):

Straight Up 17: WIN  $\rightarrow$  win\_amount =  $100 * 36 = 3,600$

Red: LOSE  $\rightarrow$  win\_amount = 0

balance:  $700 + 3,600 + 0 = 4,300$

SCENARIO B — Number 12 hits (Red):

Straight Up 17: LOSE  $\rightarrow$  win\_amount = 0

Red: WIN  $\rightarrow$  win\_amount =  $200 * 2 = 400$

balance:  $700 + 0 + 400 = 1,100$

SCENARIO C — Number 0 hits (Green):

Straight Up 17: LOSE  $\rightarrow$  win\_amount = 0

Red: LOSE  $\rightarrow$  win\_amount = 0

balance:  $700 + 0 + 0 = 700$

SCENARIO D — Number 0 hits, La Partage enabled:

Straight Up 17: LOSE  $\rightarrow$  win\_amount = 0

Red: LA PARTAGE  $\rightarrow$  win\_amount =  $200 // 2 = 100$

balance:  $700 + 0 + 100 = 800$

## 3.7 Call Bet Expansion

Call bets are **macros** that expand into multiple standard bets. The expansion happens server-side at placement time. The client sends a single call bet request; the server creates multiple individual bets.

```

python
def expand_call_bet(
    call_type: BetType,
    unit_amount: int,
    variant: str,
    target_number: Optional[int] = None, # For CALL_NEIGHBORS
    neighbor_count: int = 2,            # For CALL_NEIGHBORS
) -> list[dict]:
    """
    Expand a call bet into its constituent individual bets.
    Each returned dict has: {"type": BetType, "numbers": [...], "amount": int}
    unit_amount = the value of 1 chip unit for this call bet.
    """

    if call_type == BetType.CALL_VOISINS_ZERO:
        if variant != "EUROPEAN":
            raise ValueError("Voisins du Zéro only available on European")
        return [
            {"type": BetType.TRIO, "numbers": [0, 2, 3], "amount": unit_amount *
2},
            {"type": BetType.SPLIT, "numbers": [4, 7], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [12, 15], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [18, 21], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [19, 22], "amount": unit_amount},
            {"type": BetType.CORNER, "numbers": [25, 26, 28, 29], "amount": unit_amount *
2},
            {"type": BetType.SPLIT, "numbers": [32, 35], "amount": unit_amount},
        ]
        # Total: 9 units

    elif call_type == BetType.CALL_TIERS:
        if variant != "EUROPEAN":
            raise ValueError("Tiers du Cylindre only available on European")
        return [
            {"type": BetType.SPLIT, "numbers": [5, 8], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [10, 11], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [13, 16], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [23, 24], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [27, 30], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [33, 36], "amount": unit_amount},
        ]
        # Total: 6 units

    elif call_type == BetType.CALL_ORPHELINS:
        if variant != "EUROPEAN":
            raise ValueError("Orphelins only available on European")
        return [
            {"type": BetType.STRAIGHT_UP, "numbers": [1], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [6, 9], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [14, 17], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [17, 20], "amount": unit_amount},
            {"type": BetType.SPLIT, "numbers": [31, 34], "amount": unit_amount},
        ]
        # Total: 5 units

```



```

elif call_type == BetType.CALL_NEIGHBORS:
    if target_number is None:
        raise ValueError("CALL_NEIGHBORS requires a target_number")
    wheel = EUROPEAN_WHEEL_ORDER if variant == "EUROPEAN" else AMERICAN_WHEEL_ORDER
    neighbors = get_neighbors(target_number, neighbor_count, wheel)
    return [
        {"type": BetType.STRAIGHT_UP, "numbers": [n], "amount": unit_amount}
        for n in neighbors
    ]
    # Total: (2 * neighbor_count + 1) units

else:
    raise ValueError(f"Unknown call bet type: {call_type}")

def get_neighbors(number: int, count: int, wheel_order: list[int]) -> list[int]:
    """Get `count` neighbors on each side of `number` on the wheel."""
    idx = wheel_order.index(number)
    size = len(wheel_order)
    return [wheel_order[(idx + i) % size] for i in range(-count, count + 1)]

```

## 3.8 Multiplayer Synchronization

### 3.8.1 Concurrency Model

Multiple players betting on the same table simultaneously requires careful handling:

#### Approach: Optimistic concurrency with server-side sequencing

1. Each bet placement request is processed **sequentially per table** (single-threaded per table)
2. Use an **async lock per table** (e.g., `asyncio.Lock()`) — NOT a global lock
3. Balance deduction is immediate and atomic (database row-level lock on player balance)
4. Bet validation re-checks balance AFTER acquiring the lock (prevents race conditions)

```

python
# One lock per table – concurrent tables don't block each other
table_locks: dict[str, asyncio.Lock] = {}

async def place_bet_synchronized(table_id: str, bet_request: BetRequest):
    if table_id not in table_locks:
        table_locks[table_id] = asyncio.Lock()

    async with table_locks[table_id]:
        # Inside lock: validate and place bet atomically
        # Balance check is guaranteed accurate
        result = await _validate_and_place_bet(bet_request)
        return result

```

### 3.8.2 WebSocket Event Flow

```

mermaid

```

```
sequenceDiagram
    participant P1 as Player 1 (Unity)
    participant P2 as Player 2 (Unity)
    participant WS as WebSocket Server
    participant GM as GameManager
    participant DB as Database

    Note over GM: State: WAITING → BETTING
    GM->>WS: emit("game:betting-started", {timeRemaining: 45})
    WS->>P1: game:betting-started
    WS->>P2: game:betting-started

    P1->>WS: place_bet({type: "RED", amount: 100})
    WS->>GM: validate_and_place_bet()
    GM->>DB: deduct balance, store bet
    GM->>WS: emit("bet:placed", {playerId: P1})
    WS->>P1: bet:accepted({betId, newBalance})

    P2->>WS: place_bet({type: "STRAIGHT_UP", numbers: [17], amount: 50})
    WS->>GM: validate_and_place_bet()
    GM->>DB: deduct balance, store bet
    GM->>WS: emit("bet:placed", {playerId: P2})
    WS->>P2: bet:accepted({betId, newBalance})

    Note over GM: Timer expires → BETTING_CLOSING
    GM->>WS: emit("game:betting-closing", {timeRemaining: 5})
    WS->>P1: game:betting-closing
    WS->>P2: game:betting-closing

    Note over GM: Timer expires → SPINNING
    GM->>GM: result = spin_wheel("EUROPEAN")
    GM->>WS: emit("game:spinning", {spinDuration: 8})
    WS->>P1: game:spinning
    WS->>P2: game:spinning

    Note over GM: Animation done → RESOLVING
    GM->>GM: results = resolve_round(bets, winning_number)
    GM->>DB: update balances (transaction)
    GM->>WS: emit("game:result", {number: 17, color: "BLACK", ...})
    WS->>P1: game:result + personal_result({totalWon: 0, newBalance: 900})
    WS->>P2: game:result + personal_result({totalWon: 1800, newBalance: 2750})

    Note over GM: COMPLETED → wait → WAITING
    GM->>WS: emit("game:completed", {nextRoundIn: 8})
    WS->>P1: game:completed
    WS->>P2: game:completed
```

### 3.8.3 Event Privacy

| Event                | Broadcast to | Content                      |
|----------------------|--------------|------------------------------|
| game:state-changed   | All players  | New state, timer info        |
| game:betting-started | All players  | Time remaining, table limits |

| Event                  | Broadcast to | Content                                 |
|------------------------|--------------|-----------------------------------------|
| game:betting-closing   | All players  | Time remaining                          |
| game:spinning          | All players  | Spin duration (for animation sync)      |
| game:result            | All players  | Winning number, color, attributes       |
| bet:accepted           | Bet owner    | Bet details, new balance                |
| bet:rejected           | Bet owner    | Error code, error message               |
| personal_result        | Each player  | Their wins/losses, new balance          |
| player:balance-updated | That player  | New balance                             |
| table:player-joined    | All players  | Player display name (no financial info) |
| table:player-left      | All players  | Player display name                     |

**Critical:** Player balances and bet details are NEVER broadcast to other players. Only display names and presence are shared.

## 3.9 Timer Management

### 3.9.1 Server-Authoritative Timers

All timers are managed **server-side**. The client displays a countdown based on the `timeRemaining` value received via WebSocket, but the server is the authority on when phases transition.

```
python
class RoundTimer:
    """Manages the timing of a single game round."""

    def __init__(self, config: TableConfig):
        self.betting_duration = config.betting_duration          # e.g., 45 seconds
        self.closing_duration = config.closing_duration          # e.g., 5 seconds
        self.spinning_duration = config.spinning_duration        # e.g., 8 seconds
        self.completed_duration = config.completed_duration       # e.g., 8 seconds
        self._phase_start: Optional[float] = None

    def start_phase(self, phase: GameStatus):
        self._phase_start = time.monotonic()

    def time_remaining(self, phase: GameStatus) -> float:
        if self._phase_start is None:
            return 0.0
        elapsed = time.monotonic() - self._phase_start
        duration = {
            GameStatus.BETTING: self.betting_duration,
            GameStatus.BETTING_CLOSING: self.closing_duration,
            GameStatus.SPINNING: self.spinning_duration,
            GameStatus.COMPLETED: self.completed_duration,
        }.get(phase, 0)
        return max(0.0, duration - elapsed)
```

```
def is_expired(self, phase: GameStatus) -> bool:
    return self.time_remaining(phase) <= 0
```

### 3.9.2 Timer Synchronization

To handle clock drift between server and client:

- Server sends `timeRemaining` (in seconds, float) with every state change event
- Client starts its own local countdown from the received value
- Client never decides when a phase ends — it only animates the countdown
- If client timer reaches 0 before server event, client shows "Waiting..." until server event arrives

## 3.10 Backoffice — Table Management (Admin)

### 3.10.1 Table CRUD Operations

All table configuration is managed via the backoffice admin panel. No tables are hardcoded.

**Table creation parameters (set by admin):**

| Parameter                       | Type    | Default  | Constraints                       |
|---------------------------------|---------|----------|-----------------------------------|
| <code>name</code>               | string  | Required | Unique, 3–50 chars                |
| <code>variant</code>            | enum    | EUROPEAN | EUROPEAN or AMERICAN              |
| <code>mode</code>               | enum    | SOLO     | SOLO or MULTIPLAYER               |
| <code>min_bet</code>            | int     | 10       | $\geq 1$                          |
| <code>max_bet</code>            | int     | 1000     | $> \text{min\_bet}$               |
| <code>max_total_bet</code>      | int     | 5000     | $\geq \text{max\_bet}$            |
| <code>max_players</code>        | int     | 8        | 1–20 (multiplayer only)           |
| <code>betting_duration</code>   | int (s) | 45       | 15–120                            |
| <code>closing_duration</code>   | int (s) | 5        | 0–15                              |
| <code>spinning_duration</code>  | int (s) | 8        | 3–20                              |
| <code>completed_duration</code> | int (s) | 8        | 3–30                              |
| <code>la_partage</code>         | bool    | false    | European only                     |
| <code>is_active</code>          | bool    | true     | Inactive tables hidden from lobby |
| <code>tier</code>               | enum    | STANDARD | BEGINNER, STANDARD, PREMIUM, VIP  |

### 3.10.2 Admin Actions

| Action            | Description                                   | Effect on active players             |
|-------------------|-----------------------------------------------|--------------------------------------|
| Create table      | New table appears in lobby                    | None                                 |
| Edit table config | Modify limits, timers, rules                  | Applied at next round start          |
| Deactivate table  | Table hidden from lobby                       | Current round completes, then closed |
| Delete table      | Permanently remove table                      | Only if no active players            |
| Force close table | Immediately end current round                 | All bets refunded, players kicked    |
| View table stats  | See live player count, round history, revenue | Read-only                            |

### 3.10.3 Backoffice Additional Modules

| Module                  | Description                                       | Priority |
|-------------------------|---------------------------------------------------|----------|
| Table Management        | CRUD tables, configure limits/rules               | P0       |
| Player Management       | View/search players, view balance, ban/suspend    | P0       |
| Transaction History     | All chip purchases, bets, payouts with filters    | P0       |
| Chip Package Management | CRUD purchase packages, set prices/bonuses        | P0       |
| Dashboard               | KPIs: DAU, revenue, total wagered, top players    | P1       |
| Bonus Management        | Configure welcome bonus, promotional bonuses      | P1       |
| Responsible Gaming      | View flagged players, spending alerts             | P1       |
| System Config           | Global settings (maintenance mode, feature flags) | P1       |
| Audit Logs              | All admin actions with timestamps and actor       | P0       |

## 3.11 Sequence Diagram — Complete Game Round

```

sequenceDiagram
    participant Admin as Backoffice Admin
    participant GM as GameManager
    participant BM as BetManager
    participant WC as WheelController
    participant PM as PlayerManager
    participant DB as Database
  
```

```

participant WS as WebSocket
participant Client as Unity Client

Note over Admin: Admin creates table via backoffice
Admin->>DB: INSERT table config

Note over GM: Game Round Lifecycle

rect rgb(230, 245, 230)
  Note over GM: Phase: WAITING
  Client->>WS: join_table(table_id)
  WS->>PM: validate_player(player_id)
  PM->>DB: check player exists, balance > 0
  PM-->>WS: player_validated
  WS-->>Client: table:joined(table_state)
  GM->>GM: start_round() [auto or manual]
end

rect rgb(230, 230, 245)
  Note over GM: Phase: BETTING (45s)
  GM->>WS: game:betting-started(timeRemaining=45)
  WS->>Client: game:betting-started

  Client->>WS: place_bet(type, numbers, amount)
  WS->>BM: validate_bet(...)
  BM->>DB: check balance, deduct, store bet
  BM-->>WS: bet_accepted(bet_id, new_balance)
  WS-->>Client: bet:accepted
end

rect rgb(245, 240, 220)
  Note over GM: Phase: BETTING_CLOSING (5s)
  GM->>WS: game:betting-closing(timeRemaining=5)
  WS->>Client: game:betting-closing
end

rect rgb(245, 230, 230)
  Note over GM: Phase: SPINNING
  GM->>WC: spin_wheel(variant)
  WC->>WC: result = secrets.randbelow(N)
  WC-->>GM: SpinResult(number, color, ...)
  GM->>DB: store spin result
  GM->>WS: game:spinning(duration=8, result_hash=...)
  WS->>Client: game:spinning
  Note over Client: Wheel animation (8s)
end

rect rgb(240, 230, 245)
  Note over GM: Phase: RESOLVING
  GM->>BM: resolve_round(bets, winning_number)
  BM->>BM: calculate all win/loss
  BM->>DB: update balances (single transaction)
  BM-->>GM: list[BetResult]
  GM->>WS: game:result(number, color, attributes)

```

```

    WS->>Client: game:result + personal_result
end

rect rgb(230, 240, 240)
    Note over GM: Phase: COMPLETED (8s)
    GM->>WS: game:completed(nextRoundIn=8)
    WS->>Client: game:completed
    Note over GM: Timer expires → back to WAITING
end

```

### 3.12 Class Diagram (Core Domain)

```

mermaid
classDiagram
    class GameManager {
        -table_id: str
        -current_round: GameRound
        -timer: RoundTimer
        -lock: asyncio.Lock
        +start_round()
        +transition_state(target: GameState)
        +get_state() GameRound
    }

    class GameRound {
        +id: str
        +table_id: str
        +status: GameState
        +bets: list~Bet~
        +result: SpinResult
        +started_at: datetime
        +completed_at: datetime
    }

    class Table {
        +id: str
        +name: str
        +variant: str
        +mode: str
        +min_bet: int
        +max_bet: int
        +max_total_bet: int
        +max_players: int
        +betting_duration: int
        +closing_duration: int
        +spinning_duration: int
        +completed_duration: int
        +la_partage: bool
        +is_active: bool
        +tier: str
    }

    class BetManager {

```

```

+validate_bet(bet, player, round) tuple
+place_bet(bet_request) Bet
+cancel_bet(bet_id, player_id) bool
+resolve_round(round, result) list~BetResult~
+expand_call_bet(call_type, amount, variant) list~Bet~
}

class WheelController {
+spin(variant: str) SpinResult
+get_number_info(number: int) NumberInfo
+get_neighbors(number, count, variant) list~int~
}

class PlayerManager {
+get_balance(player_id) int
+deduct_balance(player_id, amount) bool
+credit_balance(player_id, amount) bool
+get_player(player_id) Player
}

class Bet {
+id: str
+player_id: str
+round_id: str
+type: BetType
+numbers: list~int~
+amount: int
+status: BetStatus
+placed_at: datetime
}

class SpinResult {
+id: str
+round_id: str
+number: int
+color: str
+parity: str
+range: str
+dozen: int
+column: int
+wheel_position: int
+generated_at: datetime
}

class BetResult {
+bet_id: str
+player_id: str
+is_winner: bool
+payout_multiplier: int
+win_amount: int
+net_result: int
}

class Player {

```



```

+id: str
+display_name: str
+phone: str
+balance: int
+created_at: datetime
}

GameManager "1" --> "1" GameRound : manages
GameManager "1" --> "1" Table : operates on
GameManager "1" --> "1" BetManager : uses
GameManager "1" --> "1" WheelController : uses
GameManager "1" --> "1" PlayerManager : uses
GameRound "1" --> "*" Bet : contains
GameRound "1" --> "0..1" SpinResult : has
BetManager "1" --> "*" BetResult : produces
Bet "1" --> "1" BetResult : resolved to
Player "1" --> "*" Bet : places

```

### 3.13 Complexity & Risk Assessment

| Component                    | Complexity | Risk Level | Key Risk                                          |
|------------------------------|------------|------------|---------------------------------------------------|
| State machine                | Medium     | Low        | Well-defined transitions, easy to test            |
| RNG (secrets module)         | Low        | Low        | Standard library, proven                          |
| Bet validation (inside bets) | High       | Medium     | Many edge cases (adjacency, grid math)            |
| Call bet expansion           | Medium     | Low        | Fixed decompositions, well-documented             |
| Win resolution               | Low        | Low        | Simple multiplication, formula-based              |
| Balance management           | Medium     | High       | Race conditions, double-spend, transaction safety |
| Multiplayer sync             | High       | High       | Concurrency, timer sync, disconnection handling   |
| WebSocket management         | Medium     | Medium     | Connection lifecycle, reconnection, scaling       |
| La Partage rule              | Low        | Low        | Simple conditional in resolution                  |
| Backoffice table CRUD        | Low        | Low        | Standard admin operations                         |

*End of Section 3*

## Section 4 — Backend Architecture (Platform Core + Game Engines)

### 4.1 Purpose of This Section

This section defines the **complete technical architecture** of the backend platform. The architecture is designed to support **multiple games** (Roulette V1, Blackjack, Poker, Slots planned within 6 months) while remaining **simple to deploy and operate** for a 1-2 developer team.

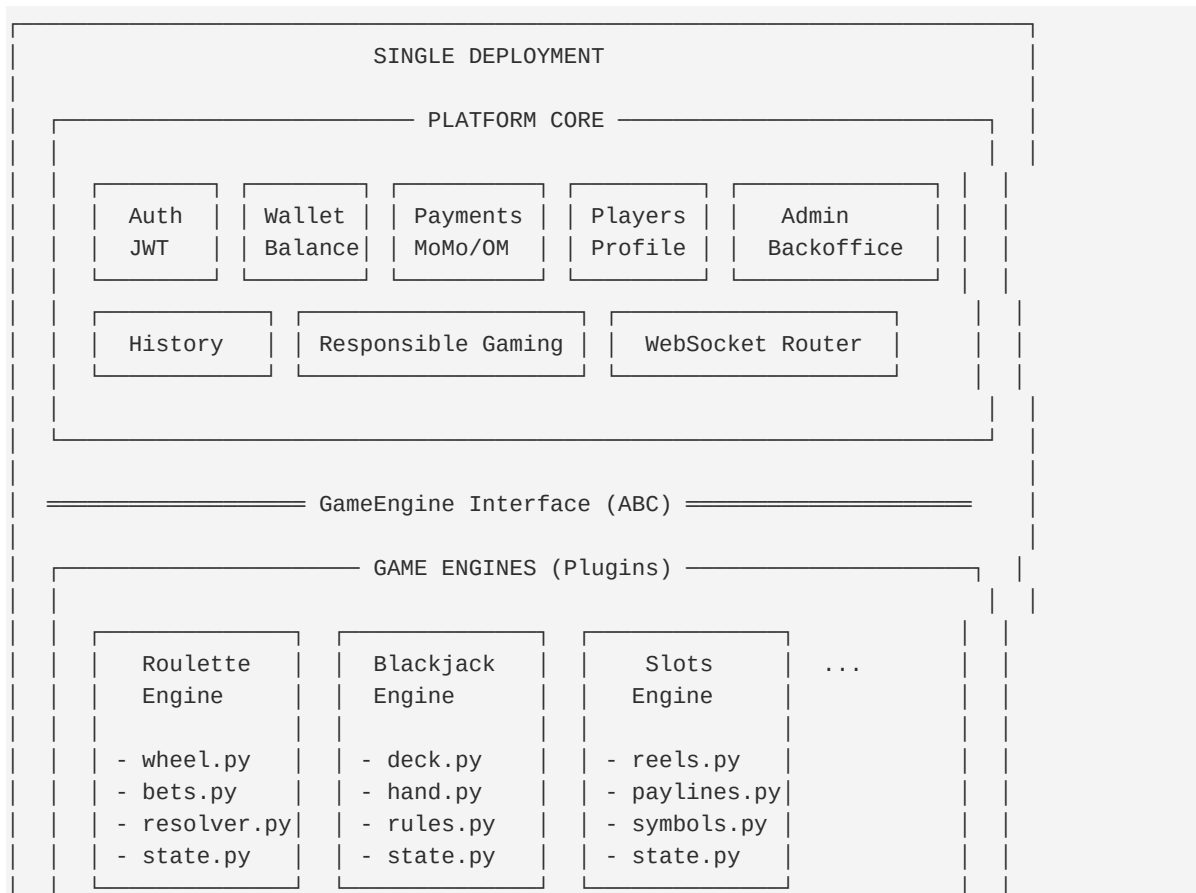
**This section must be fully validated before any code is written.** The database schema and the Game Engine interface are the two foundations — all application code depends on them.

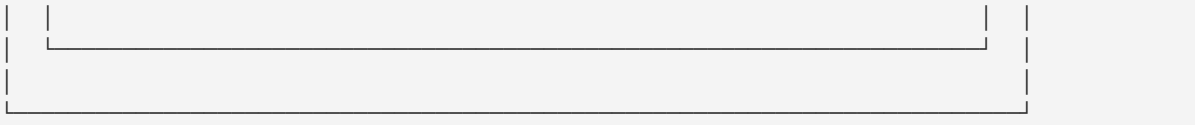
### 4.2 Architecture Pattern: Platform Core + Game Engine Plugins

#### 4.2.1 Design Philosophy

The system is a **single deployable application** with two distinct layers:

1. **Platform Core** — Shared services used by ALL games: authentication, wallet, payments, player profiles, admin backoffice, responsible gaming, history. Written once, never duplicated.
1. **Game Engines** — Self-contained modules, one per game type. Each implements a common **GameEngine** interface. Isolated from each other — Roulette knows nothing about Blackjack.





### 4.2.2 Key Benefits

| Benefit                         | How                                                                 |
|---------------------------------|---------------------------------------------------------------------|
| Add new game in days, not weeks | Create new module, implement interface, register it — done          |
| 1 deployment, 1 database        | No inter-service communication, no distributed transactions         |
| Shared wallet across all games  | Player buys chips once, plays any game                              |
| Single backoffice for all games | Admin manages tables, players, payments from one place              |
| Future microservice extraction  | Each game engine has zero dependencies on others — clean extraction |
| Low DevOps overhead             | 1 Docker container + PostgreSQL + Redis                             |

### 4.2.3 Future Extraction Path

When the team grows (5+ devs, squads per game), each game engine can be extracted:

V1 (Now): Single app → all games in /app/games/  
 V2 (Later): Platform Core stays monolith  
     Each game engine → separate service + own DB schema  
     Communication: REST/gRPC between platform and game services  
     Wallet remains centralized (platform core)

No code rewrite needed — the **GameEngine** interface becomes the API contract.

## 4.3 Technology Stack

### 4.3.1 Stack Decisions (Locked)

| Layer         | Technology | Version (LTS/Stable) | Justification                                                   |
|---------------|------------|----------------------|-----------------------------------------------------------------|
| Language      | Python     | 3.12+                | Team familiarity, strong async ecosystem, FastAPI compatibility |
| Web Framework | FastAPI    | 0.115+               | Async-native, auto OpenAPI docs, Pydantic validation, high perf |
| ASGI Server   | Uvicorn    | 0.30+                | Production-grade ASGI server, works with FastAPI out of the box |

| Layer              | Technology                    | Version (LTS/Stable) | Justification                                                      |
|--------------------|-------------------------------|----------------------|--------------------------------------------------------------------|
| WebSocket          | FastAPI WebSocket (Starlette) | Built-in             | Native integration, no extra dependency                            |
| ORM                | SQLAlchemy 2.0                | 2.0+                 | Async support, mature, type-safe with mapped_column                |
| Migrations         | Alembic                       | 1.13+                | Standard for SQLAlchemy, versioned migrations                      |
| Database           | PostgreSQL                    | 16+                  | ACID transactions, row-level locking, JSONB for game configs       |
| Cache / Pub-Sub    | Redis                         | 7+                   | Session cache, WebSocket pub-sub across workers, rate limiting     |
| Task Queue         | Celery + Redis broker         | 5.4+                 | Async background tasks (payment reconciliation, stats aggregation) |
| Auth               | PyJWT + bcrypt                | Stable               | JWT token generation, password hashing                             |
| SMS OTP            | Twilio or local provider      | API-based            | Phone verification for Cameroon numbers                            |
| Payment Aggregator | CinetPay / Flutterwave        | API-based            | MTN MoMo + Orange Money integration for Cameroon                   |
| Validation         | Pydantic v2                   | 2.5+                 | Request/response validation, built into FastAPI                    |
| Testing            | pytest + pytest-asyncio       | 8.0+ / 0.23+         | Async test support, fixtures, parametrize                          |
| API Documentation  | OpenAPI (auto-generated)      | 3.1                  | FastAPI auto-generates from Pydantic models                        |
| Containerization   | Docker + Docker Compose       | 24+                  | Reproducible environments, easy deployment                         |
| Reverse Proxy      | Nginx                         | 1.26+                | TLS termination, static files, WebSocket proxy                     |

#### 4.3.2 What We Explicitly Do NOT Use

| Technology | Reason for exclusion                        |
|------------|---------------------------------------------|
| Django     | Overhead of full MVC framework unnecessary; |

| Technology       | Reason for exclusion                                               |
|------------------|--------------------------------------------------------------------|
|                  | FastAPI is leaner                                                  |
| MongoDB          | Financial transactions require ACID guarantees<br>→ PostgreSQL     |
| Socket.IO        | Adds abstraction layer; native WebSocket is sufficient and simpler |
| GraphQL          | REST + WebSocket covers all needs; adds unnecessary complexity     |
| Kafka / RabbitMQ | Overkill for V1 scale; Redis pub-sub is sufficient                 |
| Kubernetes       | 1-2 devs, Docker Compose is sufficient for V1 scale                |
| Microservices    | Team size and DevOps maturity don't justify the overhead yet       |

## 4.4 Game Engine Interface

### 4.4.1 Abstract Base Class

This is the **contract** every game must implement. The Platform Core interacts with games **ONLY** through this interface.

```
python
# app/platform/game_engine/base.py

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any, Optional
from enum import Enum

class GameType(str, Enum):
    ROULETTE = "ROULETTE"
    BLACKJACK = "BLACKJACK"
    SLOTS = "SLOTS"
    POKER = "POKER"

class GenericGamePhase(str, Enum):
    """Universal phases that the platform understands."""
    WAITING = "WAITING"
    ACCEPTING_ACTIONS = "ACCEPTING_ACTIONS" # Betting, dealing, etc.
    PROCESSING = "PROCESSING" # Spinning, dealing, resolving
    SHOWING_RESULTS = "SHOWING_RESULTS"
    COMPLETED = "COMPLETED"

@dataclass
class BetRequest:
    """Generic bet request – game engines interpret the details."""
```

```

    player_id: str
    bet_type: str          # Game-specific (e.g., "STRAIGHT_UP" for roulette)
    amount: int            # Chip amount
    parameters: dict       # Game-specific params (e.g., {"numbers": [17]} for
roulette)

@dataclass
class BetResponse:
    success: bool
    bet_id: Optional[str] = None
    error_code: Optional[str] = None
    error_message: Optional[str] = None
    new_balance: Optional[int] = None

@dataclass
class RoundResult:
    """Result of a completed round – returned to platform for history storage."""
    round_id: str
    table_id: str
    game_type: str
    result_data: dict      # Game-specific (e.g., {"number": 17, "color": "BLACK"})
    player_results: list[dict] # Per-player outcomes: [{player_id, total_wagered,
total_won, net}]
    timestamp: str

@dataclass
class GameStateSnapshot:
    """Current state – sent to clients via WebSocket."""
    table_id: str
    game_type: str
    phase: GenericGamePhase
    time_remaining: Optional[float]
    public_data: dict      # Visible to all players (e.g., last results, table info)
    private_data: dict     # Per-player data (indexed by player_id)

class GameEngine(ABC):
    """
    Abstract interface that every game must implement.

    RULES:
    - Game engines NEVER access the wallet directly.
      They call self.platform.debit_player() and self.platform.credit_player().
    - Game engines NEVER manage authentication.
      Player identity is provided by the platform.
    - Game engines CAN have their own database tables (prefixed with game name).
    - Game engines MUST emit events through self.platform.emit_event().
    """

    def __init__(self, platform: "PlatformServices"):
        self.platform = platform

```

```

# — Identity —————

@abstractmethod
def get_game_type(self) -> GameType:
    """Return the game type identifier."""
    ...

@abstractmethod
def get_display_name(self) -> str:
    """Human-readable name (e.g., 'European Roulette')."""
    ...

# — Table Lifecycle —————

@abstractmethod
async def initialize_table(self, table_id: str, config: dict) -> None:
    """
    Called when a table is created or server starts.
    `config` is the game-specific JSON config from the database.
    Load configuration, set up initial state.
    """
    ...

@abstractmethod
async def shutdown_table(self, table_id: str) -> None:
    """Called when table is deactivated or server shuts down. Cleanup."""
    ...

# — Player Lifecycle —————

@abstractmethod
async def on_player_join(self, table_id: str, player_id: str) -> dict:
    """
    Player joins table. Return initial state for this player.
    Raise GameError if table is full or player can't join.
    """
    ...

@abstractmethod
async def on_player_leave(self, table_id: str, player_id: str) -> None:
    """Player leaves table. Handle cleanup (active bets stay valid)."""
    ...

# — Game Actions —————

@abstractmethod
async def place_bet(self, table_id: str, bet_request: BetRequest) -> BetResponse:
    """
    Validate and place a bet. Must call self.platform.debit_player()
    for the bet amount if valid.
    """
    ...

```

```

@abstractmethod
async def cancel_bet(self, table_id: str, player_id: str, bet_id: str) -> bool:
    """
    Cancel a bet. Must call self.platform.credit_player()
    to refund. Return False if cancellation not allowed.
    """
    ...

@abstractmethod
async def player_action(self, table_id: str, player_id: str, action: str, data:
dict) -> dict:
    """
    Handle game-specific actions beyond betting.
    Roulette: "spin" (solo mode)
    Blackjack: "hit", "stand", "double_down", "split"
    Poker: "fold", "call", "raise"
    Slots: "spin"
    """
    ...

# — Round Lifecycle —————

@abstractmethod
async def start_round(self, table_id: str) -> None:
    """Begin a new round. Transition to ACCEPTING_ACTIONS phase."""
    ...

@abstractmethod
async def get_state(self, table_id: str, player_id: Optional[str] = None) ->
GameStateSnapshot:
    """
    Get current game state. If player_id is provided,
    include their private data (their bets, hand, etc.).
    """
    ...

# — Configuration —————

@abstractmethod
def get_config_schema(self) -> dict:
    """
    Return JSON Schema for this game's table configuration.
    Used by the backoffice to render game-specific config forms.
    Example for Roulette: {variant, la_partage, betting_duration, ...}
    Example for Blackjack: {deck_count, dealer_stands_on, ...}
    """
    ...

@abstractmethod
def validate_config(self, config: dict) -> tuple[bool, Optional[str]]:
    """Validate a game-specific configuration. Return (valid, error_message)."""
    ...

```



## 4.4.2 Platform Services (What the Platform Gives to Game Engines)

```
python
# app/platform/game_engine/platform_services.py

class PlatformServices:
    """
    Interface that the Platform Core exposes to Game Engines.
    Game engines use this to interact with shared services.
    """

    async def debit_player(self, player_id: str, amount: int,
                           reference_type: str, reference_id: str) -> int:
        """
        Deduct chips from player wallet. Returns new balance.
        Raises InsufficientBalanceError if not enough chips.
        Creates wallet transaction record automatically.
        """
        ...

    async def credit_player(self, player_id: str, amount: int,
                            reference_type: str, reference_id: str,
                            tx_type: str = "BET_WON") -> int:
        """
        Add chips to player wallet. Returns new balance.
        Creates wallet transaction record automatically.
        """
        ...

    async def get_player_balance(self, player_id: str) -> int:
        """Get current chip balance."""
        ...

    async def emit_event(self, table_id: str, event: str, data: dict,
                         target_player_id: Optional[str] = None) -> None:
        """
        Emit a WebSocket event.
        If target_player_id is None → broadcast to all players at table.
        If target_player_id is set → send only to that player.
        """
        ...

    async def save_round_result(self, result: RoundResult) -> None:
        """Save a completed round result to history."""
        ...

    async def get_table_config(self, table_id: str) -> dict:
        """Retrieve the game-specific config for a table."""
        ...

    def get_db_session(self):
        """Get a database session for game-specific queries."""
        ...
```

### 4.4.3 Game Registry

```
python
# app/platform/game_engine/registry.py

class GameRegistry:
    """
    Singleton registry of all available game engines.
    Game engines register themselves at application startup.
    """

    _engines: dict[GameType, type[GameEngine]] = {}
    _instances: dict[str, GameEngine] = {} # table_id → engine instance

    @classmethod
    def register(cls, game_type: GameType, engine_class: type[GameEngine]):
        """Register a game engine class for a game type."""
        cls._engines[game_type] = engine_class

    @classmethod
    def get_engine_class(cls, game_type: GameType) -> type[GameEngine]:
        """Get the engine class for a game type."""
        if game_type not in cls._engines:
            raise ValueError(f"No engine registered for {game_type}")
        return cls._engines[game_type]

    @classmethod
    async def get_or_create_instance(
        cls, table_id: str, game_type: GameType, platform: PlatformServices, config:
dict
    ) -> GameEngine:
        """Get or create an engine instance for a specific table."""
        if table_id not in cls._instances:
            engine_class = cls.get_engine_class(game_type)
            engine = engine_class(platform)
            await engine.initialize_table(table_id, config)
            cls._instances[table_id] = engine
        return cls._instances[table_id]

    @classmethod
    async def remove_instance(cls, table_id: str):
        """Remove and shutdown an engine instance."""
        if table_id in cls._instances:
            await cls._instances[table_id].shutdown_table(table_id)
            del cls._instances[table_id]

    @classmethod
    def list_game_types(cls) -> list[GameType]:
        """List all registered game types."""
        return list(cls._engines.keys())
```

### 4.4.4 Roulette Engine Registration (Example)

```
python
```

```
# app/games/roulette/__init__.py

from app.platform.game_engine.registry import GameRegistry
from app.platform.game_engine.base import GameType
from app.games.roulette.engine import RouletteEngine

# Auto-register when module is imported
GameRegistry.register(GameType.ROULETTE, RouletteEngine)
```

```
python
```

```
# app/main.py (startup)

@app.on_event("startup")
async def startup():
    # Import game modules to trigger registration
    import app.games.roulette # Registers RouletteEngine
    # import app.games.blackjack # Future: registers BlackjackEngine
    # import app.games.slots # Future: registers SlotsEngine
```

## 4.5 Project Structure

```
casino-platform/
├── alembic/                                # Database migrations
│   ├── versions/
│   ├── env.py
│   └── alembic.ini
├── app/
│   ├── __init__.py
│   └── main.py                            # FastAPI app factory, startup/shutdown, game
registration
│   ├── config.py                         # Settings (env-based, Pydantic BaseSettings)
│   ├── database.py                       # SQLAlchemy async engine, session factory
│   └── dependencies.py                   # FastAPI DI (get_db, get_current_user,
get_current_admin)
│   ├── exceptions.py                    # Custom exceptions + global handlers
│   ├── middleware.py                    # CORS, rate limiting, request logging
│   └── platform/                         # ===== PLATFORM CORE =====
│       ├── __init__.py
│       └── auth/                         # Authentication (players)
│           ├── __init__.py
│           └── router.py                 # POST /auth/register, /auth/login,
/auth/refresh, /auth/otp
│       ├── service.py                   # AuthService
│       └── schemas.py                   # Pydantic: RegisterRequest, LoginResponse,
etc.
│       ├── security.py                  # JWT create/verify, password hash/verify
│       └── otp.py                       # SMS OTP send/verify
│       └── players/                     # Player profiles
```

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| └─ __init__.py               |                                                  |
| └─ router.py                 | # GET/PUT /players/me                            |
| └─ service.py                | # PlayerService                                  |
| └─ schemas.py                |                                                  |
| └─ models.py                 | # Player SQLAlchemy model                        |
| └─ wallet/                   | # Chip balance & transactions                    |
| └─ __init__.py               |                                                  |
| └─ router.py                 | # GET /wallet/balance, /wallet/transactions      |
| └─ service.py                | # WalletService (debit, credit, get_balance)     |
| └─ schemas.py                |                                                  |
| └─ models.py                 | # Wallet, WalletTransaction                      |
| └─ payments/                 | # Payment processing                             |
| └─ __init__.py               |                                                  |
| └─ router.py                 | # POST /payments/initiate, POST                  |
| /payments/webhook/{provider} |                                                  |
| └─ service.py                | # PaymentService                                 |
| └─ schemas.py                |                                                  |
| └─ models.py                 | # PaymentOrder, ChipPackage                      |
| └─ providers/                |                                                  |
| └─ __init__.py               |                                                  |
| └─ base.py                   | # Abstract PaymentProvider                       |
| └─ cinetpay.py               |                                                  |
| └─ flutterwave.py            |                                                  |
| └─ tasks.py                  | # Celery: reconciliation, timeout                |
| └─ tables/                   | # Table management (shared across all games)     |
| └─ __init__.py               |                                                  |
| └─ router.py                 | # GET /tables, GET /tables/{id} (player-facing)  |
| └─ service.py                | # TableService                                   |
| └─ schemas.py                |                                                  |
| └─ models.py                 | # GameTable (with game_type + game_config JSONB) |
| └─ history/                  | # Cross-game history & stats                     |
| └─ __init__.py               |                                                  |
| └─ router.py                 | # GET /history, GET /statistics                  |
| └─ service.py                |                                                  |
| └─ schemas.py                |                                                  |
| └─ models.py                 | # GameRoundHistory (generic)                     |
| └─ responsible_gaming/       |                                                  |
| └─ __init__.py               |                                                  |
| └─ router.py                 |                                                  |
| └─ service.py                |                                                  |
| └─ models.py                 | # SpendingLimit, SelfExclusion                   |
| └─ admin/                    | # Backoffice                                     |
| └─ __init__.py               |                                                  |
| └─ router.py                 | # CRUD tables, packages, players, dashboard      |
| └─ service.py                |                                                  |
| └─ schemas.py                |                                                  |
| └─ auth.py                   | # Admin JWT (separate from player)               |
| └─ models.py                 | # AdminUser, AuditLog                            |

|                             |                                                |
|-----------------------------|------------------------------------------------|
| └─ websocket/               | # WebSocket infrastructure                     |
| └─ ── __init__.py           |                                                |
| └─ ── router.py             | # WS /ws/game/{table_id}                       |
| └─ ── connection_manager.py | # Connection registry, broadcast               |
| └─ ── message_handler.py    | # Routes WS messages → game engine             |
| └─ ── protocol.py           | # Message envelope, event definitions          |
| └─ game_engine/             | # Game Engine framework                        |
| └─ ── __init__.py           |                                                |
| └─ ── base.py               | # GameEngine ABC, dataclasses                  |
| └─ ── registry.py           | # GameRegistry singleton                       |
| └─ ── platform_services.py  | # PlatformServices implementation              |
| └─ games/                   | # ===== GAME ENGINES (Plugins) =====           |
| └─ ── __init__.py           |                                                |
| └─ ── roulette/             | # Roulette game engine                         |
| └─ ── ── __init__.py        | # Registers RouletteEngine                     |
| └─ ── ── engine.py          | # RouletteEngine(GameEngine) – main            |
| orchestrator                |                                                |
| └─ ── constants.py          | # Wheel orders, colors, payouts, RED/BLACK     |
| sets                        |                                                |
| └─ ── wheel.py              | # WheelController: RNG, number info, neighbors |
| └─ ── bet_manager.py        | # Bet validation, placement, cancellation      |
| └─ ── validators.py         | # Bet structure validation (split adjacency,   |
| etc.)                       |                                                |
| └─ ── resolver.py           | # Win/loss resolution, La Partage              |
| └─ ── state.py              | # RouletteRoundState, phase management         |
| └─ ── timer.py              | # Round timer management                       |
| └─ ── call_bets.py          | # Call bet expansion (Voisins, Tiers, etc.)    |
| └─ ── schemas.py            | # Roulette-specific Pydantic models            |
| └─ ── models.py             | # Roulette-specific DB models (roulette_bet,   |
| roulette_spin_result)       |                                                |
| └─ ── config_schema.py      | # JSON Schema for roulette table config        |
| └─ _template/               | # Template for new game engines                |
| └─ ── __init__.py           |                                                |
| └─ ── engine.py             | # Skeleton implementing GameEngine             |
| └─ ── README.md             | # How to create a new game engine              |
| └─ shared/                  | # Shared utilities                             |
| └─ ── __init__.py           |                                                |
| └─ ── redis.py              | # Redis client                                 |
| └─ ── utils.py              | # UUID gen, timestamps, etc.                   |
| └─ tests/                   |                                                |
| └─ ── conftest.py           | # Fixtures (test DB, test client, auth)        |
| └─ ── platform/             |                                                |
| └─ ── ── test_auth.py       |                                                |
| └─ ── ── test_wallet.py     |                                                |
| └─ ── ── test_payments.py   |                                                |
| └─ ── ── test_tables.py     |                                                |
| └─ ── ── test_websocket.py  |                                                |

```

├── games/
│   ├── roulette/
│   │   ├── test_wheel.py
│   │   ├── test_validators.py
│   │   ├── test_bet_manager.py
│   │   ├── test_resolver.py
│   │   ├── test_call_bets.py
│   │   ├── test_state_machine.py
│   │   └── test_engine_integration.py
│   └── integration/
│       ├── test_full_round.py           # Complete game round E2E
│       └── test_payment_flow.py         # Purchase → play → verify balance
├── scripts/
│   ├── seed_tables.py
│   ├── seed_admin.py
│   ├── seed_packages.py
│   └── create_game_engine.py           # Scaffold a new game engine from template
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── requirements-dev.txt
├── .env.example
└── README.md

```

## 4.6 WebSocket Router (Multi-Game)

### 4.6.1 Single Entry Point, Game-Specific Routing

All games share the same WebSocket endpoint. The platform routes messages to the correct game engine based on the table's `game_type`.

```

python
# app/platform/websocket/router.py

from fastapi import WebSocket, WebSocketDisconnect, Depends
from app.platform.game_engine.registry import GameRegistry
from app.platform.tables.service import TableService

@router.websocket("/ws/game/{table_id}")
async def game_websocket(
    websocket: WebSocket,
    table_id: str,
    token: str = Query(...), # JWT in query param
):
    # 1. Authenticate
    player = await authenticate_ws(token)
    if not player:
        await websocket.close(code=4001, reason="Unauthorized")
        return

    # 2. Get table and its game type

```

```

table = await TableService.get_table(table_id)
if not table or not table.is_active:
    await websocket.close(code=4004, reason="Table not found")
    return

# 3. Get the correct game engine for this table
engine = await GameRegistry.get_or_create_instance(
    table_id=table.id,
    game_type=table.game_type,
    platform=platform_services,
    config=table.game_config,
)

# 4. Connect
await connection_manager.connect(table_id, player.id, websocket)
join_data = await engine.on_player_join(table_id, player.id)
await websocket.send_json({
    "event": "table:joined",
    "data": join_data,
})

# 5. Message loop
try:
    while True:
        message = await websocket.receive_json()
        response = await handle_message(engine, table_id, player.id, message)
        if response:
            await websocket.send_json(response)

except WebSocketDisconnect:
    await engine.on_player_leave(table_id, player.id)
    await connection_manager.disconnect(table_id, player.id)

```

#### 4.6.2 Message Handler

```

python
# app/platform/websocket/message_handler.py

async def handle_message(
    engine: GameEngine,
    table_id: str,
    player_id: str,
    message: dict,
) -> Optional[dict]:
    """
    Route incoming WebSocket messages to the appropriate game engine method.
    Message format: {"event": "...", "data": {...}}
    """
    event = message.get("event")
    data = message.get("data", {})

    match event:
        case "bet:place":

```

```

        bet_request = BetRequest(
            player_id=player_id,
            bet_type=data["type"],
            amount=data["amount"],
            parameters=data.get("parameters", {}),
        )
        response = await engine.place_bet(table_id, bet_request)
        if response.success:
            return {"event": "bet:accepted", "data": {
                "betId": response.bet_id,
                "newBalance": response.new_balance,
            }}
        else:
            return {"event": "bet:rejected", "data": {
                "errorCode": response.error_code,
                "message": response.error_message,
            }}

    case "bet:cancel":
        success = await engine.cancel_bet(table_id, player_id, data["bet_id"])
        return {"event": "bet:cancelled" if success else "bet:cancel_failed",
            "data": {}}

    case "game:action":
        result = await engine.player_action(table_id, player_id, data["action"],
            data)
        return {"event": "game:action_result", "data": result}

    case "ping":
        return {"event": "pong", "data": {}}

    case _:
        return {"event": "error", "data": {"message": f"Unknown event: {event}"}}

```

## 4.7 Database Schema

### 4.7.1 Design Principles

The database is split into two domains:

1. **Platform tables** — Shared across all games. Prefixed with nothing (they are the core).
2. **Game-specific tables** — Owned by each game engine. Prefixed with the game name (e.g., `roulette_bet`, `roulette_spin_result`).

This allows each game to define its own data model without polluting the platform schema, while the platform handles all shared concerns (players, wallets, payments).

### 4.7.2 Entity-Relationship Diagram

```

mermaid
erDiagram
    PLAYER {
        uuid id PK
    }

```



```

    varchar phone UK "NOT NULL, +237 format"
    varchar phone_hash UK "SHA256 for indexed lookup"
    varchar display_name UK "3-50 chars"
    varchar email "nullable"
    varchar password_hash "NOT NULL, bcrypt"
    date date_of_birth "NOT NULL, 18+"
    varchar avatar "default"
    boolean is_active "default true"
    boolean is_banned "default false"
    varchar ban_reason "nullable"
    timestamp created_at
    timestamp updated_at
    timestamp last_login_at
}

WALLET {
    uuid id PK
    uuid player_id FK UK "one-to-one"
    bigint balance "CHECK >= 0"
    bigint total_deposited
    bigint total_wagered
    bigint total_won
    bigint bonus_balance "CHECK >= 0"
    timestamp updated_at
}

WALLET_TRANSACTION {
    uuid id PK
    uuid wallet_id FK
    uuid player_id FK
    varchar type "DEPOSIT BET_PLACED BET_WON BET_REFUND BONUS ADMIN_ADJUSTMENT"
    bigint amount "positive or negative"
    bigint balance_before
    bigint balance_after
    varchar reference_type "nullable"
    uuid reference_id "nullable"
    varchar game_type "nullable, which game caused this"
    varchar description "nullable"
    timestamp created_at
}

PAYMENT_ORDER {
    uuid id PK
    uuid player_id FK
    uuid wallet_id FK
    varchar provider "cinetpay flutterwave"
    varchar provider_tx_id "nullable"
    varchar status "PENDING PROCESSING COMPLETED FAILED EXPIRED"
    varchar payment_method "MTN_MOMO ORANGE_MONEY"
    varchar phone_number
    bigint amount_xaf
    bigint chips_base
    bigint chips_bonus
    bigint chips_total

```

```

    uuid package_id FK "nullable"
    jsonb provider_metadata
    timestamp created_at
    timestamp completed_at
    timestamp expires_at
}

CHIP_PACKAGE {
    uuid id PK
    varchar name
    bigint price_xaf
    bigint chips_base
    bigint chips_bonus
    bigint chips_total
    integer sort_order
    boolean is_active
    timestamp created_at
}

GAME_TABLE {
    uuid id PK
    varchar name UK
    varchar game_type "ROULETTE BLACKJACK SLOTS POKER"
    varchar mode "SOLO MULTIPLAYER"
    varchar tier "BEGINNER STANDARD PREMIUM VIP"
    integer min_bet "CHECK >= 1"
    integer max_bet
    integer max_total_bet
    integer max_players "default 8"
    jsonb game_config "game-specific settings"
    boolean is_active "default true"
    timestamp created_at
    timestamp updated_at
}

GAME_ROUND_HISTORY {
    uuid id PK
    uuid table_id FK
    varchar game_type "denormalized for query perf"
    integer round_number "sequential per table"
    jsonb result_data "game-specific result"
    jsonb player_summaries "per-player wagered/won/net"
    integer total_players
    bigint total_wagered
    bigint total_paid_out
    timestamp started_at
    timestamp completed_at
}

TABLE_PLAYER_SESSION {
    uuid id PK
    uuid table_id FK
    uuid player_id FK
    varchar status "ACTIVE SPECTATING LEFT KICKED"

```

```

        timestamp joined_at
        timestamp left_at
    }

    ADMIN_USER {
        uuid id PK
        varchar username UK
        varchar email UK
        varchar password_hash
        varchar role "SUPER_ADMIN ADMIN MODERATOR"
        boolean is_active
        timestamp created_at
        timestamp last_login_at
    }

    AUDIT_LOG {
        uuid id PK
        uuid admin_id FK
        varchar action
        varchar entity_type
        uuid entity_id
        jsonb old_values
        jsonb new_values
        varchar ip_address
        timestamp created_at
    }

    SPENDING_LIMIT {
        uuid id PK
        uuid player_id FK
        varchar period "DAILY WEEKLY MONTHLY"
        bigint limit_amount
        bigint current_spent
        timestamp period_start
        timestamp period_end
        boolean is_active
    }

    SELF_EXCLUSION {
        uuid id PK
        uuid player_id FK
        timestamp excluded_until
        varchar reason
        boolean is_active
        timestamp created_at
    }

%% ===== ROULETTE-SPECIFIC TABLES =====

    ROULETTE_BET {
        uuid id PK
        uuid round_id FK "ref game_round_history"
        uuid player_id FK
        uuid table_id FK

```

```

    varchar bet_type "STRAIGHT_UP SPLIT STREET etc"
    integer[] numbers "covered numbers"
    bigint amount "positive"
    varchar status "ACTIVE CANCELLED WON LOST REFUNDED HALF_RETURNED"
    uuid parent_bet_id FK "nullable, for call bet expansion"
    timestamp placed_at
    timestamp resolved_at
}

```

```

ROULETTE_SPIN_RESULT {
    uuid id PK
    uuid round_id FK UK "one-to-one with game_round_history"
    integer winning_number "-1 for 00"
    varchar winning_color "RED BLACK GREEN"
    varchar winning_parity "EVEN ODD NONE"
    varchar winning_range "LOW HIGH NONE"
    integer winning_dozen "0 1 2 3"
    integer winning_column "0 1 2 3"
    integer wheel_position
    timestamp generated_at
}

```

```

ROULETTE_BET_RESULT {
    uuid id PK
    uuid bet_id FK UK "one-to-one with roulette_bet"
    uuid round_id FK
    uuid player_id FK
    boolean is_winner
    integer payout_multiplier
    bigint win_amount
    bigint net_result
    timestamp resolved_at
}

```

%% ===== RELATIONSHIPS =====

```

PLAYER ||--|| WALLET : has
PLAYER ||--o{ WALLET_TRANSACTION : generates
PLAYER ||--o{ PAYMENT_ORDER : initiates
PLAYER ||--o{ TABLE_PLAYER_SESSION : joins
PLAYER ||--o{ SPENDING_LIMIT : sets
PLAYER ||--o{ SELF_EXCLUSION : requests
PLAYER ||--o{ ROULETTE_BET : places

```

```

WALLET ||--o{ WALLET_TRANSACTION : records
CHIP_PACKAGE ||--o{ PAYMENT_ORDER : used_in

```

```

GAME_TABLE ||--o{ GAME_ROUND_HISTORY : hosts
GAME_TABLE ||--o{ TABLE_PLAYER_SESSION : has

```

```

GAME_ROUND_HISTORY ||--o{ ROULETTE_BET : contains
GAME_ROUND_HISTORY ||--|| ROULETTE_SPIN_RESULT : has
GAME_ROUND_HISTORY ||--o{ ROULETTE_BET_RESULT : produces

```

```

ROULETTE_BET ||--|| ROULETTE_BET_RESULT : resolved_to
ROULETTE_BET ||--o{ ROULETTE_BET : expanded_from

ADMIN_USER ||--o{ AUDIT_LOG : performs

```

### 4.7.3 Key Design Decisions

#### Platform-Level `GAME\_TABLE` with `game\_config` JSONB

```

sql
CREATE TABLE game_table (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(50) NOT NULL UNIQUE,
  game_type   VARCHAR(20) NOT NULL,
  mode        VARCHAR(15) NOT NULL,
  tier         VARCHAR(15) NOT NULL DEFAULT 'STANDARD',
  min_bet     INTEGER NOT NULL CHECK (min_bet >= 1),
  max_bet     INTEGER NOT NULL,
  max_total_bet INTEGER NOT NULL,
  max_players INTEGER NOT NULL DEFAULT 8 CHECK (max_players >= 1),
  game_config JSONB NOT NULL DEFAULT '{}',
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT chk_game_type CHECK (game_type IN ('ROULETTE', 'BLACKJACK', 'SLOTS',
'POKER')),
  CONSTRAINT chk_mode CHECK (mode IN ('SOLO', 'MULTIPLAYER')),
  CONSTRAINT chk_tier CHECK (tier IN ('BEGINNER', 'STANDARD', 'PREMIUM', 'VIP')),
  CONSTRAINT chk_bet_limits CHECK (max_bet > min_bet AND max_total_bet >= max_bet)
);

```

#### Why `game\_config` is JSONB and not separate columns:

- Each game has **different** configuration fields
- Roulette: `{"variant": "EUROPEAN", "la_partage": false, "betting_duration_sec": 45, ...}`
- Blackjack: `{"deck_count": 6, "dealer_stands_on_soft_17": true, "allow_surrender": false, ...}`
- Slots: `{"payline_count": 20, "rtp_target": 0.96, "volatility": "medium", ...}`
- Adding a new game **never** requires a schema migration on `game_table`
- Each game engine validates its own config via `validate_config()` on the `GameEngine` interface

#### Platform-Level `GAME\_ROUND\_HISTORY` with `result\_data` JSONB

```

sql
CREATE TABLE game_round_history (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  table_id    UUID NOT NULL REFERENCES game_table(id),
  game_type   VARCHAR(20) NOT NULL,
  round_number INTEGER NOT NULL,

```

```

    result_data      JSONB NOT NULL,
    player_summaries JSONB NOT NULL,
    total_players    INTEGER NOT NULL DEFAULT 0,
    total_wagered    BIGINT NOT NULL DEFAULT 0,
    total_paid_out   BIGINT NOT NULL DEFAULT 0,
    started_at       TIMESTAMPTZ NOT NULL,
    completed_at     TIMESTAMPTZ,

    CONSTRAINT uq_table_round UNIQUE (table_id, round_number)
);

CREATE INDEX idx_round_table ON game_round_history(table_id, started_at DESC);
CREATE INDEX idx_round_game_type ON game_round_history(game_type);

```

### `result\_data` examples by game:

```

json
// Roulette
{"number": 17, "color": "BLACK", "parity": "ODD", "range": "LOW", "dozen": 2, "column": 2}

// Blackjack
{"dealer_hand": ["K♠", "7♥"], "dealer_total": 17}

// Slots
{"reels": [{"🍒", "🍒", "🍒"}, {"★", "🍌", "🍒"}], "paylines_hit": [1, 5]}

```

### `player\_summaries` example:

```

json
[
  {"player_id": "uuid-1", "total_wagered": 500, "total_won": 3600, "net": 3100},
  {"player_id": "uuid-2", "total_wagered": 200, "total_won": 0, "net": -200}
]

```

## Roulette-Specific Tables (Prefixed)

Game-specific tables are owned by the game engine module. They are created via Alembic migrations but logically belong to the game.

```

sql
-- Roulette bet table
CREATE TABLE roulette_bet (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    round_id          UUID NOT NULL REFERENCES game_round_history(id),
    player_id         UUID NOT NULL REFERENCES player(id),
    table_id          UUID NOT NULL REFERENCES game_table(id),
    bet_type          VARCHAR(25) NOT NULL,
    numbers           INTEGER[] NOT NULL,
    amount            BIGINT NOT NULL CHECK (amount > 0),
    status            VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    parent_bet_id     UUID REFERENCES roulette_bet(id),
    placed_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved_at       TIMESTAMPTZ,

```

```

        CONSTRAINT chk_roulette_bet_status CHECK (status IN (
            'ACTIVE', 'CANCELLED', 'WON', 'LOST', 'REFUNDED', 'HALF_RETURNED'
        ))
    );

CREATE INDEX idx_rbet_round ON roulette_bet(round_id);
CREATE INDEX idx_rbet_player ON roulette_bet(player_id);
CREATE INDEX idx_rbet_round_player ON roulette_bet(round_id, player_id);

-- Roulette spin result
CREATE TABLE roulette_spin_result (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    round_id          UUID NOT NULL UNIQUE REFERENCES game_round_history(id),
    winning_number    INTEGER NOT NULL,
    winning_color      VARCHAR(5) NOT NULL,
    winning_parity     VARCHAR(4) NOT NULL,
    winning_range      VARCHAR(4) NOT NULL,
    winning_dozen       INTEGER NOT NULL,
    winning_column     INTEGER NOT NULL,
    wheel_position     INTEGER NOT NULL,
    generated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT chk_roulette_color CHECK (winning_color IN ('RED', 'BLACK', 'GREEN'))
);

-- Roulette bet result
CREATE TABLE roulette_bet_result (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    bet_id            UUID NOT NULL UNIQUE REFERENCES roulette_bet(id),
    round_id          UUID NOT NULL REFERENCES game_round_history(id),
    player_id         UUID NOT NULL REFERENCES player(id),
    is_winner         BOOLEAN NOT NULL,
    payout_multiplier INTEGER NOT NULL DEFAULT 0,
    win_amount        BIGINT NOT NULL DEFAULT 0,
    net_result        BIGINT NOT NULL,
    resolved_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_rbetresult_round ON roulette_bet_result(round_id);
CREATE INDEX idx_rbetresult_player ON roulette_bet_result(player_id);

```

#### 4.7.4 How Future Games Add Tables

When adding Blackjack, the developer:

1. Creates `app/games/blackjack/models.py` with: - `blackjack_hand` (player hands per round) - `blackjack_round_detail` (dealer hand, deck state)
2. Creates an Alembic migration for these tables
3. The platform tables (`game_table`, `game_round_history`, `wallet_transaction`) are **unchanged**

- A new table in the backoffice is created with `game_type = "BLACKJACK"` and the appropriate `game_config`

No platform schema migration needed to add a new game.

#### 4.7.5 Normalization Notes

| Denormalization                                                              | Reason                                                                |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>game_round_history.game_type</code> (also on <code>game_table</code> ) | Query performance — filter history by game without join               |
| <code>game_round_history.result_data</code> JSONB                            | Each game has different result structures — avoids polymorphic tables |
| <code>game_round_history.player_summaries</code> JSONB                       | Cross-game summary without joining game-specific bet tables           |
| <code>wallet_transaction.game_type</code>                                    | Track revenue per game without joining round tables                   |
| <code>game_table.game_config</code> JSONB                                    | Each game has different config fields — no schema changes per game    |
| <code>roulette_bet.table_id</code> (also derivable from round)               | Query performance — frequent filter by table + player                 |

#### 4.7.6 Full SQL — Platform Tables

```

sql
-- ===== PLATFORM TABLES =====

CREATE TABLE player (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone       VARCHAR(20) NOT NULL UNIQUE,
  phone_hash  VARCHAR(64) NOT NULL UNIQUE,
  display_name VARCHAR(50) NOT NULL UNIQUE,
  email       VARCHAR(255),
  password_hash VARCHAR(255) NOT NULL,
  date_of_birth DATE NOT NULL,
  avatar      VARCHAR(50) DEFAULT 'default',
  is_active   BOOLEAN DEFAULT TRUE,
  is_banned   BOOLEAN DEFAULT FALSE,
  ban_reason  TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_login_at TIMESTAMPTZ,
  CONSTRAINT chk_age CHECK (date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
);
CREATE INDEX idx_player_phone_hash ON player(phone_hash);
CREATE INDEX idx_player_created_at ON player(created_at);

CREATE TABLE wallet (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id   UUID NOT NULL UNIQUE REFERENCES player(id) ON DELETE CASCADE,
  balance     BIGINT NOT NULL DEFAULT 0 CHECK (balance >= 0),
  total_deposited BIGINT NOT NULL DEFAULT 0,

```



```

    total_wagered    BIGINT NOT NULL DEFAULT 0,
    total_won        BIGINT NOT NULL DEFAULT 0,
    bonus_balance    BIGINT NOT NULL DEFAULT 0 CHECK (bonus_balance >= 0),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE wallet_transaction (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wallet_id         UUID NOT NULL REFERENCES wallet(id),
    player_id         UUID NOT NULL REFERENCES player(id),
    type              VARCHAR(30) NOT NULL,
    amount            BIGINT NOT NULL,
    balance_before    BIGINT NOT NULL,
    balance_after     BIGINT NOT NULL,
    reference_type     VARCHAR(30),
    reference_id       UUID,
    game_type         VARCHAR(20),
    description        TEXT,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_tx_type CHECK (type IN (
        'DEPOSIT', 'BET_PLACED', 'BET_WON', 'BET_REFUND', 'BONUS', 'ADMIN_ADJUSTMENT'
    ))
);

CREATE INDEX idx_tx_wallet ON wallet_transaction(wallet_id);
CREATE INDEX idx_tx_player ON wallet_transaction(player_id);
CREATE INDEX idx_tx_created ON wallet_transaction(created_at);
CREATE INDEX idx_tx_game_type ON wallet_transaction(game_type);

CREATE TABLE chip_package (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name              VARCHAR(50) NOT NULL,
    price_xaf         BIGINT NOT NULL CHECK (price_xaf > 0),
    chips_base        BIGINT NOT NULL CHECK (chips_base > 0),
    chips_bonus       BIGINT NOT NULL DEFAULT 0,
    chips_total       BIGINT NOT NULL,
    sort_order        INTEGER NOT NULL DEFAULT 0,
    is_active         BOOLEAN DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE payment_order (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    player_id         UUID NOT NULL REFERENCES player(id),
    wallet_id         UUID NOT NULL REFERENCES wallet(id),
    provider          VARCHAR(30) NOT NULL,
    provider_tx_id     VARCHAR(255),
    status            VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    payment_method     VARCHAR(30) NOT NULL,
    phone_number       VARCHAR(20) NOT NULL,
    amount_xaf        BIGINT NOT NULL CHECK (amount_xaf > 0),
    chips_base        BIGINT NOT NULL,
    chips_bonus       BIGINT NOT NULL DEFAULT 0,

```

```

    chips_total      BIGINT NOT NULL,
    package_id       UUID REFERENCES chip_package(id),
    currency         VARCHAR(5) NOT NULL DEFAULT 'XAF',
    provider_metadata JSONB,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at     TIMESTAMPTZ,
    expires_at       TIMESTAMPTZ NOT NULL,
    CONSTRAINT chk_pay_status CHECK (status IN (
        'PENDING', 'PROCESSING', 'COMPLETED', 'FAILED', 'EXPIRED', 'REFUNDED'
    )),
    CONSTRAINT chk_pay_method CHECK (payment_method IN ('MTN_MOMO', 'ORANGE_MONEY'))
);
CREATE INDEX idx_pay_player ON payment_order(player_id);
CREATE INDEX idx_pay_status ON payment_order(status);
CREATE UNIQUE INDEX idx_pay_provider_tx ON payment_order(provider, provider_tx_id)
    WHERE provider_tx_id IS NOT NULL;

CREATE TABLE game_table (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name              VARCHAR(50) NOT NULL UNIQUE,
    game_type         VARCHAR(20) NOT NULL,
    mode              VARCHAR(15) NOT NULL,
    tier               VARCHAR(15) NOT NULL DEFAULT 'STANDARD',
    min_bet           INTEGER NOT NULL CHECK (min_bet >= 1),
    max_bet           INTEGER NOT NULL,
    max_total_bet     INTEGER NOT NULL,
    max_players       INTEGER NOT NULL DEFAULT 8 CHECK (max_players >= 1),
    game_config       JSONB NOT NULL DEFAULT '{}',
    is_active         BOOLEAN DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_game_type CHECK (game_type IN ('ROULETTE', 'BLACKJACK', 'SLOTS',
'POKER')),
    CONSTRAINT chk_mode CHECK (mode IN ('SOLO', 'MULTIPLAYER')),
    CONSTRAINT chk_tier CHECK (tier IN ('BEGINNER', 'STANDARD', 'PREMIUM', 'VIP')),
    CONSTRAINT chk_bet_limits CHECK (max_bet > min_bet AND max_total_bet >= max_bet)
);

CREATE TABLE game_round_history (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    table_id          UUID NOT NULL REFERENCES game_table(id),
    game_type         VARCHAR(20) NOT NULL,
    round_number      INTEGER NOT NULL,
    result_data       JSONB NOT NULL DEFAULT '{}',
    player_summaries  JSONB NOT NULL DEFAULT '[]',
    total_players     INTEGER NOT NULL DEFAULT 0,
    total_wagered     BIGINT NOT NULL DEFAULT 0,
    total_paid_out    BIGINT NOT NULL DEFAULT 0,
    started_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at      TIMESTAMPTZ,
    CONSTRAINT uq_table_round UNIQUE (table_id, round_number)
);
CREATE INDEX idx_rh_table ON game_round_history(table_id, started_at DESC);

```

```

CREATE INDEX idx_rh_game_type ON game_round_history(game_type);

CREATE TABLE table_player_session (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    table_id          UUID NOT NULL REFERENCES game_table(id),
    player_id         UUID NOT NULL REFERENCES player(id),
    status            VARCHAR(15) NOT NULL DEFAULT 'ACTIVE',
    joined_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    left_at           TIMESTAMPTZ,
    CONSTRAINT chk_session_status CHECK (status IN ('ACTIVE', 'SPECTATING', 'LEFT',
'KICKED'))
);
CREATE INDEX idx_tps_table ON table_player_session(table_id);
CREATE INDEX idx_tps_player ON table_player_session(player_id);

CREATE TABLE admin_user (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username          VARCHAR(50) NOT NULL UNIQUE,
    email             VARCHAR(255) NOT NULL UNIQUE,
    password_hash     VARCHAR(255) NOT NULL,
    role              VARCHAR(15) NOT NULL DEFAULT 'ADMIN',
    is_active         BOOLEAN DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_login_at     TIMESTAMPTZ,
    CONSTRAINT chk_admin_role CHECK (role IN ('SUPER_ADMIN', 'ADMIN', 'MODERATOR'))
);

CREATE TABLE audit_log (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    admin_id          UUID NOT NULL REFERENCES admin_user(id),
    action            VARCHAR(100) NOT NULL,
    entity_type       VARCHAR(30),
    entity_id         UUID,
    old_values         JSONB,
    new_values         JSONB,
    ip_address        VARCHAR(45),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_audit_admin ON audit_log(admin_id);
CREATE INDEX idx_audit_created ON audit_log(created_at);

CREATE TABLE spending_limit (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    player_id         UUID NOT NULL REFERENCES player(id),
    period            VARCHAR(10) NOT NULL,
    limit_amount      BIGINT NOT NULL CHECK (limit_amount > 0),
    current_spent     BIGINT NOT NULL DEFAULT 0,
    period_start      TIMESTAMPTZ NOT NULL,
    period_end        TIMESTAMPTZ NOT NULL,
    is_active         BOOLEAN DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_limit_period CHECK (period IN ('DAILY', 'WEEKLY', 'MONTHLY'))
);

```

```
CREATE TABLE self_exclusion (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    player_id     UUID NOT NULL REFERENCES player(id),
    excluded_until TIMESTAMPTZ NOT NULL,
    reason        TEXT,
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

## 4.8 Configuration Management

```
python
# app/config.py
from pydantic_settings import BaseSettings
from typing import Optional

class Settings(BaseSettings):
    # — Application —
    APP_NAME: str = "Casino Platform"
    APP_ENV: str = "development"
    DEBUG: bool = False
    SECRET_KEY: str # Required

    # — Database —
    DATABASE_URL: str # postgresql+asyncpg://user:pass@host:5432/dbname
    DB_POOL_SIZE: int = 20
    DB_MAX_OVERFLOW: int = 10

    # — Redis —
    REDIS_URL: str = "redis://localhost:6379/0"

    # — Auth (Player) —
    JWT_SECRET_KEY: str
    JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int = 15
    JWT_REFRESH_TOKEN_EXPIRE_DAYS: int = 30
    BCrypt_ROUNDS: int = 12

    # — Auth (Admin) —
    ADMIN_JWT_SECRET_KEY: str
    ADMIN_TOKEN_EXPIRE_MINUTES: int = 60

    # — SMS OTP —
    OTP_PROVIDER: str = "twilio"
    OTP_EXPIRY_SECONDS: int = 300
    TWILIO_ACCOUNT_SID: Optional[str] = None
    TWILIO_AUTH_TOKEN: Optional[str] = None
    TWILIO_FROM_NUMBER: Optional[str] = None

    # — Payments —
    PAYMENT_PROVIDER: str = "cinetpay"
    CINETPAY_API_KEY: Optional[str] = None
    CINETPAY_SITE_ID: Optional[str] = None
    CINETPAY_SECRET_KEY: Optional[str] = None
```

```

PAYMENT_WEBHOOK_SECRET: Optional[str] = None
PAYMENT_TIMEOUT_MINUTES: int = 5

# — Platform Defaults —
DEFAULT_WELCOME_BONUS: int = 500
MAX_TABLES_PER_PLAYER: int = 3
ENABLED_GAMES: str = "ROULETTE" # Comma-separated: "ROULETTE,BLACKJACK"

class Config:
    env_file = ".env"
    case_sensitive = True

```

## 4.9 Authentication & Authorization

### 4.9.1 JWT Strategy

Same as original — access token (15min) + refresh token (30 days). Admin tokens use separate secret.

### 4.9.2 WebSocket Authentication

Token passed as query parameter during handshake:

```
ws://api.example.com/ws/game/{table_id}?token={access_token}
```

### 4.9.3 Authorization Matrix

| Resource                      | Player | Spectator | Admin | Super Admin |
|-------------------------------|--------|-----------|-------|-------------|
| View table list (any game)    | ✓      | ✓         | ✓     | ✓           |
| Join table (any game)         | ✓      | ✓         | ✓     | ✓           |
| Place bet (any game)          | ✓      | ✗         | ✗     | ✗           |
| View own balance              | ✓      | ✓         | ✗     | ✗           |
| Purchase chips                | ✓      | ✓         | ✗     | ✗           |
| View own history (all games)  | ✓      | ✓         | ✗     | ✗           |
| View any player's data        | ✗      | ✗         | ✓     | ✓           |
| Create/edit tables (any game) | ✗      | ✗         | ✓     | ✓           |
| Manage chip packages          | ✗      | ✗         | ✓     | ✓           |
| Ban/suspend players           | ✗      | ✗         | ✓     | ✓           |
| Manage admin                  | ✗      | ✗         | ✗     | ✓           |

| Resource                  | Player | Spectator | Admin | Super Admin |
|---------------------------|--------|-----------|-------|-------------|
| users                     |        |           |       |             |
| View audit logs           | ✗      | ✗         | ✓     | ✓           |
| Enable/disable game types | ✗      | ✗         | ✗     | ✓           |

## 4.10 Error Handling

### 4.10.1 Exception Hierarchy

```
python
# app/exceptions.py

class AppError(Exception):
    def __init__(self, code: str, message: str, status_code: int = 400):
        self.code = code
        self.message = message
        self.status_code = status_code

class AuthenticationError(AppError):
    def __init__(self, message="Authentication failed"):
        super().__init__("AUTH_ERROR", message, 401)

class AuthorizationError(AppError):
    def __init__(self, message="Insufficient permissions"):
        super().__init__("FORBIDDEN", message, 403)

class NotFoundError(AppError):
    def __init__(self, entity: str, entity_id: str):
        super().__init__("NOT_FOUND", f"{entity} {entity_id} not found", 404)

class GameError(AppError):
    """Game-specific errors (E1001-E1012 for roulette, E2001+ for blackjack, etc.)"""
    def __init__(self, code: str, message: str):
        super().__init__(code, message, 400)

class InsufficientBalanceError(AppError):
    def __init__(self):
        super().__init__("E0001", "Insufficient balance", 400)

class PaymentError(AppError):
    def __init__(self, code: str, message: str):
        super().__init__(code, message, 400)
```

### 4.10.2 Standard Response Format

```
json
// Success
{"success": true, "data": { ... }}

// Error
```

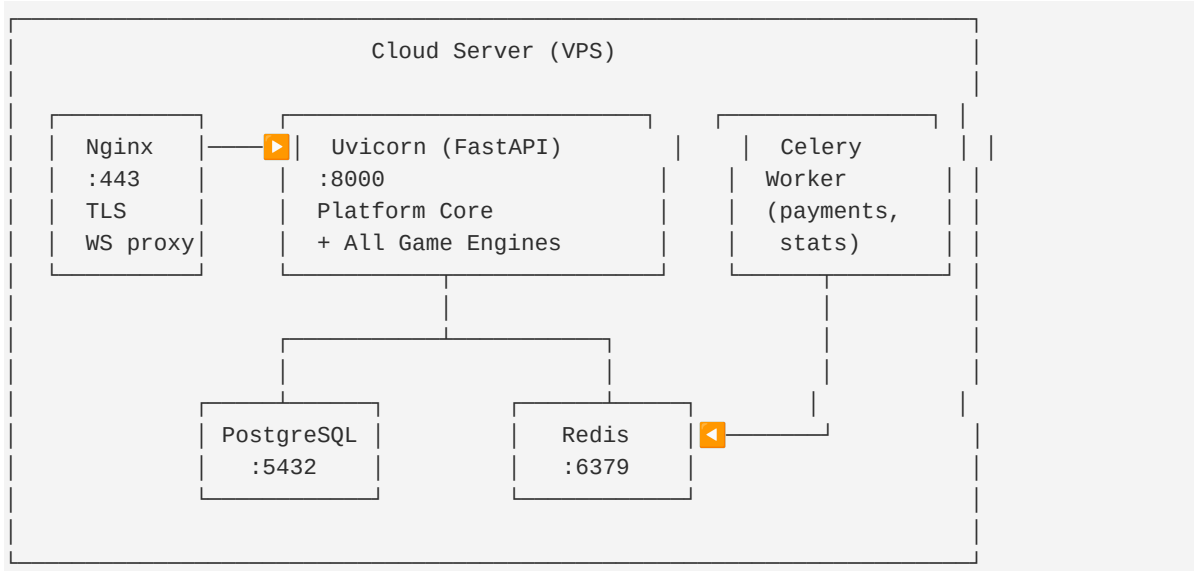
```

{"success": false, "error": {"code": "E1003", "message": "Insufficient balance"}}

// Paginated
{"success": true, "data": [...], "pagination": {"page": 1, "page_size": 20,
"total_items": 156, "total_pages": 8}}

```

## 4.11 Deployment Architecture



### 4.11.1 Docker Compose

```

yaml
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8000:8000"
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./app:/code/app
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  celery:
    build: .
    env_file: .env
    depends_on: [redis, postgres]
    command: celery -A app.platform.payments.tasks worker --loglevel=info

```

```

postgres:
  image: postgres:16-alpine
  environment:
    POSTGRES_DB: casino
    POSTGRES_USER: casino_user
    POSTGRES_PASSWORD: dev_password
  ports: ["5432:5432"]
  volumes: [pgdata:/var/lib/postgresql/data]
  healthcheck:
    test: pg_isready -U casino_user -d casino
    interval: 5s
    retries: 5

redis:
  image: redis:7-alpine
  ports: ["6379:6379"]

volumes:
  pgdata:

```

#### 4.11.2 Scaling Path

| Concern      | V1 (Now)                     | V2 (Scale-Up)                                  |
|--------------|------------------------------|------------------------------------------------|
| Server       | Single VPS (4 vCPU, 8GB RAM) | Multiple VPS + load balancer                   |
| Database     | Single PostgreSQL            | Read replicas + connection pooling (PgBouncer) |
| Redis        | Single instance              | Redis Sentinel                                 |
| WebSocket    | Single worker                | Multi-worker + Redis Pub/Sub                   |
| Game engines | All in same process          | Extract to separate services if needed         |
| SSL/TLS      | Let's Encrypt via Nginx      | CloudFlare                                     |
| Backups      | Daily pg_dump                | WAL archiving, PITR                            |
| Monitoring   | Basic logging                | Prometheus + Grafana                           |
| CI/CD        | Manual Docker deploy         | GitHub Actions → auto deploy                   |

#### 4.12 Rate Limiting

| Endpoint Category     | Rate Limit   | Window     | Scope      |
|-----------------------|--------------|------------|------------|
| Auth (login/register) | 5 requests   | 1 minute   | Per IP     |
| OTP send              | 3 requests   | 5 minutes  | Per phone  |
| Bet placement         | 30 requests  | 10 seconds | Per player |
| Game actions          | 30 requests  | 10 seconds | Per player |
| Payment initiate      | 3 requests   | 1 minute   | Per player |
| General API           | 100 requests | 1 minute   | Per player |



| Endpoint Category  | Rate Limit  | Window     | Scope     |
|--------------------|-------------|------------|-----------|
| Admin API          | 60 requests | 1 minute   | Per admin |
| WebSocket messages | 60 messages | 10 seconds | Per conn  |

## 4.13 Adding a New Game — Step-by-Step Checklist

When a developer wants to add Blackjack (or any game):

- ☐ 1. Copy /app/games/\_template/ to /app/games/blackjack/
- ☐ 2. Implement GameEngine interface in engine.py
- ☐ 3. Define game constants (deck, hand values, etc.)
- ☐ 4. Implement game-specific logic (deal, hit, stand, etc.)
- ☐ 5. Define game-specific DB models in models.py (blackjack\_hand, etc.)
- ☐ 6. Create Alembic migration for new tables
- ☐ 7. Define config\_schema.py (deck\_count, rules, etc.)
- ☐ 8. Register engine in \_\_init\_\_.py: GameRegistry.register(GameType.BLACKJACK, BlackjackEngine)
- ☐ 9. Add "BLACKJACK" to GameType enum and game\_table CHECK constraint
- ☐ 10. Import module in main.py startup
- ☐ 11. Write tests in tests/games/blackjack/
- ☐ 12. Create tables in backoffice with game\_type=BLACKJACK and appropriate game\_config
- ☐ Done – Blackjack is live

**What does NOT change:**

- Platform auth, wallet, payments, player management
- WebSocket infrastructure
- Backoffice (auto-discovers game types via GameRegistry)
- Client-side connection flow (same WebSocket endpoint)
- Deployment (same Docker container)

*End of Section 4 (Revised — Platform Core + Game Engines)*

## Section 5 — API Reference

### 5.1 Purpose of This Section

This section is the **complete API contract** between the backend and the Unity frontend. Every endpoint, every request/response schema, every WebSocket event, and every error code is documented here.

**Audience:** Unity frontend developers, QA testers, and any third-party integration.

**Base URL:** `https://api.{domain}/v1`

**WebSocket URL:** `wss://api.{domain}/v1/ws/game/{table_id}?token={jwt}`

### 5.2 API Conventions

#### 5.2.1 General Rules

| Convention          | Standard                                                                         |
|---------------------|----------------------------------------------------------------------------------|
| Protocol            | HTTPS only (TLS 1.2+)                                                            |
| Content-Type        | <code>application/json</code>                                                    |
| Authentication      | <code>Authorization: Bearer {access_token}</code><br>(except public endpoints)   |
| Date format         | ISO 8601 with timezone: <code>2026-02-11T14:30:00Z</code>                        |
| ID format           | UUID v4: <code>550e8400-e29b-41d4-a716-446655440000</code>                       |
| Currency amounts    | Integer (chips). No decimals. <code>1000</code> = 1,000 chips                    |
| Money amounts (XAF) | Integer. <code>5000</code> = 5,000 XAF                                           |
| Pagination          | Query params: <code>?page=1&amp;page_size=20</code>                              |
| Sorting             | Query param: <code>?sort=-created_at</code> (prefix <code>-</code> = descending) |
| API versioning      | URL path: <code>/v1/</code>                                                      |

#### 5.2.2 Standard Response Envelope

**Success (single object):**

```
json
{
  "success": true,
  "data": { ... }
}
```

**Success (list with pagination):**

```
json
{
  "success": true,
```

```

    "data": [ ... ],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total_items": 156,
      "total_pages": 8
    }
  }
}

```

**Error:**

```

json
{
  "success": false,
  "error": {
    "code": "E1003",
    "message": "Insufficient balance"
  }
}

```

**Validation Error (422):**

```

json
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [
      {"field": "amount", "message": "Value must be greater than 0"},
      {"field": "phone", "message": "Invalid phone number format"}
    ]
  }
}

```

## 5.3 Authentication Endpoints

### POST `/v1/auth/register`

Create a new player account.

**Auth required:** No

**Request:**

```

json
{
  "phone": "+237612345678",
  "display_name": "Lucky237",
  "password": "MySecure8Pass",
  "date_of_birth": "2000-05-15",
  "email": "player@example.com"
}

```

| Field         | Type   | Required | Constraints                                  |
|---------------|--------|----------|----------------------------------------------|
| phone         | string | Yes      | Cameroon format:<br>+237[6-9]XXXXXXX         |
| display_name  | string | Yes      | 3–50 chars, unique,<br>alphanumeric + spaces |
| password      | string | Yes      | Min 8 chars, 1<br>uppercase, 1 digit         |
| date_of_birth | string | Yes      | ISO date, must be 18+<br>years old           |
| email         | string | No       | Valid email format                           |

**Response (201 Created):**

```

json
{
  "success": true,
  "data": {
    "player_id": "uuid",
    "display_name": "Lucky237",
    "otp_sent": true,
    "message": "OTP sent to +237612***678. Please verify your phone."
  }
}

```

**Errors:**

| Code             | Status | Condition                          |
|------------------|--------|------------------------------------|
| PHONE_EXISTS     | 409    | Phone number already<br>registered |
| NAME_EXISTS      | 409    | Display name already taken         |
| UNDERAGE         | 400    | Date of birth indicates under 18   |
| VALIDATION_ERROR | 422    | Invalid field values               |

**POST `/v1/auth/verify-otp`**

Verify phone number with OTP code sent via SMS.

**Auth required:** No

**Request:**

```

json
{
  "phone": "+237612345678",
  "otp_code": "483921"
}

```

**Response (200 OK):**

```

json

```

```
{
  "success": true,
  "data": {
    "verified": true,
    "access_token": "eyJhbGciOi...",
    "refresh_token": "eyJhbGciOi...",
    "token_type": "Bearer",
    "expires_in": 900,
    "player": {
      "id": "uuid",
      "display_name": "Lucky237",
      "phone": "+237612***678",
      "balance": 500
    }
  }
}
```

> **Note:** On successful first verification, the welcome bonus (500 chips) is automatically credited.

#### Errors:

| Code             | Status | Condition                |
|------------------|--------|--------------------------|
| INVALID_OTP      | 400    | Wrong code               |
| OTP_EXPIRED      | 400    | Code expired (5 min)     |
| OTP_MAX_ATTEMPTS | 429    | Too many failed attempts |

## POST `/v1/auth/login`

Authenticate an existing player.

**Auth required:** No

#### Request:

```
json
{
  "phone": "+237612345678",
  "password": "MySecure8Pass"
}
```

#### Response (200 OK):

```
json
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOi...",
    "refresh_token": "eyJhbGciOi...",
    "token_type": "Bearer",
    "expires_in": 900,
    "player": {
      "id": "uuid",
      "display_name": "Lucky237",

```

```

    "phone": "+237612***678",
    "balance": 4350,
    "avatar": "default"
  }
}

```

**Errors:**

| Code                | Status | Condition                        |
|---------------------|--------|----------------------------------|
| INVALID_CREDENTIALS | 401    | Wrong phone or password          |
| ACCOUNT_BANNED      | 403    | Account has been banned          |
| ACCOUNT_INACTIVE    | 403    | Account deactivated              |
| PHONE_NOT_VERIFIED  | 403    | Phone number not yet verified    |
| SELF_EXCLUDED       | 403    | Player has active self-exclusion |

**POST `/v1/auth/refresh`**

Refresh an expired access token.

**Auth required:** No (uses refresh\_token)

**Request:**

```

json
{
  "refresh_token": "eyJhbGciOi..."
}

```

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOi...",
    "token_type": "Bearer",
    "expires_in": 900
  }
}

```

**Errors:**

| Code                  | Status | Condition                |
|-----------------------|--------|--------------------------|
| INVALID_REFRESH_TOKEN | 401    | Token invalid or revoked |
| REFRESH_TOKEN_EXPIRED | 401    | Token expired (30 days)  |

**POST `/v1/auth/request-otp`**

Request a new OTP code (for password reset or re-verification).

**Auth required:** No**Request:**

```
json
{
  "phone": "+237612345678",
  "purpose": "password_reset"
}
```

| Field   | Type   | Required | Values                         |
|---------|--------|----------|--------------------------------|
| phone   | string | Yes      | Registered phone number        |
| purpose | string | Yes      | password_reset or verification |

**Response (200 OK):**

```
json
{
  "success": true,
  "data": {
    "otp_sent": true,
    "expires_in": 300
  }
}
```

## POST `/v1/auth/reset-password`

Reset password using OTP code.

**Auth required:** No**Request:**

```
json
{
  "phone": "+237612345678",
  "otp_code": "483921",
  "new_password": "NewSecure9Pass"
}
```

**Response (200 OK):**

```
json
{
  "success": true,
  "data": {
    "message": "Password reset successfully. Please login."
  }
}
```

## 5.4 Player Endpoints

### GET `/v1/players/me`

Get current player's profile.

**Auth required:** Yes

**Response (200 OK):**

```
json
{
  "success": true,
  "data": {
    "id": "uuid",
    "display_name": "Lucky237",
    "phone": "+237612***678",
    "email": "pla***@example.com",
    "date_of_birth": "2000-05-15",
    "avatar": "avatar_3",
    "balance": 4350,
    "stats": {
      "total_games": 245,
      "total_wagered": 125000,
      "total_won": 118500,
      "net_result": -6500,
      "member_since": "2026-01-15T00:00:00Z",
      "favorite_game": "ROULETTE"
    },
    "created_at": "2026-01-15T10:30:00Z"
  }
}
```

### PUT `/v1/players/me`

Update player profile.

**Auth required:** Yes

**Request:**

```
json
{
  "display_name": "LuckyKing237",
  "avatar": "avatar_5",
  "email": "new_email@example.com"
}
```

All fields are optional — only provided fields are updated.

**Response (200 OK):**

```
json
{
  "success": true,
  "data": {
    "id": "uuid",
```



```

    "display_name": "LuckyKing237",
    "avatar": "avatar_5",
    "email": "new***@example.com"
  }
}

```

### PUT `/v1/players/me/password`

Change password.

**Auth required:** Yes

**Request:**

```

json
{
  "current_password": "MySecure8Pass",
  "new_password": "NewSecure9Pass"
}

```

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {
    "message": "Password changed. All other sessions have been logged out."
  }
}

```

## 5.5 Wallet Endpoints

### GET `/v1/wallet/balance`

Get current chip balance.

**Auth required:** Yes

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {
    "balance": 4350,
    "bonus_balance": 500,
    "total_deposited": 15000,
    "total_wagered": 125000,
    "total_won": 118500
  }
}

```

## GET `/v1/wallet/transactions`

Get transaction history.

**Auth required:** Yes

**Query params:**

| Param     | Type   | Default | Description                                             |
|-----------|--------|---------|---------------------------------------------------------|
| page      | int    | 1       | Page number                                             |
| page_size | int    | 20      | Items per page (max 100)                                |
| type      | string | all     | Filter: DEPOSIT, BET_PLACED, BET_WON, BET_REFUND, BONUS |
| game_type | string | all     | Filter: ROULETTE, BLACKJACK, etc.                       |
| from      | string | —       | ISO date, start of range                                |
| to        | string | —       | ISO date, end of range                                  |

**Response (200 OK):**

```

json
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "type": "BET_WON",
      "amount": 3600,
      "balance_after": 4350,
      "game_type": "ROULETTE",
      "description": "Straight Up #17 – Won 3,600",
      "created_at": "2026-02-11T14:32:00Z"
    },
    {
      "id": "uuid",
      "type": "BET_PLACED",
      "amount": -100,
      "balance_after": 750,
      "game_type": "ROULETTE",
      "description": "Straight Up #17 – 100 chips",
      "created_at": "2026-02-11T14:31:45Z"
    }
  ],
  "pagination": {
    "page": 1,
    "page_size": 20,
    "total_items": 312,
    "total_pages": 16
  }
}

```

```
}
```

## 5.6 Payment Endpoints

### GET `/v1/payments/packages`

List available chip purchase packages.

**Auth required:** Yes

**Response (200 OK):**

```
json
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "Starter",
      "price_xaf": 500,
      "chips_base": 500,
      "chips_bonus": 0,
      "chips_total": 500
    },
    {
      "id": "uuid",
      "name": "Popular",
      "price_xaf": 2000,
      "chips_base": 2000,
      "chips_bonus": 200,
      "chips_total": 2200
    },
    {
      "id": "uuid",
      "name": "Best Value",
      "price_xaf": 5000,
      "chips_base": 5000,
      "chips_bonus": 1000,
      "chips_total": 6000
    }
  ]
}
```

### POST `/v1/payments/initiate`

Initiate a chip purchase via Mobile Money.

**Auth required:** Yes

**Request:**

```
json
{
  "package_id": "uuid",
  "payment_method": "MTN_MOMO",
```

```
{
  "phone_number": "+237612345678"
}
```

| Field          | Type   | Required | Values                      |
|----------------|--------|----------|-----------------------------|
| package_id     | string | Yes      | UUID of chip package        |
| payment_method | string | Yes      | MTN_MOMO or ORANGE_MONEY    |
| phone_number   | string | Yes      | Phone for Mobile Money push |

### Response (201 Created):

```
json
{
  "success": true,
  "data": {
    "order_id": "uuid",
    "status": "PENDING",
    "payment_method": "MTN_MOMO",
    "amount_xaf": 5000,
    "chips_total": 6000,
    "message": "A payment request has been sent to your phone. Please confirm.",
    "expires_at": "2026-02-11T14:37:00Z"
  }
}
```

### Errors:

| Code              | Status | Condition                            |
|-------------------|--------|--------------------------------------|
| PACKAGE_NOT_FOUND | 404    | Invalid package ID                   |
| PACKAGE_INACTIVE  | 400    | Package disabled by admin            |
| PAYMENT_PENDING   | 409    | Player already has a pending payment |
| RATE_LIMITED      | 429    | Too many payment attempts            |

### GET `/v1/payments/orders/{order\_id}`

Check status of a payment order.

**Auth required:** Yes

### Response (200 OK):

```
json
{
  "success": true,
  "data": {
    "order_id": "uuid",
    "status": "COMPLETED",
    "payment_method": "MTN_MOMO",
  }
}
```

```

    "amount_xaf": 5000,
    "chips_credited": 6000,
    "created_at": "2026-02-11T14:32:00Z",
    "completed_at": "2026-02-11T14:33:15Z"
  }
}

```

| Status     | Meaning                                          |
|------------|--------------------------------------------------|
| PENDING    | Payment initiated, waiting for user confirmation |
| PROCESSING | Payment confirmed, being processed               |
| COMPLETED  | Payment successful, chips credited               |
| FAILED     | Payment failed (insufficient funds, cancelled)   |
| EXPIRED    | Payment not confirmed within timeout (5 min)     |

## POST `/v1/payments/webhook/{provider}`

Webhook callback from payment provider. **NOT called by the client** — called by CinetPay/Flutterwave servers.

**Auth required:** No (validated via HMAC signature in headers)

**Headers:**

```
X-Webhook-Signature: hmac_sha256_signature
```

**Request (from CinetPay):**

```

json
{
  "transaction_id": "CP_tx_12345",
  "status": "COMPLETED",
  "amount": 5000,
  "currency": "XAF",
  "metadata": {
    "order_id": "uuid"
  }
}

```

**Response (200 OK):**

```

json
{"received": true}

```

> **Implementation note:** This endpoint is idempotent. Duplicate callbacks with the same `transaction_id` are safely ignored.

## GET `/v1/payments/history`

Get player's purchase history.

**Auth required:** Yes

**Query params:** `page`, `page_size`, `from`, `to`

**Response (200 OK):**

```

json
{
  "success": true,
  "data": [
    {
      "order_id": "uuid",
      "package_name": "Best Value",
      "status": "COMPLETED",
      "payment_method": "MTN_MOMO",
      "amount_xaf": 5000,
      "chips_credited": 6000,
      "created_at": "2026-02-11T14:32:00Z"
    }
  ],
  "pagination": { "page": 1, "page_size": 20, "total_items": 8, "total_pages": 1 }
}

```

## 5.7 Table Endpoints

### GET `/v1/tables`

List available tables (all games).

**Auth required:** Yes

**Query params:**

| Param                  | Type   | Default | Description                      |
|------------------------|--------|---------|----------------------------------|
| <code>game_type</code> | string | all     | ROULETTE, BLACKJACK, etc.        |
| <code>mode</code>      | string | all     | SOLO, MULTIPLAYER                |
| <code>tier</code>      | string | all     | BEGINNER, STANDARD, PREMIUM, VIP |

**Response (200 OK):**

```

json
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "Douala Lounge",
      "game_type": "ROULETTE",
      "mode": "MULTIPLAYER",
      "tier": "STANDARD",
      "min_bet": 50,
      "max_bet": 2000,
      "max_total_bet": 10000,
    }
  ]
}

```

```

    "max_players": 8,
    "current_players": 3,
    "game_config": {
      "variant": "EUROPEAN",
      "la_partage": false
    },
    "current_phase": "BETTING",
    "time_remaining": 32.5,
    "last_results": [
      {"number": 17, "color": "BLACK"},
      {"number": 0, "color": "GREEN"},
      {"number": 23, "color": "RED"}
    ]
  },
  {
    "id": "uuid",
    "name": "Solo Training",
    "game_type": "ROULETTE",
    "mode": "SOLO",
    "tier": "BEGINNER",
    "min_bet": 10,
    "max_bet": 500,
    "max_total_bet": 2000,
    "max_players": 1,
    "current_players": 0,
    "game_config": {
      "variant": "EUROPEAN",
      "la_partage": false
    },
    "current_phase": "WAITING",
    "time_remaining": null,
    "last_results": []
  }
]
}

```

> **Note:** `current_players`, `current_phase`, `time_remaining`, and `last_results` are live data fetched from the game engine, not the database. They may be cached in Redis for performance.

### GET `/v1/tables/{table\_id}`

Get detailed table information.

**Auth required:** Yes

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "Douala Lounge",
    "game_type": "ROULETTE",
    "mode": "MULTIPLAYER",

```

```

    "tier": "STANDARD",
    "min_bet": 50,
    "max_bet": 2000,
    "max_total_bet": 10000,
    "max_players": 8,
    "current_players": 3,
    "players": [
      {"display_name": "Lucky237", "avatar": "avatar_3"},
      {"display_name": "KingBet", "avatar": "avatar_1"},
      {"display_name": "ProGamer", "avatar": "avatar_7"}
    ],
    "game_config": {
      "variant": "EUROPEAN",
      "la_partage": false,
      "betting_duration_sec": 45,
      "closing_duration_sec": 5,
      "spinning_duration_sec": 8,
      "completed_duration_sec": 8
    },
    "current_phase": "BETTING",
    "time_remaining": 28.3,
    "last_results": [
      {"number": 17, "color": "BLACK", "parity": "ODD", "range": "LOW"},
      {"number": 0, "color": "GREEN", "parity": "NONE", "range": "NONE"},
      {"number": 23, "color": "RED", "parity": "ODD", "range": "HIGH"}
    ],
    "statistics": {
      "total_spins": 1247,
      "red_percentage": 48.2,
      "black_percentage": 49.1,
      "green_percentage": 2.7,
      "hot_numbers": [7, 17, 23, 32, 11],
      "cold_numbers": [13, 26, 34, 6, 29]
    }
  }
}

```

## 5.8 Game History Endpoints

### GET `/v1/history/rounds`

Get player's game round history (cross-game).

**Auth required:** Yes

**Query params:**

| Param     | Type   | Default | Description              |
|-----------|--------|---------|--------------------------|
| page      | int    | 1       | Page number              |
| page_size | int    | 20      | Items per page (max 100) |
| game_type | string | all     | Filter by game type      |



| Param    | Type   | Default | Description              |
|----------|--------|---------|--------------------------|
| table_id | string | all     | Filter by specific table |
| from     | string | —       | ISO date, start          |
| to       | string | —       | ISO date, end            |

**Response (200 OK):**

```

json
{
  "success": true,
  "data": [
    {
      "round_id": "uuid",
      "table_name": "Douala Lounge",
      "game_type": "ROULETTE",
      "result_summary": "17 - BLACK, ODD, LOW",
      "total_wagered": 300,
      "total_won": 3600,
      "net_result": 3300,
      "played_at": "2026-02-11T14:32:00Z"
    },
    {
      "round_id": "uuid",
      "table_name": "Douala Lounge",
      "game_type": "ROULETTE",
      "result_summary": "0 - GREEN",
      "total_wagered": 200,
      "total_won": 0,
      "net_result": -200,
      "played_at": "2026-02-11T14:28:00Z"
    }
  ],
  "pagination": { "page": 1, "page_size": 20, "total_items": 245, "total_pages": 13 }
}

```

**GET `/v1/history/rounds/{round\_id}`**

Get detailed round result including individual bets.

**Auth required:** Yes

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {
    "round_id": "uuid",
    "table_name": "Douala Lounge",
    "game_type": "ROULETTE",
    "result": {
      "number": 17,
      "color": "BLACK",

```

```

        "parity": "ODD",
        "range": "LOW",
        "dozen": 2,
        "column": 2
    },
    "my_bets": [
        {
            "bet_id": "uuid",
            "bet_type": "STRAIGHT_UP",
            "numbers": [17],
            "amount": 100,
            "status": "WON",
            "win_amount": 3600,
            "net_result": 3500
        },
        {
            "bet_id": "uuid",
            "bet_type": "RED",
            "numbers": [],
            "amount": 200,
            "status": "LOST",
            "win_amount": 0,
            "net_result": -200
        }
    ],
    "summary": {
        "total_wagered": 300,
        "total_won": 3600,
        "net_result": 3300
    },
    "played_at": "2026-02-11T14:32:00Z"
}

```

### GET `/v1/history/statistics`

Get player's aggregated statistics.

**Auth required:** Yes

**Query params:**

| Param     | Type   | Default | Description             |
|-----------|--------|---------|-------------------------|
| game_type | string | all     | Filter by game type     |
| period    | string | all     | today, week, month, all |

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {

```

```

    "period": "all",
    "total_rounds": 245,
    "total_wagered": 125000,
    "total_won": 118500,
    "net_result": -6500,
    "win_rate": 41.2,
    "biggest_win": 17500,
    "biggest_loss": -2000,
    "favorite_game": "ROULETTE",
    "by_game": {
      "ROULETTE": {
        "total_rounds": 245,
        "total_wagered": 125000,
        "total_won": 118500,
        "net_result": -6500
      }
    },
    "session_count": 34,
    "avg_session_duration_min": 28
  }
}

```

## 5.9 Responsible Gaming Endpoints

### GET `/v1/responsible-gaming/limits`

Get player's spending limits.

**Auth required:** Yes

**Response (200 OK):**

```

json
{
  "success": true,
  "data": {
    "limits": [
      {
        "id": "uuid",
        "period": "DAILY",
        "limit_amount": 5000,
        "current_spent": 1200,
        "remaining": 3800,
        "period_start": "2026-02-11T00:00:00Z",
        "period_end": "2026-02-12T00:00:00Z",
        "is_active": true
      }
    ],
    "self_exclusion": null
  }
}

```

## POST `/v1/responsible-gaming/limits`

Set a spending limit.

**Auth required:** Yes

**Request:**

```
json
{
  "period": "DAILY",
  "limit_amount": 5000
}
```

| Field        | Type   | Required | Values                   |
|--------------|--------|----------|--------------------------|
| period       | string | Yes      | DAILY, WEEKLY, MONTHLY   |
| limit_amount | int    | Yes      | Positive integer (chips) |

**Response (201 Created):** Returns the created limit.

> **Rule:** Limits can be decreased immediately but increases take effect after the current period ends (prevents impulse override).

## POST `/v1/responsible-gaming/self-exclude`

Self-exclude from all games.

**Auth required:** Yes

**Request:**

```
json
{
  "duration_days": 30,
  "reason": "Taking a break"
}
```

**Response (200 OK):**

```
json
{
  "success": true,
  "data": {
    "excluded_until": "2026-03-13T14:30:00Z",
    "message": "You have been excluded until March 13, 2026. You cannot play or purchase chips during this period."
  }
}
```

> **Rule:** Self-exclusion cannot be reversed early. The player is immediately logged out of all active tables and prevented from logging into game sessions.

## 5.10 WebSocket Protocol

### 5.10.1 Connection

WSS `wss://api.{domain}/v1/ws/game/{table_id}?token={access_token}`

#### Handshake flow:

1. Client connects with JWT in query string
2. Server validates token
3. If valid → connection accepted, server sends `table:joined`
4. If invalid → connection closed with code `4001`

#### Close codes:

| Code | Reason                               |
|------|--------------------------------------|
| 4001 | Unauthorized (invalid/expired token) |
| 4003 | Forbidden (banned, self-excluded)    |
| 4004 | Table not found or inactive          |
| 4008 | Table full                           |
| 4009 | Already connected to this table      |
| 4029 | Rate limited                         |

### 5.10.2 Message Format

All messages follow the same envelope:

#### Client → Server:

```
json
{
  "event": "string",
  "data": {},
  "request_id": "optional-uuid"
}
```

#### Server → Client:

```
json
{
  "event": "string",
  "data": {},
  "timestamp": "2026-02-11T14:30:00Z",
  "request_id": "optional-uuid-echo"
}
```

> `request_id` is optional. If the client includes it, the server echoes it back in the response, allowing the client to match responses to requests.

### 5.10.3 Client → Server Events

#### `bet:place`

Place a bet at the current table.

```

json
{
  "event": "bet:place",
  "data": {
    "type": "STRAIGHT_UP",
    "amount": 100,
    "parameters": {
      "numbers": [17]
    }
  },
  "request_id": "req-001"
}

```

#### Parameters by bet type (Roulette):

| Bet Type          | Parameters                      | Example                               |
|-------------------|---------------------------------|---------------------------------------|
| STRAIGHT_UP       | {"numbers": [n]}                | {"numbers": [17]}                     |
| SPLIT             | {"numbers": [n1, n2]}           | {"numbers": [17, 20]}                 |
| STREET            | {"numbers": [n, n+1, n+2]}      | {"numbers": [16, 17, 18]}             |
| TRIO              | {"numbers": [0, 1, 2]}          | {"numbers": [0, 2, 3]}                |
| CORNER            | {"numbers": [n, n+1, n+3, n+4]} | {"numbers": [16, 17, 19, 20]}         |
| SIX_LINE          | {"numbers": [n..n+5]}           | {"numbers": [16, 17, 18, 19, 20, 21]} |
| FIRST_FOUR        | {}                              | {}                                    |
| TOP_LINE          | {}                              | {}                                    |
| RED               | {}                              | {}                                    |
| BLACK             | {}                              | {}                                    |
| EVEN              | {}                              | {}                                    |
| ODD               | {}                              | {}                                    |
| LOW               | {}                              | {}                                    |
| HIGH              | {}                              | {}                                    |
| DOZEN_1           | {}                              | {}                                    |
| DOZEN_2           | {}                              | {}                                    |
| DOZEN_3           | {}                              | {}                                    |
| COLUMN_1          | {}                              | {}                                    |
| COLUMN_2          | {}                              | {}                                    |
| COLUMN_3          | {}                              | {}                                    |
| CALL_VOISINS_ZERO | {"unit_amount": 50}             | {"unit_amount": 50}                   |
| CALL_TIERS        | {"unit_amount": 50}             | {"unit_amount": 50}                   |

| Bet Type       | Parameters                           | Example                              |
|----------------|--------------------------------------|--------------------------------------|
| CALL_ORPHELINS | {"unit_amount": 50}                  | {"unit_amount": 50}                  |
| CALL_NEIGHBORS | {"target_number": 17,<br>"count": 2} | {"target_number": 17,<br>"count": 2} |

> **Note for call bets:** `amount` in the outer object is ignored. The `unit_amount` in parameters defines the chip unit. Total cost = `unit_amount × chip_count` (9 for Voisins, 6 for Tiers, 5 for Orphelins, 2N+1 for Neighbors).

#### `bet:cancel`

Cancel a previously placed bet.

```
json
{
  "event": "bet:cancel",
  "data": {
    "bet_id": "uuid"
  },
  "request_id": "req-002"
}
```

> Only allowed during `BETTING` phase (not `BETTING_CLOSING`).

#### `game:action`

Game-specific player action.

```
json
{
  "event": "game:action",
  "data": {
    "action": "spin"
  }
}
```

#### Roulette actions:

| Action | When               | Description              |
|--------|--------------------|--------------------------|
| spin   | Solo mode, BETTING | Player triggers the spin |

#### Future — Blackjack actions:

| Action      | When          | Description               |
|-------------|---------------|---------------------------|
| hit         | Player's turn | Draw another card         |
| stand       | Player's turn | Keep current hand         |
| double_down | Initial deal  | Double bet, one more card |
| split       | Matching pair | Split into two hands      |

**`ping`**

Keepalive ping.

```
json
{"event": "ping", "data": {}}
```

> Client should send a ping every 30 seconds. Server responds with **pong**. If no ping received for 60 seconds, server may close the connection.

#### 5.10.4 Server → Client Events

**`table:joined`**

Sent immediately after successful WebSocket connection.

```
json
{
  "event": "table:joined",
  "data": {
    "table_id": "uuid",
    "table_name": "Douala Lounge",
    "game_type": "ROULETTE",
    "mode": "MULTIPLAYER",
    "game_config": {
      "variant": "EUROPEAN",
      "la_partage": false
    },
    "limits": {
      "min_bet": 50,
      "max_bet": 2000,
      "max_total_bet": 10000
    },
    "current_phase": "BETTING",
    "time_remaining": 28.3,
    "your_balance": 4350,
    "your_bets": [],
    "players": [
      {"display_name": "Lucky237", "avatar": "avatar_3"},
      {"display_name": "KingBet", "avatar": "avatar_1"}
    ],
    "last_results": [
      {"number": 17, "color": "BLACK"},
      {"number": 0, "color": "GREEN"}
    ]
  },
  "timestamp": "2026-02-11T14:30:00Z"
}
```

**`game:phase-changed`**

Emitted on every game phase transition.

```
json
{
```



```

    "event": "game:phase-changed",
    "data": {
      "phase": "BETTING",
      "time_remaining": 45.0,
      "round_id": "uuid",
      "message": "Place your bets!"
    },
    "timestamp": "2026-02-11T14:30:00Z"
  }

```

### Phase-specific data:

| Phase             | Extra fields                                    |
|-------------------|-------------------------------------------------|
| WAITING           | {}                                              |
| ACCEPTING_ACTIONS | {"time_remaining": 45.0} (BETTING for roulette) |
| PROCESSING        | {"time_remaining": 8.0} (SPINNING for roulette) |
| SHOWING_RESULTS   | {"time_remaining": 8.0}                         |
| COMPLETED         | {"next_round_in": 5.0}                          |

> **Note:** For roulette, the game engine also sends game-specific sub-events (see below).

### `game:roulette:betting-closing`

Roulette-specific: last seconds before betting closes.

```

json
{
  "event": "game:roulette:betting-closing",
  "data": {
    "time_remaining": 5.0,
    "message": "Rien ne va plus!"
  },
  "timestamp": "2026-02-11T14:30:40Z"
}

```

### `game:roulette:spinning`

Roulette-specific: wheel is spinning.

```

json
{
  "event": "game:roulette:spinning",
  "data": {
    "spin_duration": 8.0,
    "wheel_position_hint": 22,
    "provably_fair_hash": "a3f2b8c9d1e4..."
  },
  "timestamp": "2026-02-11T14:30:45Z"
}

```

> **wheel\_position\_hint** tells Unity which position on the wheel the ball will land (index 0–36 or 0–37), so the animation can target the correct pocket. The actual number is NOT revealed until **game:roulette:result**.

### **`game:roulette:result`**

Roulette-specific: the result of the spin. Broadcast to ALL players.

```
json
{
  "event": "game:roulette:result",
  "data": {
    "number": 17,
    "color": "BLACK",
    "parity": "ODD",
    "range": "LOW",
    "dozen": 2,
    "column": 2,
    "wheel_position": 8,
    "provably_fair_seed": "b7e9a2f1..."
  },
  "timestamp": "2026-02-11T14:30:53Z"
}
```

### **`game:personal-result`**

Sent to EACH player individually with their results.

```
json
{
  "event": "game:personal-result",
  "data": {
    "round_id": "uuid",
    "bets": [
      {
        "bet_id": "uuid",
        "bet_type": "STRAIGHT_UP",
        "numbers": [17],
        "amount": 100,
        "status": "WON",
        "payout_multiplier": 35,
        "win_amount": 3600
      },
      {
        "bet_id": "uuid",
        "bet_type": "RED",
        "numbers": [],
        "amount": 200,
        "status": "LOST",
        "payout_multiplier": 0,
        "win_amount": 0
      }
    ],
    "summary": {
```

```

        "total_wagered": 300,
        "total_won": 3600,
        "net_result": 3300
      },
      "new_balance": 4300
    },
    "timestamp": "2026-02-11T14:30:53Z"
  }
}

```

### **`bet:accepted`**

Confirmation that a bet was successfully placed.

```

json
{
  "event": "bet:accepted",
  "data": {
    "bet_id": "uuid",
    "bet_type": "STRAIGHT_UP",
    "numbers": [17],
    "amount": 100,
    "new_balance": 4250
  },
  "request_id": "req-001",
  "timestamp": "2026-02-11T14:30:12Z"
}

```

### **`bet:rejected`**

Bet placement failed.

```

json
{
  "event": "bet:rejected",
  "data": {
    "error_code": "E1003",
    "message": "Insufficient balance",
    "details": {
      "required": 100,
      "available": 50
    }
  },
  "request_id": "req-001",
  "timestamp": "2026-02-11T14:30:12Z"
}

```

### **`bet:cancelled`**

Bet successfully cancelled.

```

json
{
  "event": "bet:cancelled",
  "data": {

```

```

    "bet_id": "uuid",
    "refunded_amount": 100,
    "new_balance": 4350
  },
  "request_id": "req-002",
  "timestamp": "2026-02-11T14:30:15Z"
}

```

### **`bet:cancel-failed`**

Bet cancellation denied.

```

json
{
  "event": "bet:cancel-failed",
  "data": {
    "bet_id": "uuid",
    "error_code": "E1010",
    "message": "Cannot cancel bet in BETTING_CLOSING phase"
  },
  "request_id": "req-002",
  "timestamp": "2026-02-11T14:30:41Z"
}

```

### **`table:player-joined`**

Another player joined the table (multiplayer).

```

json
{
  "event": "table:player-joined",
  "data": {
    "display_name": "NewPlayer99",
    "avatar": "avatar_2",
    "player_count": 4
  },
  "timestamp": "2026-02-11T14:30:20Z"
}

```

### **`table:player-left`**

A player left the table.

```

json
{
  "event": "table:player-left",
  "data": {
    "display_name": "NewPlayer99",
    "player_count": 3
  },
  "timestamp": "2026-02-11T14:35:00Z"
}

```

### `player:balance-updated`

Player balance changed outside of game context (e.g., purchase completed).

```

json
{
  "event": "player:balance-updated",
  "data": {
    "balance": 10350,
    "reason": "Chip purchase completed"
  },
  "timestamp": "2026-02-11T14:33:15Z"
}

```

### `auth:token-expiring`

Sent 60 seconds before access token expiration.

```

json
{
  "event": "auth:token-expiring",
  "data": {
    "expires_in": 60
  },
  "timestamp": "2026-02-11T14:44:00Z"
}

```

> Client should call **POST /v1/auth/refresh** via REST to get a new token, then send **auth:refresh** via WebSocket.

### `pong`

Response to client ping.

```

json
{"event": "pong", "data": {}}

```

### `error`

Generic error (malformed message, unknown event, etc.).

```

json
{
  "event": "error",
  "data": {
    "code": "UNKNOWN_EVENT",
    "message": "Unknown event: bet:invalid_action"
  },
  "timestamp": "2026-02-11T14:30:12Z"
}

```

## 5.11 Admin API Endpoints

All admin endpoints are prefixed with `/v1/admin/` and require admin JWT authentication via `Authorization: Bearer {admin_token}`.

### 5.11.1 Admin Authentication

#### POST `/v1/admin/auth/login``

```
json
// Request
{"username": "admin", "password": "AdminSecure123"}

// Response
{
  "success": true,
  "data": {
    "access_token": "eyJ...",
    "role": "SUPER_ADMIN",
    "expires_in": 3600
  }
}
```

### 5.11.2 Table Management

#### GET `/v1/admin/tables``

List all tables with live status.

#### POST `/v1/admin/tables``

Create a new table.

```
json
{
  "name": "Yaoundé VIP",
  "game_type": "ROULETTE",
  "mode": "MULTIPLAYER",
  "tier": "VIP",
  "min_bet": 500,
  "max_bet": 10000,
  "max_total_bet": 50000,
  "max_players": 8,
  "game_config": {
    "variant": "EUROPEAN",
    "la_partage": true,
    "betting_duration_sec": 60,
    "closing_duration_sec": 10,
    "spinning_duration_sec": 10,
    "completed_duration_sec": 10
  }
}
```

> `game_config` is validated by the corresponding game engine's `validate_config()` method.

**PUT `/v1/admin/tables/{table\_id}`**

Update table configuration. Changes apply at next round start.

**DELETE `/v1/admin/tables/{table\_id}`**

Deactivate a table (soft delete). Fails if players are active — use `force_close` first.

**POST `/v1/admin/tables/{table\_id}/force-close`**

Force close a table. Current round ends, all bets refunded.

**5.11.3 Package Management****GET `/v1/admin/packages`****POST `/v1/admin/packages`****PUT `/v1/admin/packages/{package\_id}`****DELETE `/v1/admin/packages/{package\_id}`**

CRUD for chip purchase packages. Same fields as described in Section 2.3.3.

**5.11.4 Player Management****GET `/v1/admin/players`**

List/search players.

**Query params:** `search` (phone or display\_name), `is_banned`, `page`, `page_size`, `sort`

**GET `/v1/admin/players/{player\_id}`**

Full player details including balance, transaction history, game history.

**POST `/v1/admin/players/{player\_id}/ban`**

```
json
{"reason": "Suspicious activity", "duration_days": 30}
```

> `duration_days = null` → permanent ban.

**POST `/v1/admin/players/{player\_id}/unban`****POST `/v1/admin/players/{player\_id}/adjust-balance`**

```
json
{"amount": 5000, "reason": "Compensation for technical issue"}
```

> Creates a `ADMIN_ADJUSTMENT` transaction. Logged in audit trail.

**5.11.5 Dashboard****GET `/v1/admin/dashboard`**

```
json
{
```

```

    "success": true,
    "data": {
      "period": "today",
      "active_players": 34,
      "active_tables": 8,
      "total_rounds": 1247,
      "total_wagered": 4500000,
      "total_paid_out": 4125000,
      "gross_revenue": 375000,
      "total_deposits": 250000,
      "new_registrations": 12,
      "by_game": {
        "ROULETTE": {
          "rounds": 1247,
          "wagered": 4500000,
          "paid_out": 4125000
        }
      }
    }
  }
}

```

Query params: `period` = `today`, `yesterday`, `week`, `month`, `all`

### 5.11.6 Audit Logs

GET `/v1/admin/audit-logs``

```

json
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "admin_username": "admin",
      "action": "TABLE_CREATED",
      "entity_type": "TABLE",
      "entity_id": "uuid",
      "new_values": {"name": "Yaoundé VIP", "game_type": "ROULETTE"},
      "ip_address": "102.16.x.x",
      "created_at": "2026-02-11T10:00:00Z"
    }
  ],
  "pagination": { ... }
}

```

## 5.12 Complete Error Code Registry

### 5.12.1 Platform Errors (E0xxx)

| Code  | HTTP | Description          |
|-------|------|----------------------|
| E0001 | 400  | Insufficient balance |



| Code  | HTTP | Description                   |
|-------|------|-------------------------------|
| E0002 | 400  | Spending limit exceeded       |
| E0003 | 403  | Self-exclusion active         |
| E0004 | 400  | Invalid payment method        |
| E0005 | 409  | Payment already pending       |
| E0006 | 400  | Package not found or inactive |

### 5.12.2 Roulette Errors (E1xxx)

| Code  | HTTP | Description                        |
|-------|------|------------------------------------|
| E1001 | 400  | Betting is closed                  |
| E1002 | 400  | Player not at this table           |
| E1003 | 400  | Insufficient balance               |
| E1004 | 400  | Below minimum bet                  |
| E1005 | 400  | Above maximum bet                  |
| E1006 | 400  | Exceeds maximum total bet          |
| E1007 | 400  | Invalid bet type                   |
| E1008 | 400  | Invalid numbers for bet type       |
| E1009 | 400  | Bet type not available for variant |
| E1010 | 400  | Cannot cancel in BETTING_CLOSING   |
| E1011 | 400  | Cannot modify in BETTING_CLOSING   |
| E1012 | 400  | Table full                         |

### 5.12.3 Future Game Error Ranges

| Range       | Game      |
|-------------|-----------|
| E1000–E1999 | Roulette  |
| E2000–E2999 | Blackjack |
| E3000–E3999 | Slots     |
| E4000–E4999 | Poker     |

### 5.12.4 Authentication Errors

| Code                | HTTP | Description          |
|---------------------|------|----------------------|
| AUTH_ERROR          | 401  | Invalid credentials  |
| INVALID_CREDENTIALS | 401  | Wrong phone/password |
| INVALID_OTP         | 400  | Wrong OTP code       |

| Code                  | HTTP | Description               |
|-----------------------|------|---------------------------|
| OTP_EXPIRED           | 400  | OTP code expired          |
| OTP_MAX_ATTEMPTS      | 429  | Too many OTP attempts     |
| INVALID_REFRESH_TOKEN | 401  | Refresh token invalid     |
| REFRESH_TOKEN_EXPIRED | 401  | Refresh token expired     |
| PHONE_NOT_VERIFIED    | 403  | Phone not yet verified    |
| ACCOUNT_BANNED        | 403  | Account banned            |
| ACCOUNT_INACTIVE      | 403  | Account deactivated       |
| SELF_EXCLUDED         | 403  | Self-exclusion active     |
| PHONE_EXISTS          | 409  | Phone already registered  |
| NAME_EXISTS           | 409  | Display name taken        |
| UNDERAGE              | 400  | Under 18                  |
| FORBIDDEN             | 403  | Insufficient permissions  |
| NOT_FOUND             | 404  | Resource not found        |
| VALIDATION_ERROR      | 422  | Request validation failed |
| RATE_LIMITED          | 429  | Too many requests         |

## 5.13 API Endpoint Summary Table

### 5.13.1 Player-Facing (24 endpoints)

| Method | Endpoint                | Auth | Description             |
|--------|-------------------------|------|-------------------------|
| POST   | /v1/auth/register       | No   | Create account          |
| POST   | /v1/auth/verify-otp     | No   | Verify phone with OTP   |
| POST   | /v1/auth/login          | No   | Login                   |
| POST   | /v1/auth/refresh        | No   | Refresh access token    |
| POST   | /v1/auth/request-otp    | No   | Request new OTP         |
| POST   | /v1/auth/reset-password | No   | Reset password with OTP |
| GET    | /v1/players/me          | Yes  | Get profile             |
| PUT    | /v1/players/me          | Yes  | Update profile          |
| PUT    | /v1/players/me/password | Yes  | Change password         |
| GET    | /v1/wallet/balance      | Yes  | Get balance             |
| GET    | /v1/wallet/transactions | Yes  | Transaction history     |
| GET    | /v1/payments/packages   | Yes  | List chip packages      |
| POST   | /v1/payments/initiate   | Yes  | Start purchase          |

| Method | Endpoint                            | Auth | Description                     |
|--------|-------------------------------------|------|---------------------------------|
| GET    | /v1/payments/orders/{id}            | Yes  | Check order status              |
| POST   | /v1/payments/webhook/{provider}     | No*  | Payment callback (HMAC)         |
| GET    | /v1/payments/history                | Yes  | Purchase history                |
| GET    | /v1/tables                          | Yes  | List tables                     |
| GET    | /v1/tables/{id}                     | Yes  | Table details                   |
| GET    | /v1/history/rounds                  | Yes  | Game round history              |
| GET    | /v1/history/rounds/{id}             | Yes  | Round detail with bets          |
| GET    | /v1/history/statistics              | Yes  | Player statistics               |
| GET    | /v1/responsible-gaming/limits       | Yes  | Get spending limits             |
| POST   | /v1/responsible-gaming/limits       | Yes  | Set spending limit              |
| POST   | /v1/responsible-gaming/self-exclude | Yes  | Self-exclude                    |
| WS     | /v1/ws/game/{table_id}              | Yes* | Game WebSocket (token in query) |

### 5.13.2 Admin-Facing (16 endpoints)

| Method | Endpoint                          | Role      | Description        |
|--------|-----------------------------------|-----------|--------------------|
| POST   | /v1/admin/auth/login              | Any admin | Admin login        |
| GET    | /v1/admin/tables                  | Admin+    | List tables        |
| POST   | /v1/admin/tables                  | Admin+    | Create table       |
| PUT    | /v1/admin/tables/{id}             | Admin+    | Update table       |
| DELETE | /v1/admin/tables/{id}             | Admin+    | Deactivate table   |
| POST   | /v1/admin/tables/{id}/force-close | Admin+    | Force close        |
| GET    | /v1/admin/packages                | Admin+    | List packages      |
| POST   | /v1/admin/packages                | Admin+    | Create package     |
| PUT    | /v1/admin/packages/{id}           | Admin+    | Update package     |
| DELETE | /v1/admin/packages/{id}           | Admin+    | Deactivate package |
| GET    | /v1/admin/players                 | Admin+    | Search players     |
| GET    | /v1/admin/players/{id}            | Admin+    | Player details     |

| Method | Endpoint                              | Role        | Description    |
|--------|---------------------------------------|-------------|----------------|
| POST   | /v1/admin/players/{id}/ban            | Admin+      | Ban player     |
| POST   | /v1/admin/players/{id}/unban          | Admin+      | Unban player   |
| POST   | /v1/admin/players/{id}/adjust-balance | Super Admin | Adjust balance |
| GET    | /v1/admin/dashboard                   | Admin+      | KPI dashboard  |
| GET    | /v1/admin/audit-logs                  | Admin+      | Audit trail    |

*End of Section 5*

## Section 6 — Testing & Quality Strategy

### 6.1 Purpose of This Section

This section defines **what to test, how to test, and when to test**. In a game involving financial transactions (chip purchases, bets, payouts), correctness is non-negotiable. A single bug in bet resolution or balance management can erode player trust permanently.

#### Testing philosophy:

- The game engine (RNG, validation, resolution) must have **near-100% unit test coverage**
- The wallet (debit, credit, transactions) must have **100% coverage with concurrency tests**
- Every API endpoint must have at least one **happy path + one error path** integration test
- Every complete game round must be tested **end-to-end**

### 6.2 Testing Stack

| Tool                  | Purpose                               | Version |
|-----------------------|---------------------------------------|---------|
| pytest                | Test framework                        | 8.0+    |
| pytest-asyncio        | Async test support for FastAPI        | 0.23+   |
| httpx                 | Async HTTP test client (FastAPI)      | 0.27+   |
| factory_boy           | Test data factories                   | 3.3+    |
| faker                 | Fake data generation                  | 22+     |
| pytest-cov            | Coverage reporting                    | 5.0+    |
| testcontainers-python | Disposable PostgreSQL/Redis for tests | 4.0+    |
| pytest-xdist          | Parallel test execution               | 3.5+    |

#### 6.2.1 Test Database Strategy

- **Unit tests:** In-memory SQLite (fast) OR mock database layer
- **Integration tests:** Disposable PostgreSQL via testcontainers (real DB, real constraints)
- **Each test runs in a transaction that is rolled back** — no test data leaks between tests

```
python
# tests/conftest.py – Core fixtures

import pytest
import pytest_asyncio
from httpx import AsyncClient, ASGITransport
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from app.main import app
from app.database import Base, get_db_session

TEST_DATABASE_URL = "postgresql+asyncpg://test:test@localhost:5433/test_casino"
```

```

@pytest_asyncio.fixture
async def db_session():
    """Provide a transactional database session that rolls back after each test."""
    engine = create_async_engine(TEST_DATABASE_URL)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

    async with AsyncSession(engine) as session:
        async with session.begin():
            yield session
        await session.rollback()

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
    await engine.dispose()

@pytest_asyncio.fixture
async def client(db_session):
    """Provide an async test client with overridden DB dependency."""
    async def override_get_db():
        yield db_session

    app.dependency_overrides[get_db_session] = override_get_db
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as c:
        yield c
    app.dependency_overrides.clear()

@pytest_asyncio.fixture
async def authenticated_client(client, db_session):
    """Provide a client with a valid player JWT."""
    # Create test player + wallet
    player = await create_test_player(db_session)
    token = create_test_token(player.id)
    client.headers["Authorization"] = f"Bearer {token}"
    client._test_player = player
    return client

```

## 6.3 Unit Tests

### 6.3.1 Roulette — Wheel & RNG

**File:** `tests/games/roulette/test_wheel.py`

| Test ID | Test Case                             | Expected Result                       |
|---------|---------------------------------------|---------------------------------------|
| W-001   | European wheel has exactly 37 numbers | len == 37, no duplicates              |
| W-002   | American wheel has exactly 38 numbers | len == 38, no duplicates, includes -1 |
| W-003   | European wheel contains 0–36          | set equals {0, 1, ..., 36}            |
| W-004   | American wheel contains -1            | set equals {-1, 0, 1, ..., 36}        |

| Test ID | Test Case                                                   | Expected Result                       |
|---------|-------------------------------------------------------------|---------------------------------------|
|         | and 0–36                                                    |                                       |
| W-005   | spin_wheel("EUROPEAN") returns value in 0–36                | 0 <= result <= 36                     |
| W-006   | spin_wheel("AMERICAN") returns value in {-1, 0..36}         | result in valid set                   |
| W-007   | spin_wheel with invalid variant raises ValueError           | ValueError raised                     |
| W-008   | 10,000 spins produce roughly uniform distribution (chi-sq)  | p-value > 0.01 (not biased)           |
| W-009   | get_color returns correct color for all numbers             | Matches hardcoded reference           |
| W-010   | get_color(0) returns GREEN                                  | "GREEN"                               |
| W-011   | get_color(-1, "AMERICAN") returns GREEN                     | "GREEN"                               |
| W-012   | get_color with invalid number raises ValueError             | ValueError raised                     |
| W-013   | get_neighbors(17, 2, EUROPEAN) returns correct 5 numbers    | [25, 2, 17, 34, 6] (ordered by wheel) |
| W-014   | get_neighbors wraps around wheel edges correctly            | Wrap from end to start                |
| W-015   | get_neighbors(0, 2, EUROPEAN) includes last/first positions | Correct wrap                          |

```
python
```

```
# Example test implementation
class TestWheel:
    def test_european_wheel_completeness(self):
        assert len(EUROPEAN_WHEEL_ORDER) == 37
        assert set(EUROPEAN_WHEEL_ORDER) == set(range(37))

    def test_american_wheel_completeness(self):
        assert len(AMERICAN_WHEEL_ORDER) == 38
        assert set(AMERICAN_WHEEL_ORDER) == set(range(37)) | {DOUBLE_ZERO}

    def test_spin_distribution(self):
        """Chi-squared test for uniform distribution over 10,000 spins."""
        from collections import Counter
        from scipy.stats import chisquare

        results = Counter(spin_wheel("EUROPEAN") for _ in range(10000))
        observed = [results.get(n, 0) for n in range(37)]
        expected_freq = 10000 / 37
        chi2, p_value = chisquare(observed, [expected_freq] * 37)
        assert p_value > 0.01, f"Distribution appears biased (p={p_value})"
```

```
@pytest.mark.parametrize("number, expected_color", [
    (0, "GREEN"), (1, "RED"), (2, "BLACK"), (3, "RED"),
    (10, "BLACK"), (11, "BLACK"), (12, "RED"), (36, "RED"),
])
def test_color_mapping(self, number, expected_color):
    assert get_color(number) == expected_color
```

### 6.3.2 Roulette — Bet Validation

**File:** `tests/games/roulette/test_validators.py`

#### Straight Up Validation

| Test ID | Input                             | Expected            |
|---------|-----------------------------------|---------------------|
| SU-001  | numbers=[17],<br>variant=EUROPEAN | Valid               |
| SU-002  | numbers=[0],<br>variant=EUROPEAN  | Valid               |
| SU-003  | numbers=[-1],<br>variant=AMERICAN | Valid (00)          |
| SU-004  | numbers=[-1],<br>variant=EUROPEAN | Invalid             |
| SU-005  | numbers=[37], any variant         | Invalid             |
| SU-006  | numbers=[17, 18], any             | Invalid (2 numbers) |
| SU-007  | numbers=[], any                   | Invalid (empty)     |

#### Split Validation

| Test ID | Input                        | Expected | Reason                              |
|---------|------------------------------|----------|-------------------------------------|
| SP-001  | numbers=[1, 2]               | Valid    | Horizontal neighbors                |
| SP-002  | numbers=[1, 4]               | Valid    | Vertical neighbors                  |
| SP-003  | numbers=[1, 3]               | Invalid  | Same row but not adjacent           |
| SP-004  | numbers=[3, 4]               | Invalid  | Adjacent numbers but different rows |
| SP-005  | numbers=[0, 1]               | Valid    | Zero split                          |
| SP-006  | numbers=[0, 2]               | Valid    | Zero split                          |
| SP-007  | numbers=[0, 3]               | Valid    | Zero split                          |
| SP-008  | numbers=[0, 4]               | Invalid  | 0 and 4 not adjacent                |
| SP-009  | numbers=[-1, 0],<br>AMERICAN | Valid    | 00-0 split                          |
| SP-010  | numbers=[-1, 2],<br>AMERICAN | Valid    | 00-2 split                          |
| SP-011  | numbers=[-1, 3],             | Valid    | 00-3 split                          |



| Test ID | Input                        | Expected | Reason                   |
|---------|------------------------------|----------|--------------------------|
|         | AMERICAN                     |          |                          |
| SP-012  | numbers=[-1, 1],<br>AMERICAN | Invalid  | 00-1 not adjacent        |
| SP-013  | numbers=[-1, 0],<br>EUROPEAN | Invalid  | 00 not in European       |
| SP-014  | numbers=[11, 12]             | Valid    | Horizontal: same row     |
| SP-015  | numbers=[12, 13]             | Invalid  | 12 and 13 different rows |
| SP-016  | numbers=[34, 35]             | Valid    | Horizontal bottom row    |
| SP-017  | numbers=[33, 36]             | Valid    | Vertical last column     |

### Street Validation

| Test ID | Input                | Expected                  |
|---------|----------------------|---------------------------|
| ST-001  | numbers=[1, 2, 3]    | Valid                     |
| ST-002  | numbers=[34, 35, 36] | Valid                     |
| ST-003  | numbers=[2, 3, 4]    | Invalid (not a row start) |
| ST-004  | numbers=[1, 3, 5]    | Invalid (not consecutive) |

### Corner Validation

| Test ID | Input                    | Expected                                |
|---------|--------------------------|-----------------------------------------|
| CO-001  | numbers=[1, 2, 4, 5]     | Valid                                   |
| CO-002  | numbers=[32, 33, 35, 36] | Valid                                   |
| CO-003  | numbers=[2, 3, 5, 6]     | Valid                                   |
| CO-004  | numbers=[3, 4, 6, 7]     | Invalid (3 is col 3, can't be top-left) |
| CO-005  | numbers=[33, 34, 36, 37] | Invalid (37 doesn't exist)              |
| CO-006  | numbers=[1, 2, 3, 4]     | Invalid (not a square)                  |

### Six Line Validation

| Test ID | Input                       | Expected                                |
|---------|-----------------------------|-----------------------------------------|
| SL-001  | numbers=[1,2,3,4,5,6]       | Valid                                   |
| SL-002  | numbers=[31,32,33,34,35,36] | Valid                                   |
| SL-003  | numbers=[2,3,4,5,6,7]       | Invalid (doesn't start at row boundary) |
| SL-004  | numbers=[34,35,36,37,38,39] | Invalid (numbers don't exist)           |

## Complete Parametrized Test

```
python
class TestBetValidation:
    @pytest.mark.parametrize("bet_type,numbers,variant,expected", [
        # Straight Up
        (BetType.STRAIGHT_UP, [17], "EUROPEAN", True),
        (BetType.STRAIGHT_UP, [-1], "AMERICAN", True),
        (BetType.STRAIGHT_UP, [-1], "EUROPEAN", False),

        # Split – valid
        (BetType.SPLIT, [1, 2], "EUROPEAN", True),
        (BetType.SPLIT, [1, 4], "EUROPEAN", True),
        (BetType.SPLIT, [0, 3], "EUROPEAN", True),

        # Split – invalid
        (BetType.SPLIT, [1, 3], "EUROPEAN", False),
        (BetType.SPLIT, [3, 4], "EUROPEAN", False),
        (BetType.SPLIT, [12, 13], "EUROPEAN", False),

        # Street
        (BetType.STREET, [1, 2, 3], "EUROPEAN", True),
        (BetType.STREET, [2, 3, 4], "EUROPEAN", False),

        # Corner
        (BetType.CORNER, [1, 2, 4, 5], "EUROPEAN", True),
        (BetType.CORNER, [3, 4, 6, 7], "EUROPEAN", False),

        # Variant-restricted
        (BetType.FIRST_FOUR, [0, 1, 2, 3], "EUROPEAN", True),
        (BetType.FIRST_FOUR, [0, 1, 2, 3], "AMERICAN", False),
        (BetType.TOP_LINE, [-1, 0, 1, 2, 3], "AMERICAN", True),
        (BetType.TOP_LINE, [-1, 0, 1, 2, 3], "EUROPEAN", False),
    ])
    def test_bet_structure_validation(self, bet_type, numbers, variant, expected):
        is_valid, _ = validate_bet_structure(bet_type, numbers, variant)
        assert is_valid == expected
```

### 6.3.3 Roulette — Win/Loss Resolution

**File:** tests/games/roulette/test\_resolver.py

| Test ID | Bet                            | Result | Expected                         |
|---------|--------------------------------|--------|----------------------------------|
| R-001   | STRAIGHT_UP [17],<br>100 chips | 17     | Win: 3600 returned,<br>+3500 net |
| R-002   | STRAIGHT_UP [17],<br>100 chips | 23     | Lose: 0 returned, -100<br>net    |
| R-003   | SPLIT [17,20], 100<br>chips    | 17     | Win: 1800 returned,<br>+1700 net |
| R-004   | SPLIT [17,20], 100<br>chips    | 20     | Win: 1800 returned,<br>+1700 net |

| Test ID | Bet                       | Result | Expected                         |
|---------|---------------------------|--------|----------------------------------|
| R-005   | SPLIT [17,20], 100 chips  | 5      | Lose: 0 returned, -100 net       |
| R-006   | RED, 100 chips            | 1      | Win: 200 returned, +100 net      |
| R-007   | RED, 100 chips            | 2      | Lose: 0 returned, -100 net       |
| R-008   | RED, 100 chips            | 0      | Lose: 0 returned, -100 net       |
| R-009   | RED, 100, La Partage ON   | 0      | Half: 50 returned, -50 net       |
| R-010   | EVEN, 100 chips           | 2      | Win: 200 returned                |
| R-011   | EVEN, 100 chips           | 0      | Lose (0 is not even for betting) |
| R-012   | LOW, 100 chips            | 18     | Win: 200 returned                |
| R-013   | LOW, 100 chips            | 19     | Lose: 0 returned                 |
| R-014   | HIGH, 100 chips           | 19     | Win: 200 returned                |
| R-015   | DOZEN_1, 100 chips        | 12     | Win: 300 returned                |
| R-016   | DOZEN_1, 100 chips        | 13     | Lose: 0 returned                 |
| R-017   | COLUMN_1, 100 chips       | 1      | Win: 300 returned                |
| R-018   | COLUMN_1, 100 chips       | 2      | Lose: 0 returned                 |
| R-019   | CORNER [1,2,4,5], 100     | 4      | Win: 900 returned                |
| R-020   | SIX_LINE [1..6], 100      | 6      | Win: 600 returned                |
| R-021   | TOP_LINE, 100, AMERICAN   | -1     | Win: 700 returned (6:1)          |
| R-022   | TOP_LINE, 100, AMERICAN   | 0      | Win: 700 returned                |
| R-023   | FIRST_FOUR, 100, EUROPEAN | 0      | Win: 900 returned (8:1)          |

```
python
class TestResolver:

@pytest.mark.parametrize("bet_type,numbers,amount,result_number,la_partage,expected_win"
, [
    (BetType.STRAIGHT_UP, [17], 100, 17, False, 3600),
    (BetType.STRAIGHT_UP, [17], 100, 23, False, 0),
    (BetType.RED, [], 100, 1, False, 200),
    (BetType.RED, [], 100, 0, False, 0),
    (BetType.RED, [], 100, 0, True, 50), # La Partage
    (BetType.EVEN, [], 100, 0, False, 0), # 0 is not even
    (BetType.TOP_LINE, [-1, 0, 1, 2, 3], 100, -1, False, 700),
])
```

```
def test_resolve_bet(self, bet_type, numbers, amount, result_number, la_partage,
expected_win):
    bet = make_bet(bet_type=bet_type, numbers=numbers, amount=amount)
    result = resolve_bet(bet, result_number, la_partage=la_partage)
    assert result.win_amount == expected_win
```

### 6.3.4 Roulette — Call Bet Expansion

File: `tests/games/roulette/test_call_bets.py`

| Test ID | Call Bet                                              | Expected                                 |
|---------|-------------------------------------------------------|------------------------------------------|
| CB-001  | VOISINS_ZERO, 50                                      | 7 bets, total cost = 450 (9 units × 50)  |
| CB-002  | TIERS, 50                                             | 6 bets, total cost = 300 (6 units × 50)  |
| CB-003  | ORPHELINS, 50                                         | 5 bets, total cost = 250 (5 units × 50)  |
| CB-004  | NEIGHBORS(17, 2)                                      | 5 STRAIGHT_UP bets on [2, 25, 17, 34, 6] |
| CB-005  | VOISINS_ZERO, AMERICAN                                | Raises ValueError (European only)        |
| CB-006  | NEIGHBORS covers correct numbers on American wheel    | Correct                                  |
| CB-007  | All 37 numbers covered by VOISINS + TIERS + ORPHELINS | Union = {0..36}                          |

**python**

```
class TestCallBets:
    def test_voisins_tiers_orphelins_cover_all_numbers(self):
        """The three call bets together cover all 37 European numbers."""
        voisins = expand_call_bet(BetType.CALL_VOISINS_ZERO, 1, "EUROPEAN")
        tiers = expand_call_bet(BetType.CALL_TIERS, 1, "EUROPEAN")
        orphelins = expand_call_bet(BetType.CALL_ORPHELINS, 1, "EUROPEAN")

        all_numbers = set()
        for bets in [voisins, tiers, orphelins]:
            for bet in bets:
                all_numbers.update(bet["numbers"])

        assert all_numbers == set(range(37))
```

### 6.3.5 Roulette — State Machine

File: `tests/games/roulette/test_state_machine.py`

| Test ID | Test Case                    | Expected |
|---------|------------------------------|----------|
| SM-001  | WAITING → BETTING transition | Success  |

| Test ID | Test Case                                                | Expected         |
|---------|----------------------------------------------------------|------------------|
| SM-002  | BETTING → SPINNING transition (skip closing not allowed) | Fail             |
| SM-003  | BETTING → BETTING_CLOSING transition                     | Success          |
| SM-004  | SPINNING → BETTING transition (backward)                 | Fail             |
| SM-005  | COMPLETED → WAITING transition                           | Success          |
| SM-006  | Place bet in BETTING phase                               | Accepted         |
| SM-007  | Place bet in SPINNING phase                              | Rejected (E1001) |
| SM-008  | Cancel bet in BETTING phase                              | Accepted         |
| SM-009  | Cancel bet in BETTING_CLOSING phase                      | Rejected (E1010) |
| SM-010  | Place bet in BETTING_CLOSING phase                       | Accepted         |

### 6.3.6 Wallet — Balance Operations

File: `tests/platform/test_wallet.py`

| Test ID | Test Case                                                | Expected                          |
|---------|----------------------------------------------------------|-----------------------------------|
| WL-001  | Credit 500 chips to new wallet (balance 0)               | Balance = 500                     |
| WL-002  | Debit 100 from wallet with 500                           | Balance = 400                     |
| WL-003  | Debit 600 from wallet with 500                           | InsufficientBalanceError          |
| WL-004  | Debit exactly all chips (500 from 500)                   | Balance = 0                       |
| WL-005  | Debit 0 chips                                            | Rejected (amount must be > 0)     |
| WL-006  | Credit negative amount                                   | Rejected                          |
| WL-007  | Transaction record created on credit                     | Record exists with correct fields |
| WL-008  | Transaction balance_before/after are accurate            | Match actual balances             |
| WL-009  | Concurrent debit (race condition): two 300 debits on 500 | Only one succeeds                 |
| WL-010  | Concurrent credit + debit: credit 200, debit 100 on 500  | Final balance = 600               |

```
python
class TestWalletConcurrency:
```

```

@pytest.mark.asyncio
async def test_concurrent_debits_no_overdraw(self, db_session):
    """Two concurrent 300-chip debits on a 500-chip wallet. Only one should
succeed."""
    wallet = await create_wallet(db_session, balance=500)

    async def attempt_debit():
        try:
            return await wallet_service.debit(wallet.player_id, 300, "BET", "test")
        except InsufficientBalanceError:
            return None

    results = await asyncio.gather(attempt_debit(), attempt_debit())
    successes = [r for r in results if r is not None]
    assert len(successes) == 1

    # Verify final balance
    final = await wallet_service.get_balance(wallet.player_id)
    assert final == 200

```

## 6.4 Integration Tests

### 6.4.1 Authentication Flow

**File:** `tests/integration/test_auth_flow.py`

| Test ID | Test Case                               | Endpoints Hit                        |
|---------|-----------------------------------------|--------------------------------------|
| IA-001  | Full registration → OTP → login flow    | register → verify-otp → login        |
| IA-002  | Register with duplicate phone           | register → 409                       |
| IA-003  | Login with wrong password               | login → 401                          |
| IA-004  | Access protected endpoint without token | GET /players/me → 401                |
| IA-005  | Access with expired token               | GET /players/me → 401                |
| IA-006  | Refresh token flow                      | refresh → new access token works     |
| IA-007  | Password reset flow                     | request-otp → reset-password → login |
| IA-008  | Banned player cannot login              | login → 403<br>ACCOUNT_BANNED        |
| IA-009  | Self-excluded player cannot login       | login → 403<br>SELF_EXCLUDED         |

### 6.4.2 Payment Flow

**File:** `tests/integration/test_payment_flow.py`

| Test ID | Test Case                                 | Flow                                           |
|---------|-------------------------------------------|------------------------------------------------|
| IP-001  | Successful chip purchase                  | initiate → webhook(success) → balance updated  |
| IP-002  | Failed payment (webhook failure)          | initiate → webhook(failed) → balance unchanged |
| IP-003  | Payment timeout (no webhook within 5 min) | initiate → wait → status = EXPIRED             |
| IP-004  | Duplicate webhook (idempotency)           | webhook × 2 → balance credited once            |
| IP-005  | Purchase while another is pending         | initiate → initiate → 409 PAYMENT_PENDING      |
| IP-006  | Invalid HMAC signature on webhook         | webhook → rejected                             |
| IP-007  | Welcome bonus credited on first purchase  | first purchase → bonus applied                 |

### 6.4.3 Full Game Round

File: `tests/integration/test_full_round.py`

| Test ID | Test Case                                                         | Description                            |
|---------|-------------------------------------------------------------------|----------------------------------------|
| IG-001  | Solo: place bet → spin → win → balance correct                    | Complete happy path                    |
| IG-002  | Solo: place bet → spin → lose → balance correct                   | Balance only decreases by bet amount   |
| IG-003  | Solo: multiple bets → spin → mixed results                        | Some bets win, others lose             |
| IG-004  | Solo: place bet → cancel → spin → bet not resolved                | Cancelled bet refunded, not in results |
| IG-005  | Solo: insufficient balance → bet rejected → spin works            | Error doesn't break game flow          |
| IG-006  | Multiplayer: 3 players bet → same spin → each gets correct result | Independent resolution                 |
| IG-007  | Multiplayer: player joins mid-round → can bet next round          | No crash, smooth join                  |
| IG-008  | Player disconnects during SPINNING → balance still updated        | Server-side resolution continues       |
| IG-009  | Call bet (Voisins) → expanded → resolved correctly                | 9 sub-bets created and resolved        |
| IG-010  | La Partage: even-money bet → 0 hits → half returned               | Correct La Partage behavior            |

python

```

class TestFullGameRound:
    @pytest.mark.asyncio
    async def test_solo_complete_round(self, authenticated_client, db_session):
        """Complete solo round: join → bet → spin → verify result."""
        player = authenticated_client._test_player

        # 1. Get table
        response = await authenticated_client.get("/v1/tables")
        table_id = response.json()["data"][0]["id"]

        # 2. Connect WebSocket
        async with authenticated_client.websocket_connect(
            f"/v1/ws/game/{table_id}?token={get_token(player)}"
        ) as ws:
            joined = await ws.receive_json()
            assert joined["event"] == "table:joined"

            # 3. Place bet
            await ws.send_json({
                "event": "bet:place",
                "data": {"type": "RED", "amount": 100, "parameters": {}},
                "request_id": "test-1"
            })
            bet_response = await ws.receive_json()
            assert bet_response["event"] == "bet:accepted"
            old_balance = bet_response["data"]["newBalance"]

            # 4. Trigger spin (solo action)
            await ws.send_json({
                "event": "game:action",
                "data": {"action": "spin"}
            })

            # 5. Collect events until personal result
            events = []
            while True:
                msg = await ws.receive_json()
                events.append(msg)
                if msg["event"] == "game:personal-result":
                    break

            # 6. Verify
            result = next(e for e in events if e["event"] == "game:personal-result")
            summary = result["data"]["summary"]
            new_balance = result["data"]["new_balance"]

            assert summary["total_wagered"] == 100
            if summary["net_result"] > 0:
                assert new_balance == old_balance + summary["total_won"]
            else:
                assert new_balance == old_balance # Already deducted at bet time

```



## 6.5 Test Coverage Targets

| Module                        | Target Coverage | Critical Level                                                                               |
|-------------------------------|-----------------|----------------------------------------------------------------------------------------------|
| games/roulette/wheel.py       | 100%            |  Critical |
| games/roulette/validators.py  | 100%            |  Critical |
| games/roulette/resolver.py    | 100%            |  Critical |
| games/roulette/call_bets.py   | 100%            |  High     |
| games/roulette/state.py       | 95%+            |  Critical |
| games/roulette/bet_manager.py | 95%+            |  Critical |
| platform/wallet/service.py    | 100%            |  Critical |
| platform/auth/service.py      | 90%+            |  High     |
| platform/auth/security.py     | 95%+            |  High     |
| platform/payments/service.py  | 90%+            |  High     |
| platform/tables/service.py    | 85%+            |  Medium   |
| platform/admin/service.py     | 80%+            |  Medium   |
| platform/websocket/           | 80%+            |  High   |
| <b>Overall project</b>        | <b>85%+</b>     |                                                                                              |

## 6.6 Test Data Factories

```
python
# tests/factories.py
import factory
from app.platform.players.models import Player
from app.platform.wallet.models import Wallet

class PlayerFactory(factory.Factory):
    class Meta:
        model = Player

    id = factory.LazyFunction(uuid4)
    phone = factory.Sequence(lambda n: f"+23761{n:07d}")
    phone_hash = factory.LazyAttribute(lambda o: sha256(o.phone.encode()).hexdigest())
    display_name = factory.Sequence(lambda n: f"Player{n}")
    password_hash = factory.LazyFunction(lambda: hash_password("TestPass123"))
    date_of_birth = factory.LazyFunction(lambda: date(2000, 1, 1))
    is_active = True
    is_banned = False

class WalletFactory(factory.Factory):
    class Meta:
```

```

    model = Wallet

    id = factory.LazyFunction(uuid4)
    player_id = factory.LazyAttribute(lambda o: o.player.id if hasattr(o, 'player') else
    uuid4())
    balance = 10000 # Default 10K chips for testing
    total_deposited = 10000
    bonus_balance = 0

class BetFactory(factory.Factory):
    class Meta:
        model = dict # Returns dict for flexibility

    bet_type = "STRAIGHT_UP"
    numbers = [17]
    amount = 100

```

## 6.7 Performance & Load Tests

### 6.7.1 Key Performance Benchmarks

| Scenario                          | Target               | Measurement                   |
|-----------------------------------|----------------------|-------------------------------|
| Bet placement (API response time) | < 50ms (p95)         | From request to response      |
| Spin result generation            | < 1ms                | RNG + attribute computation   |
| Round resolution (50 bets)        | < 100ms              | All bets + balance updates    |
| Round resolution (200 bets)       | < 500ms              | Multiplayer worst case        |
| WebSocket event delivery          | < 20ms               | Server emit to client receive |
| Concurrent players per table      | 8 players, 30 bets/s | No errors, no timeouts        |
| Concurrent tables                 | 50 active tables     | No degradation                |
| Payment webhook processing        | < 200ms              | Webhook to balance update     |

### 6.7.2 Load Test Scenarios

**Tool:** locust (Python-based load testing)

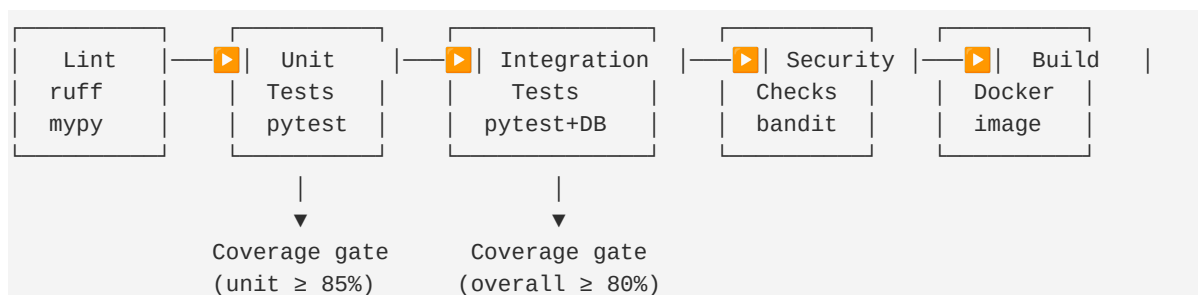
| Scenario                 | Virtual Users | Duration | Success Criteria        |
|--------------------------|---------------|----------|-------------------------|
| Sustained betting        | 50 users      | 10 min   | 0 errors, p95 < 100ms   |
| Peak betting (end timer) | 100 users     | 2 min    | 0 errors, p95 < 200ms   |
| Mass connection          | 200 WS conns  | 5 min    | All connected, no drops |
| Payment spike            | 20 users      | 5 min    | All processed correctly |

## 6.8 Security Tests

| Test ID | Test Case                                           | Expected                 |
|---------|-----------------------------------------------------|--------------------------|
| SEC-001 | SQL injection in phone number field                 | Rejected by Pydantic     |
| SEC-002 | JWT token tampering (modified payload)              | 401 Unauthorized         |
| SEC-003 | Expired JWT token                                   | 401 Unauthorized         |
| SEC-004 | Access other player's data via ID manipulation      | 403 Forbidden            |
| SEC-005 | Place bet with negative amount                      | Validation error         |
| SEC-006 | Place bet with amount exceeding MAX_INT             | Validation error         |
| SEC-007 | WebSocket connection without token                  | 4001 close               |
| SEC-008 | WebSocket message with invalid JSON                 | Error event, no crash    |
| SEC-009 | Payment webhook with invalid HMAC                   | Rejected                 |
| SEC-010 | Admin endpoint with player token                    | 403 Forbidden            |
| SEC-011 | Rate limit exceeded (login brute force)             | 429 after 5 attempts     |
| SEC-012 | XSS in display_name field                           | Sanitized/rejected       |
| SEC-013 | Player tries to bet more than balance via race cond | Only valid bets accepted |

## 6.9 CI Pipeline

### 6.9.1 Pipeline Stages



### 6.9.2 Pre-Commit Checks (Local)

| Check   | Tool | Purpose                    |
|---------|------|----------------------------|
| Linting | ruff | Code style, import sorting |

| Check    | Tool   | Purpose                          |
|----------|--------|----------------------------------|
| Types    | mypy   | Static type checking             |
| Format   | black  | Code formatting                  |
| Security | bandit | Common security issues in Python |

## 6.9.3 GitHub Actions Workflow

```

yaml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: test_casino
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
        ports: ["5433:5432"]
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7-alpine
        ports: ["6379:6379"]

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements-dev.txt

      - name: Lint
        run: |
          ruff check app/ tests/
          mypy app/

      - name: Unit tests
        run: pytest tests/ -m "not integration" --cov=app --cov-report=xml

      - name: Integration tests
        run: pytest tests/ -m "integration" --cov=app --cov-report=xml --cov-append
        env:

```

```
DATABASE_URL: postgresql+asyncpg://test:test@localhost:5433/test_casino
REDIS_URL: redis://localhost:6379/0

- name: Coverage check
  run: |
    coverage report --fail-under=80
```

## 6.10 Validation Checklist (Per Feature)

Before any feature is considered "done", it must pass this checklist:

- ☐ Unit tests written and passing
- ☐ Integration test for happy path
- ☐ Integration test for main error paths
- ☐ Edge cases documented and tested
- ☐ No regressions (full suite passes)
- ☐ Coverage meets module target
- ☐ API response matches documented schema (Section 5)
- ☐ WebSocket events match documented protocol (Section 5)
- ☐ Error codes match registry (Section 5.12)
- ☐ Manually verified in dev environment
- ☐ Code reviewed by at least one other developer

*End of Section 6*

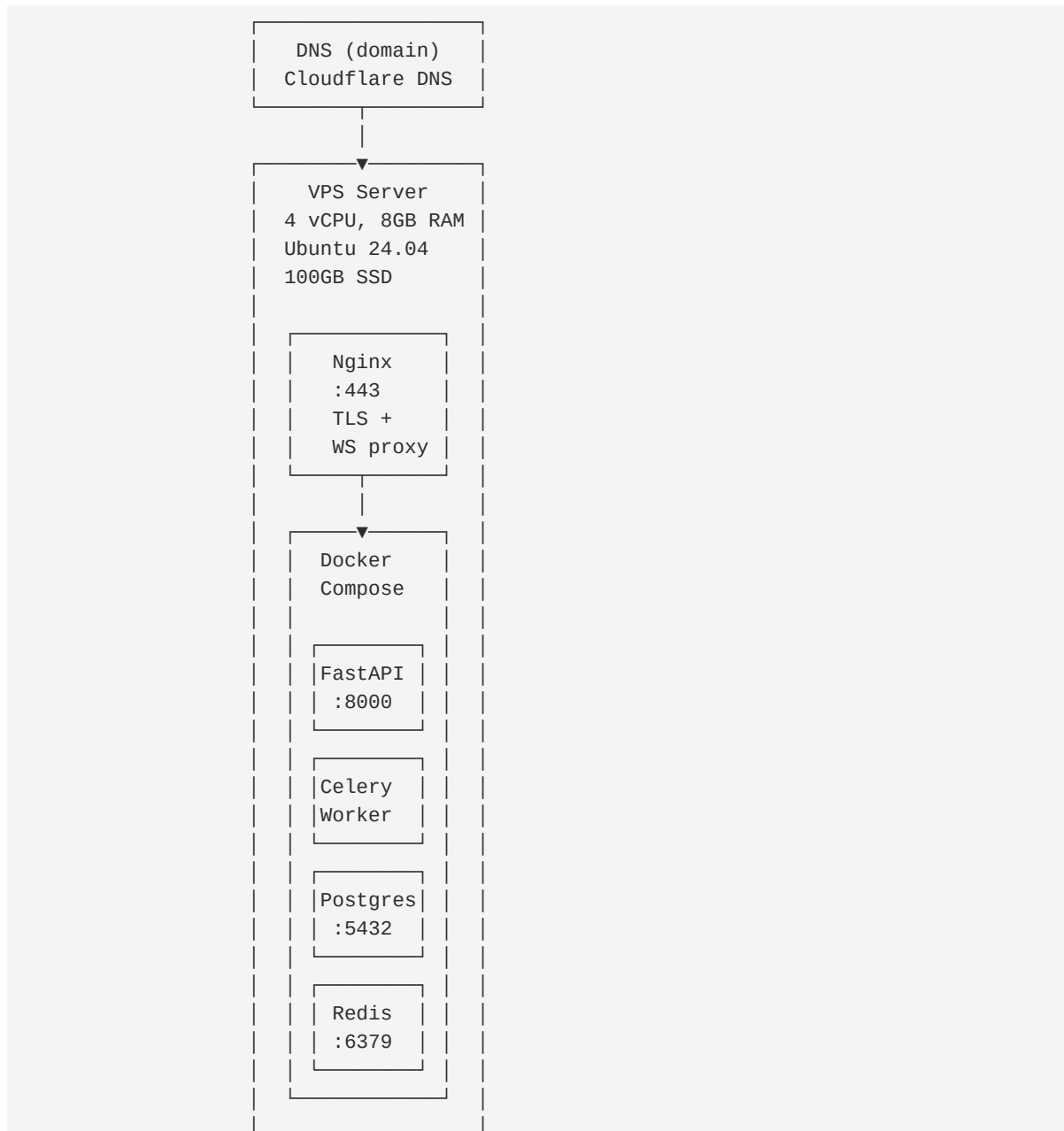
## Section 7 — Deployment & DevOps

### 7.1 Purpose of This Section

This section covers **how to deploy, monitor, secure, and operate** the casino platform in production. Designed for a 1-2 developer team with basic Docker knowledge, it prioritizes simplicity while maintaining production-grade reliability.

### 7.2 Infrastructure Overview

#### 7.2.1 V1 Target Architecture



## 7.2.2 VPS Recommendations

| Provider     | Plan            | Specs              | Estimated Cost |
|--------------|-----------------|--------------------|----------------|
| Contabo      | Cloud VPS M     | 4 vCPU, 8GB, 200GB | ~8 EUR/month   |
| Hetzner      | CX31            | 4 vCPU, 8GB, 80GB  | ~12 EUR/month  |
| DigitalOcean | Droplet Regular | 4 vCPU, 8GB, 160GB | ~48 USD/month  |
| OVH          | VPS Comfort     | 4 vCPU, 8GB, 160GB | ~20 EUR/month  |

**Recommendation:** Contabo or Hetzner for V1 (best price/performance for Africa-targeting servers). Choose a European data center (Frankfurt or Paris) for reasonable latency to Cameroon (~80-120ms).

## 7.3 Nginx Configuration

### 7.3.1 Main Config

```

nginx
# /etc/nginx/sites-available/casino-api

upstream fastapi {
    server 127.0.0.1:8000;
}

server {
    listen 80;
    server_name api.yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.yourdomain.com;

    # TLS – Let's Encrypt
    ssl_certificate /etc/letsencrypt/live/api.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.yourdomain.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    # Security headers
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

    # REST API
    location /v1/ {

```

```

    proxy_pass http://fastapi;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Timeouts
    proxy_connect_timeout 10s;
    proxy_send_timeout 30s;
    proxy_read_timeout 30s;
}

# WebSocket
location /v1/ws/ {
    proxy_pass http://fastapi;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

    # WebSocket timeouts
    proxy_connect_timeout 10s;
    proxy_send_timeout 3600s;    # 1 hour max connection
    proxy_read_timeout 3600s;
}

# Payment webhooks – longer timeout
location /v1/payments/webhook/ {
    proxy_pass http://fastapi;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 60s;
}

# Rate limiting zones (defined in nginx.conf http block)
# limit_req_zone $binary_remote_addr zone=auth:10m rate=5r/m;
# limit_req_zone $binary_remote_addr zone=general:10m rate=100r/m;
}

```

### 7.3.2 TLS Setup

```

bash
# Install certbot
sudo apt install certbot python3-certbot-nginx

# Obtain certificate
sudo certbot --nginx -d api.yourdomain.com

# Auto-renewal (cron)
sudo certbot renew --dry-run

```



```
# Certbot installs a systemd timer automatically
```

## 7.4 Production Docker Compose

```
yaml
# docker-compose.prod.yml
version: "3.9"

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    restart: always
    env_file: .env.production
    ports:
      - "127.0.0.1:8000:8000"    # Only accessible via Nginx
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    command: >
      uvicorn app.main:app
      --host 0.0.0.0
      --port 8000
      --workers 1
      --log-level info
      --access-log
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 15s
    deploy:
      resources:
        limits:
          cpus: "2.0"
          memory: 2G

  celery:
    build: .
    restart: always
    env_file: .env.production
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    command: >
      celery -A app.platform.payments.tasks worker
      --loglevel=info
```

```

    --concurrency=2
  deploy:
    resources:
      limits:
        cpus: "0.5"
        memory: 512M

celery-beat:
  build: .
  restart: always
  env_file: .env.production
  depends_on: [redis]
  command: >
    celery -A app.platform.payments.tasks beat
    --loglevel=info
  deploy:
    resources:
      limits:
        cpus: "0.25"
        memory: 256M

postgres:
  image: postgres:16-alpine
  restart: always
  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  ports:
    - "127.0.0.1:5432:5432" # Local only
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./backups:/backups # Backup mount
  healthcheck:
    test: pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}
    interval: 10s
    timeout: 5s
    retries: 5
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 2G
  command: >
    postgres
    -c shared_buffers=512MB
    -c effective_cache_size=1536MB
    -c work_mem=16MB
    -c maintenance_work_mem=128MB
    -c max_connections=100
    -c log_min_duration_statement=500

redis:
  image: redis:7-alpine

```

```

restart: always
ports:
  - "127.0.0.1:6379:6379"    # Local only
volumes:
  - redisdata:/data
healthcheck:
  test: redis-cli ping
  interval: 10s
  timeout: 5s
  retries: 3
command: >
  redis-server
  --maxmemory 256mb
  --maxmemory-policy allkeys-lru
  --appendonly yes
deploy:
  resources:
    limits:
      cpus: "0.5"
      memory: 512M
volumes:
  pgdata:
  redisdata:

```

### 7.4.1 Production Dockerfile

```

dockerfile
# Dockerfile
FROM python:3.12-slim AS base

WORKDIR /code

# System dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Application code
COPY app/ /code/app/
COPY alembic/ /code/alembic/
COPY alembic.ini /code/

# Non-root user
RUN adduser --disabled-password --gecos "" appuser
USER appuser

EXPOSE 8000

```

```
# Health check endpoint
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

## 7.5 Database Operations

### 7.5.1 Migrations

```
bash
# Create a new migration after model changes
alembic revision --autogenerate -m "description_of_change"

# Apply migrations
alembic upgrade head

# Rollback last migration
alembic downgrade -1

# View migration history
alembic history
```

#### Migration rules:

- Every schema change goes through Alembic — no manual SQL in production
- Test migrations on a copy of production data before deploying
- Migrations must be backward-compatible when possible (add columns as nullable first)
- Include both `upgrade()` and `downgrade()` functions

### 7.5.2 Backup Strategy

```
bash
#!/bin/bash
# scripts/backup_db.sh – Run daily via cron

BACKUP_DIR="/backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/casino_${TIMESTAMP}.sql.gz"

# Dump and compress
docker compose -f docker-compose.prod.yml exec -T postgres \
  pg_dump -U ${POSTGRES_USER} -d ${POSTGRES_DB} \
  --format=custom --compress=9 \
  > "${BACKUP_FILE}"

# Keep only last 30 days
find ${BACKUP_DIR} -name "casino_*.sql.gz" -mtime +30 -delete

# Optional: upload to object storage (S3-compatible)
# rclone copy ${BACKUP_FILE} remote:casino-backups/
```

```
echo "Backup completed: ${BACKUP_FILE}"
```

### Cron schedule:

```
bash
# /etc/cron.d/casino-backup
0 3 * * * root /opt/casino/scripts/backup_db.sh >> /var/log/casino-backup.log 2>&1
```

## 7.5.3 Restore

```
bash
# Restore from backup
docker compose -f docker-compose.prod.yml exec -T postgres \
    pg_restore -U ${POSTGRES_USER} -d ${POSTGRES_DB} \
    --clean --if-exists \
    < /backups/casino_20260211_030000.sql.gz
```

## 7.6 Deployment Procedure

### 7.6.1 Initial Server Setup

```
bash
# 1. Server setup (Ubuntu 24.04)
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io docker-compose-v2 nginx certbot python3-certbot-nginx ufw

# 2. Firewall
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable

# 3. Create app directory
sudo mkdir -p /opt/casino
sudo chown $USER:$USER /opt/casino

# 4. Clone repo
cd /opt/casino
git clone https://github.com/yourrepo/casino-platform.git .

# 5. Environment file
cp .env.example .env.production
nano .env.production # Fill in production values

# 6. SSL certificate
sudo certbot --nginx -d api.yourdomain.com

# 7. Nginx config
sudo cp nginx/casino-api.conf /etc/nginx/sites-available/
sudo ln -s /etc/nginx/sites-available/casino-api.conf /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# 8. Start services
```

```

docker compose -f docker-compose.prod.yml up -d

# 9. Run migrations
docker compose -f docker-compose.prod.yml exec app alembic upgrade head

# 10. Seed initial data
docker compose -f docker-compose.prod.yml exec app python scripts/seed_admin.py
docker compose -f docker-compose.prod.yml exec app python scripts/seed_tables.py
docker compose -f docker-compose.prod.yml exec app python scripts/seed_packages.py

```

### 7.6.2 Update Deployment

```

bash
#!/bin/bash
# scripts/deploy.sh – Zero-downtime-ish deploy

set -e

cd /opt/casino

echo "=== Pulling latest code ==="
git pull origin main

echo "=== Building new image ==="
docker compose -f docker-compose.prod.yml build app celery

echo "=== Running migrations ==="
docker compose -f docker-compose.prod.yml exec app alembic upgrade head

echo "=== Restarting services ==="
docker compose -f docker-compose.prod.yml up -d --no-deps app celery celery-beat

echo "=== Health check ==="
sleep 5
curl -f http://localhost:8000/health || (echo "HEALTH CHECK FAILED" && exit 1)

echo "=== Deploy complete ==="

```

> **Note:** This causes a brief downtime (~5-10 seconds) during container restart. For true zero-downtime, use blue-green deployment with two app containers behind Nginx. This is a V2 improvement.

### 7.6.3 Rollback

```

bash
# 1. Revert code
git checkout <previous-commit>

# 2. Rebuild and restart
docker compose -f docker-compose.prod.yml build app celery
docker compose -f docker-compose.prod.yml up -d --no-deps app celery

# 3. If migration needs rollback

```

```
docker compose -f docker-compose.prod.yml exec app alembic downgrade -1
```

## 7.7 Monitoring & Logging

### 7.7.1 V1 Logging Strategy

```
python
# app/main.py – Structured logging
import logging
import json
from datetime import datetime

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_entry = {
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "level": record.levelname,
            "module": record.module,
            "message": record.getMessage(),
        }
        if hasattr(record, "player_id"):
            log_entry["player_id"] = record.player_id
        if hasattr(record, "table_id"):
            log_entry["table_id"] = record.table_id
        if record.exc_info:
            log_entry["exception"] = self.formatException(record.exc_info)
        return json.dumps(log_entry)

# Configure logging
handler = logging.StreamHandler()
handler.setFormatter(JSONFormatter())
logging.root.addHandler(handler)
logging.root.setLevel(logging.INFO)
```

#### Log levels:

| Level   | Usage                                              |
|---------|----------------------------------------------------|
| ERROR   | Unhandled exceptions, payment failures, DB errors  |
| WARNING | Failed logins, rate limit hits, invalid bets       |
| INFO    | Round completed, payment processed, player joined  |
| DEBUG   | Bet details, state transitions, WebSocket messages |

### 7.7.2 Log Rotation

```
bash
# /etc/logrotate.d/casino
/var/log/casino/*.log {
```

```

daily
rotate 14
compress
delaycompress
missingok
notifempty
create 0640 appuser appuser
}

```

### 7.7.3 Health Check Endpoint

```

python
# app/main.py
@app.get("/health")
async def health_check():
    """Basic health check for monitoring and load balancer."""
    checks = {}

    # Database
    try:
        async with get_db_session() as session:
            await session.execute(text("SELECT 1"))
        checks["database"] = "ok"
    except Exception:
        checks["database"] = "error"

    # Redis
    try:
        await redis.ping()
        checks["redis"] = "ok"
    except Exception:
        checks["redis"] = "error"

    all_ok = all(v == "ok" for v in checks.values())
    return JSONResponse(
        status_code=200 if all_ok else 503,
        content={"status": "healthy" if all_ok else "degraded", "checks": checks}
    )

```

### 7.7.4 Uptime Monitoring

| Tool              | Purpose                  | Cost      |
|-------------------|--------------------------|-----------|
| UptimeRobot       | Ping /health every 5 min | Free tier |
| Better Stack      | Structured log ingestion | Free tier |
| Sentry (optional) | Error tracking           | Free tier |



## 7.8 Security Hardening

### 7.8.1 Server-Level

```
bash
# SSH hardening
sudo nano /etc/ssh/sshd_config
# PermitRootLogin no
# PasswordAuthentication no (use SSH keys only)
# MaxAuthTries 3

# Fail2ban
sudo apt install fail2ban
sudo systemctl enable fail2ban
```

### 7.8.2 Application-Level

| Measure                        | Implementation                                               |
|--------------------------------|--------------------------------------------------------------|
| Input validation               | Pydantic V2 on every endpoint                                |
| SQL injection prevention       | SQLAlchemy ORM (parameterized queries only)                  |
| Password storage               | bcrypt with 12 rounds                                        |
| JWT secret rotation            | Planned for V2 (dual-key support)                            |
| CORS                           | Whitelist specific origins only                              |
| Rate limiting                  | Redis-based sliding window (Section 4.12)                    |
| Payment webhook HMAC           | SHA-256 signature verification                               |
| Sensitive data masking in logs | Phone: +237612* <b>678</b> , email: <b>pla*</b> @example.com |
| Environment variables          | Never in code, always .env                                   |
| Docker non-root user           | <b>USER</b> <b>appuser</b> in Dockerfile                     |

### 7.8.3 Database-Level

| Measure                       | Implementation                                          |
|-------------------------------|---------------------------------------------------------|
| Dedicated DB user per service | <b>casino_app</b> (limited), <b>casino_admin</b> (full) |
| Connection encryption         | <b>sslmode=require</b> in connection string             |
| Row-level locking             | <b>SELECT FOR UPDATE</b> on wallet operations           |
| CHECK constraints             | Balance $\geq 0$ , amounts $> 0$ , valid enums          |
| Audit trail                   | All admin actions logged to audit_log                   |

## 7.9 Scheduled Tasks (Celery Beat)

| Task                              | Schedule     | Description                 |
|-----------------------------------|--------------|-----------------------------|
| <b>reconcile_pending_payments</b> | Every 15 min | Check pending payments with |

| Task                               | Schedule       | Description                            |
|------------------------------------|----------------|----------------------------------------|
|                                    |                | provider API                           |
| <code>expire_stale_payments</code> | Every 5 min    | Mark expired payment orders as EXPIRED |
| <code>reset_spending_limits</code> | Daily at 00:00 | Reset daily spending counters          |
| <code>aggregate_daily_stats</code> | Daily at 01:00 | Compute daily KPIs for admin dashboard |
| <code>cleanup_old_sessions</code>  | Daily at 02:00 | Remove expired table_player_sessions   |
| <code>database_backup</code>       | Daily at 03:00 | Trigger pg_dump backup                 |

```
python
# app/platform/payments/tasks.py

from celery import Celery
from celery.schedules import crontab

celery_app = Celery("casino", broker=settings.REDIS_URL)

celery_app.conf.beat_schedule = {
    "reconcile-payments": {
        "task": "app.platform.payments.tasks.reconcile_pending_payments",
        "schedule": crontab(minute="*/15"),
    },
    "expire-stale-payments": {
        "task": "app.platform.payments.tasks.expire_stale_payments",
        "schedule": crontab(minute="*/5"),
    },
    "reset-spending-limits": {
        "task": "app.platform.responsible_gaming.tasks.reset_daily_limits",
        "schedule": crontab(hour=0, minute=0),
    },
    "aggregate-daily-stats": {
        "task": "app.platform.admin.tasks.aggregate_daily_stats",
        "schedule": crontab(hour=1, minute=0),
    },
    "cleanup-sessions": {
        "task": "app.platform.tables.tasks.cleanup_old_sessions",
        "schedule": crontab(hour=2, minute=0),
    },
}
```

*End of Section 7*

## Section 8 — Development Roadmap

### 8.1 Purpose of This Section

This section defines the **phased development plan** from zero to production, with estimated timelines, dependencies, and milestones. It is structured around **sprints of 2 weeks** for a **1-2 developer team**.

### 8.2 Development Phases Overview

|                                  |               |                                     |
|----------------------------------|---------------|-------------------------------------|
| Phase 0: Foundation              | (Weeks 1-2)   | – Project setup, DB, auth           |
| Phase 1: Platform Core           | (Weeks 3-6)   | – Wallet, payments, tables, admin   |
| Phase 2: Roulette Engine         | (Weeks 7-10)  | – Game logic, WebSocket, full round |
| Phase 3: Polish & Testing        | (Weeks 11-12) | – Integration tests, bug fixes, UX  |
| Phase 4: Deployment & Launch     | (Week 13-14)  | – Production deploy, soft launch    |
| Total V1: ~14 weeks (3.5 months) |               |                                     |
| Phase 5: Second Game             | (Weeks 15-20) | – Blackjack or Slots engine         |
| Phase 6: Growth Features         | (Weeks 21-24) | – Tournaments, chat, social         |

### 8.3 Phase 0 — Foundation (Weeks 1-2)

#### Sprint 0.1 (Week 1): Project Scaffolding

| Task                                          | Est. | Priority |
|-----------------------------------------------|------|----------|
| Initialize Git repo, Python project structure | 2h   | P0       |
| Set up Docker Compose (dev environment)       | 3h   | P0       |
| Configure FastAPI app factory (main.py)       | 2h   | P0       |
| Set up SQLAlchemy 2.0 async + Alembic         | 4h   | P0       |
| Create Pydantic BaseSettings (config.py)      | 2h   | P0       |
| Implement global exception handlers           | 2h   | P0       |
| Set up pytest + conftest.py with test DB      | 4h   | P0       |
| Create initial Alembic migration (all tables) | 6h   | P0       |
| Set up ruff + mypy + pre-commit hooks         | 2h   | P1       |

**Deliverable:** Running FastAPI app with empty routes, working DB, test suite runs.

## Sprint 0.2 (Week 2): Authentication

| Task                                          | Est. | Priority |
|-----------------------------------------------|------|----------|
| Player model + Wallet model (auto-created)    | 3h   | P0       |
| Registration endpoint + phone validation      | 4h   | P0       |
| OTP service (mock provider for dev)           | 4h   | P0       |
| OTP verification endpoint                     | 3h   | P0       |
| Login endpoint + JWT generation               | 4h   | P0       |
| Token refresh endpoint                        | 2h   | P0       |
| Password reset flow                           | 3h   | P1       |
| Auth middleware (get_current_user dependency) | 3h   | P0       |
| Auth unit + integration tests                 | 6h   | P0       |

**Deliverable:** Players can register, verify phone, login, get JWT, access protected endpoints.

**Milestone:**  M0 — Authentication working

## 8.4 Phase 1 — Platform Core (Weeks 3-6)

### Sprint 1.1 (Week 3): Wallet & Transactions

| Task                                          | Est. | Priority |
|-----------------------------------------------|------|----------|
| WalletService (debit, credit, get_balance)    | 6h   | P0       |
| Row-level locking for balance operations      | 4h   | P0       |
| WalletTransaction auto-creation on every op   | 3h   | P0       |
| Wallet REST endpoints (balance, transactions) | 4h   | P0       |
| Transaction filtering + pagination            | 3h   | P1       |
| Concurrency tests (race conditions)           | 6h   | P0       |
| Welcome bonus credit on registration          | 2h   | P1       |

**Deliverable:** Wallet works with guaranteed consistency. No overdrafts possible.

## Sprint 1.2 (Week 4): Payments

| Task                                      | Est. | Priority |
|-------------------------------------------|------|----------|
| ChipPackage model + seed data             | 2h   | P0       |
| PaymentOrder model + statuses             | 3h   | P0       |
| Abstract PaymentProvider interface        | 2h   | P0       |
| CinetPay adapter (API integration)        | 8h   | P0       |
| Payment initiation endpoint               | 4h   | P0       |
| Webhook endpoint + HMAC verification      | 6h   | P0       |
| Webhook → credit wallet (idempotent)      | 4h   | P0       |
| Celery tasks (reconciliation, expiry)     | 4h   | P1       |
| Payment integration tests (mock provider) | 6h   | P0       |

**Deliverable:** Players can buy chips via Mobile Money. Balance updates on webhook.

**Milestone:**  **M1 — Players can register and buy chips**

## Sprint 1.3 (Week 5): Tables & Admin Foundation

| Task                                           | Est. | Priority |
|------------------------------------------------|------|----------|
| GameTable model (with game_type + game_config) | 3h   | P0       |
| GameEngine ABC + PlatformServices interface    | 6h   | P0       |
| GameRegistry (register, get_or_create)         | 3h   | P0       |
| Table listing endpoint (player-facing)         | 3h   | P0       |
| AdminUser model + admin auth (separate JWT)    | 4h   | P0       |
| Admin table CRUD endpoints                     | 6h   | P0       |
| Admin package CRUD endpoints                   | 3h   | P1       |
| AuditLog on all admin actions                  | 3h   | P1       |
| Seed script for default tables                 | 2h   | P0       |

**Deliverable:** Admin can create/manage tables. Players can list tables. Game engine framework ready.

### Sprint 1.4 (Week 6): WebSocket Infrastructure

| Task                                  | Est. | Priority |
|---------------------------------------|------|----------|
| ConnectionManager implementation      | 4h   | P0       |
| WebSocket endpoint with JWT auth      | 4h   | P0       |
| Message protocol (envelope, router)   | 4h   | P0       |
| Message handler → game engine routing | 4h   | P0       |
| Ping/pong keepalive                   | 2h   | P0       |
| Connection close handling (cleanup)   | 3h   | P0       |
| WebSocket integration tests           | 6h   | P0       |
| Player profile endpoints (GET/PUT)    | 3h   | P1       |

**Deliverable:** WebSocket connects, authenticates, routes messages to game engines. Platform core complete.

**Milestone:**  M2 — Platform core complete, ready for game engines

## 8.5 Phase 2 — Roulette Engine (Weeks 7-10)

### Sprint 2.1 (Week 7): Wheel & Bet Validation

| Task                                         | Est. | Priority |
|----------------------------------------------|------|----------|
| RouletteEngine class implementing GameEngine | 4h   | P0       |
| Wheel constants (both variants)              | 2h   | P0       |
| WheelController (spin, get_color, neighbors) | 4h   | P0       |
| All bet validators (inside bets)             | 8h   | P0       |
| All bet validators (outside bets)            | 3h   | P0       |
| getCoveredNumbers for all bet types          | 4h   | P0       |
| Validator unit tests (full matrix)           | 8h   | P0       |
| RNG distribution test (chi-squared)          | 2h   | P0       |

**Deliverable:** Wheel spins correctly, all bet types validated, 100% test coverage on validators.

## Sprint 2.2 (Week 8): Bet Placement & State Machine

| Task                                        | Est. | Priority |
|---------------------------------------------|------|----------|
| RouletteRoundState (phase management)       | 4h   | P0       |
| RoundTimer (phase transitions with async)   | 4h   | P0       |
| BetManager (place_bet with full validation) | 6h   | P0       |
| BetManager (cancel_bet)                     | 3h   | P0       |
| Roulette DB models (bet, spin_result, etc.) | 3h   | P0       |
| Balance deduction on bet placement          | 3h   | P0       |
| Balance refund on bet cancellation          | 2h   | P0       |
| State machine unit tests                    | 4h   | P0       |
| Bet placement integration tests             | 6h   | P0       |

**Deliverable:** Players can place/cancel bets. State machine transitions correctly. Balance accurate.

## Sprint 2.3 (Week 9): Resolution & Results

| Task                                       | Est. | Priority |
|--------------------------------------------|------|----------|
| Resolver (win/loss for all bet types)      | 6h   | P0       |
| La Partage implementation                  | 2h   | P0       |
| Provably fair (hash before, reveal after)  | 4h   | P1       |
| Balance credit for winners                 | 3h   | P0       |
| GameRoundHistory creation on completion    | 3h   | P0       |
| WebSocket events (phase changes, results)  | 6h   | P0       |
| Personal result per player                 | 3h   | P0       |
| Resolution unit tests (full payout matrix) | 6h   | P0       |
| Full round integration test                | 4h   | P0       |

**Deliverable:** Complete game round works: bet → spin → resolve → payout. WebSocket events fire.

**Milestone:**  M3 — Roulette is playable end-to-end

## Sprint 2.4 (Week 10): Call Bets & Multiplayer

| Task                                            | Est. | Priority |
|-------------------------------------------------|------|----------|
| Call bet expansion (Voisins, Tiers, Orphelins)  | 6h   | P1       |
| Neighbors bet implementation                    | 3h   | P1       |
| Multiplayer: shared round with multiple players | 6h   | P0       |
| Multiplayer: per-table asyncio.Lock             | 3h   | P0       |
| Table player session management                 | 3h   | P0       |
| Player join/leave WebSocket events              | 2h   | P0       |
| Solo mode: player-triggered spin                | 3h   | P0       |
| Multiplayer integration tests                   | 6h   | P0       |

**Deliverable:** Both solo and multiplayer modes work. Call bets functional. Concurrency safe.

**Milestone:**  **M4 — Roulette feature-complete**

## 8.6 Phase 3 — Polish & Testing (Weeks 11-12)

### Sprint 3.1 (Week 11): History, Stats & Responsible Gaming

| Task                                          | Est. | Priority |
|-----------------------------------------------|------|----------|
| History endpoints (rounds, round detail)      | 4h   | P0       |
| Statistics endpoint (aggregated)              | 4h   | P1       |
| Hot/cold numbers calculation                  | 3h   | P1       |
| SpendingLimit enforcement on bet placement    | 4h   | P0       |
| Self-exclusion (block login + game access)    | 4h   | P0       |
| Responsible gaming endpoints                  | 3h   | P0       |
| Admin dashboard endpoint (KPIs)               | 4h   | P1       |
| Admin player management (ban, adjust balance) | 4h   | P1       |
| Admin audit log endpoint                      | 3h   | P1       |



## Sprint 3.2 (Week 12): Testing & Bug Fixes

| Task                                          | Est. | Priority |
|-----------------------------------------------|------|----------|
| Full integration test suite review            | 6h   | P0       |
| Security tests (injection, auth bypass, etc.) | 6h   | P0       |
| Edge case testing (disconnect, race cond.)    | 6h   | P0       |
| Load testing with locust (50 concurrent)      | 4h   | P1       |
| Bug fixes from testing                        | 8h   | P0       |
| API documentation review (matches Section 5)  | 4h   | P1       |
| Code review + refactoring                     | 6h   | P1       |

**Milestone:**  **M5 — All tests pass, production-ready**

## 8.7 Phase 4 — Deployment & Launch (Weeks 13-14)

### Sprint 4.1 (Week 13): Production Setup

| Task                                           | Est. | Priority |
|------------------------------------------------|------|----------|
| Provision VPS                                  | 2h   | P0       |
| Server hardening (firewall, SSH, fail2ban)     | 3h   | P0       |
| Nginx config + TLS certificate                 | 3h   | P0       |
| Production Docker Compose                      | 3h   | P0       |
| Production .env configuration                  | 2h   | P0       |
| Database migration on production               | 1h   | P0       |
| Seed production data (admin, tables, packages) | 2h   | P0       |
| CinetPay production API keys + webhook config  | 4h   | P0       |
| Twilio/SMS provider production config          | 3h   | P0       |
| Smoke test on production                       | 4h   | P0       |
| Backup cron setup                              | 2h   | P0       |
| Monitoring setup (UptimeRobot)                 | 1h   | P0       |

## Sprint 4.2 (Week 14): Soft Launch

| Task                                   | Est. | Priority |
|----------------------------------------|------|----------|
| Internal testing with real payments    | 8h   | P0       |
| Invite 10-20 beta testers              | -    | P0       |
| Monitor logs and fix issues            | 8h   | P0       |
| Performance monitoring under real load | 4h   | P0       |
| Adjust table configs based on feedback | 2h   | P1       |
| Deploy script finalization             | 3h   | P1       |
| Documentation final review             | 4h   | P1       |

**Milestone:**  **M6 — V1 LAUNCHED**

## 8.8 Phase 5 — Second Game (Weeks 15-20)

| Sprint | Focus                                        |
|--------|----------------------------------------------|
| 5.1    | Blackjack: game rules, card deck, hand logic |
| 5.2    | Blackjack: engine implementation, DB models  |
| 5.3    | Blackjack: WebSocket events, multiplayer     |
| 5.4    | Blackjack: testing, integration              |
| 5.5    | Polish, deploy, launch second game           |

> Estimated **5-6 weeks** for second game because the platform core, wallet, auth, payments, WebSocket infrastructure, and admin backoffice are already done. Only the game engine module is new.

## 8.9 Phase 6 — Growth Features (Weeks 21-24+)

| Feature       | Description                                   | Complexity |
|---------------|-----------------------------------------------|------------|
| Tournaments   | Scheduled events with leaderboards and prizes | High       |
| In-game chat  | WebSocket-based text chat per table           | Medium     |
| Friend system | Add friends, see online status                | Medium     |
| Daily bonuses | Login streak rewards                          | Low        |
| VIP program   | Tiered rewards based on wagering volume       | Medium     |
| Cashout (V2)  | Real money withdrawal with KYC                | Very High  |

| Feature            | Description                | Complexity |
|--------------------|----------------------------|------------|
| Push notifications | Game invites, bonus alerts | Medium     |
| Multi-language     | French/English UI          | Low        |

## 8.10 Milestone Summary

| Milestone | Description                   | Target Week | Dependencies |
|-----------|-------------------------------|-------------|--------------|
| M0        | Authentication working        | Week 2      | —            |
| M1        | Register + buy chips          | Week 4      | M0           |
| M2        | Platform core complete        | Week 6      | M1           |
| M3        | Roulette playable E2E         | Week 9      | M2           |
| M4        | Roulette feature-complete     | Week 10     | M3           |
| M5        | Production-ready (tests pass) | Week 12     | M4           |
| M6        | V1 Launched                   | Week 14     | M5           |
| M7        | Second game live              | Week 20     | M6           |

## 8.11 Risk Register

| Risk                                 | Probability | Impact   | Mitigation                                           |
|--------------------------------------|-------------|----------|------------------------------------------------------|
| CinetPay API issues or delays        | Medium      | High     | Implement Flutterwave adapter as fallback            |
| SMS delivery reliability in Cameroon | Medium      | Medium   | Support multiple OTP providers, add email backup     |
| Latency Europe → Cameroon            | Low         | Medium   | Choose Frankfurt/Paris DC, optimize payloads         |
| Wallet race conditions               | Low         | Critical | Row-level locking, extensive concurrency tests       |
| WebSocket drops on poor connections  | Medium      | Medium   | Auto-reconnect on client, bets persist server-side   |
| Game rule bugs (wrong payouts)       | Low         | Critical | 100% unit test coverage on resolver                  |
| Payment double-credit                | Low         | Critical | Idempotent webhooks, unique constraint on tx_id      |
| VPS outage                           | Low         | High     | Daily backups, monitoring alerts, documented restore |

| Risk | Probability | Impact | Mitigation |
|------|-------------|--------|------------|
|      |             |        | procedure  |

*End of Section 8*